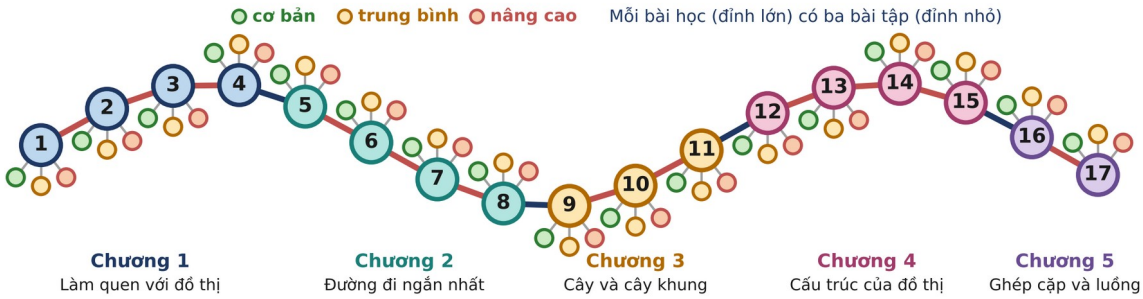
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

LÝ THUYẾT ĐỒ THỊ

Từ cơ bản đến nâng cao

TẬP 2 · BÀI TẬP



Chương 1. LÀM QUEN VỚI ĐỒ THỊ
Chương 2. ĐƯỜNG ĐI NGẮN NHẤT
Chương 3. CÂY VÀ CÂY KHUNG
Chương 4. CẤU TRÚC CỦA ĐỒ THỊ
Chương 5. GHÉP CẶP VÀ LUỒNG

51 bài tập · ràng buộc theo subtask · ví dụ có giải thích và hình minh họa

LỜI NÓI ĐẦU

Học thuật toán mà không làm bài tập cũng giống như học bơi trên cạn. Ở Tập 1, các em đã được dẫn từng bước từ ý tưởng ngây thơ tới thuật toán tốt, đã chạy tay trên những ví dụ nhỏ và hiểu **sao** mỗi thuật toán đúng. Tập 2 này là "bể bơi": nơi các em tự mình gặp đề bài mới, tự nhận ra đồ thị ẩn bên trong, tự chọn thuật toán và tự xử lí những chi tiết mà chỉ khi bắt tay vào code mới thấy.

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** gồm 17 bài học trong 5 chương.
- **Tập 2: Bài tập** (cuốn sách này) gồm **51 bài**: mỗi bài học của Tập 1 có đúng ba bài tập ở ba mức độ **cơ bản**, **trung bình** và **nâng cao**. Tên và ý tưởng các bài trùng với mục "Luyện tập" ở cuối mỗi bài học của Tập 1.
- **Tập 3: Hướng dẫn giải** phân tích từng bài từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh.

Mỗi bài tập được trình bày như một đề thi thật: có tên tệp, giới hạn thời gian và bộ nhớ, ràng buộc chia theo **subtask** để em biết với mỗi thuật toán mình nghĩ ra sẽ được bao nhiêu điểm. Mọi ví dụ đều có **giải thích** và phần lớn có **hình minh họa** vẽ cùng bảng màu với Tập 1, để em hình dung đề bài trước khi bắt tay vào code. Cuối mỗi bài là khung **CHÚ Ý** chỉ ra những bẫy thường gặp và khung **HƯỚNG SUY NGHĨ** gợi mở hướng đi mà không "lộ" lời giải.

Để đảm bảo độ tin cậy, mỗi bài tập đều có một chương trình giải mẫu; mọi kết quả trong các ví dụ được sinh ra bằng cách **chạy chính chương trình đó**, và chương trình giải mẫu lại được đối chiếu với một chương trình "vét cạn" trên hàng trăm bộ dữ liệu ngẫu nhiên, cũng như được chạy thử trên dữ liệu lớn nhất để kiểm tra thời gian.

Lời khuyên quan trọng nhất: **hãy tự làm trước khi đọc gợi ý**. Một bài tự nghĩ ra sau hai giờ có giá trị hơn mười bài đọc lời giải. Khi bế tắc thật sự, hãy đọc lại phần lí thuyết tương ứng ở Tập 1, rồi mới đọc khung "Hướng suy nghĩ", và chỉ mở Tập 3 khi đã thử hết sức.

Chúc các em luyện tập bền bỉ và đạt kết quả cao!

Lê Thành Nam

CÁCH SỬ DỤNG TẬP BÀI TẬP

Mỗi bài tập trong sách đều được trình bày theo cùng một cấu trúc, để em quen mắt và tìm thông tin thật nhanh:

Thành phần	Nội dung
Thanh tiêu đề	Mã bài (số bài học . thứ tự), tên bài và mức độ khó.
Dải thông tin	Tên tệp chương trình, tệp dữ liệu vào, tệp kết quả; giới hạn thời gian và bộ nhớ.
Kiến thức	Kiến thức cần dùng và bài học tương ứng ở Tập 1.
Đề bài	Câu chuyện và yêu cầu của bài toán.
Dữ liệu vào / Kết quả	Khuôn dạng chính xác của tệp vào và tệp ra.
Ràng buộc	Giới hạn chung và bảng **subtask**: mỗi subtask có thêm ràng buộc riêng và chiếm một tỉ lệ điểm.
Ví dụ	Một hoặc vài bộ dữ liệu mẫu, kèm cột giải thích và hình minh họa.
CHÚ Ý	Những bẫy làm mất điểm: tràn số, tràn ngăn xếp, trường hợp đặc biệt...
HƯỚNG SUY NGHĨ	Vài câu gợi mở để đọc khi đã thật sự bế tắc.

Ba mức độ của bài tập



Mức độ	Ý nghĩa
★☆☆ CƠ BẢN	Áp dụng gần như trực tiếp thuật toán của bài học; trọng tâm là cài đặt đúng, đủ nhanh, xử lí cẩn thận dữ liệu vào ra.
★★☆ TRUNG BÌNH	Cần thêm một quan sát, một bước biến đổi hoặc một chi tiết cài đặt quan trọng trước khi dùng được thuật toán.
★★★ NÂNG CAO	Kết hợp nhiều kĩ thuật, mô hình hóa khó, hoặc phải dựng lại đáp án; tương đương bài khó của kì thi học sinh giỏi cấp tỉnh, cấp quốc gia.

Subtask và chiến lược lấy điểm

Bảng subtask cho biết với ràng buộc nhỏ hơn em được bao nhiêu phần trăm số điểm. Chẳng hạn ở bài 1.3, chương trình thử mọi bộ ba người ($O(n^3)$) đã được 20% số điểm. Trong phòng thi, hãy luôn:

1) Viết lời giải ngây thơ cho subtask nhỏ nhất trước: vừa lấy điểm chắc chắn, vừa có công cụ để kiểm tra lời giải tốt.
2) Đọc kĩ các subtask ở giữa: chúng thường gợi ý từng bước tiến tới lời giải đầy đủ.
3) Khi làm xong lời giải tốt, đối chiếu với lời giải ngây thơ trên nhiều bộ dữ liệu nhỏ (xem Phụ lục B).

Quy ước chung

- Các đỉnh (thành phố, học sinh, máy tính...) được đánh số từ 1. Các số trên cùng một dòng cách nhau bởi dấu cách.
- Nếu không ghi khác, giới hạn thời gian là 1 giây và bộ nhớ là 256 MB cho mỗi test.
- Với bài có **nhiều đáp án đúng**, đề bài ghi rõ "in cách bất kì"; bài làm được chấm bằng chương trình kiểm tra (checker).
- Chương trình đọc và ghi theo kiểu "có tệp thì đọc tệp" như ở Tập 1, để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến:

BATTAY.CPP: khung chương trình dùng cho mọi bài trong sách

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Có tệp thì đọc tệp, không có thì đọc từ bàn phím
    if (fopen("BATTAY.INP", "r")) {
        freopen("BATTAY.INP", "r", stdin);
        freopen("BATTAY.OUT", "w", stdout);
    }
    int n;
    scanf("%d", &n);
    // ... phần giải bài toán ...
    return 0;
}
```

Quy ước màu trong hình vẽ

Hình minh họa dùng chung bảng màu với Tập 1:

Màu	Ý nghĩa
Đỉnh **tím**	Đỉnh xuất phát (nguồn).
Đỉnh **vàng** viền đỏ	Đỉnh cần chú ý: đích, đỉnh thuộc đáp án, đỉnh "quan trọng".
Đỉnh **xanh dương**, **xanh ngọc**, **cam**...	Phân biệt các nhóm, thành phần, phe...
Cạnh **đỏ**	Cạnh thuộc đáp án (đường đi, cây, bộ ghép, lát cắt...).
Cạnh **xanh đậm**	Cạnh của đồ thị hoặc của cây đang xét.
Cạnh **xám**	Cạnh không được dùng tới trong đáp án.
Cạnh **nét đứt**	Cạnh bị xóa, cạnh được thêm vào, hoặc quan hệ "cùng phe".
Ô vuông nhỏ cạnh đỉnh	Một giá trị gắn với đỉnh: khoảng cách, bậc, số vàng...

LỘ TRÌNH LUYỆN TẬP GỢI Ý

- **Vòng 1:** làm 17 bài ★☆☆ theo thứ tự các bài học, ngay sau khi học xong bài lí thuyết tương ứng.
- **Vòng 2:** làm 17 bài ★★☆; mỗi bài cố gắng tự tìm ra "bước biến đổi" trước khi đọc Hướng suy nghĩ.
- **Vòng 3:** làm 17 bài ★★★ như một kì thi thử: bấm giờ, tự lấy điểm từng subtask.
- Ghi lại tiến độ vào bảng ở Phụ lục C để biết mình còn yếu ở chương nào.

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH SỬ DỤNG TẬP BÀI TẬP **3**

BẢNG TỔNG HỢP 51 BÀI TẬP **7**

CHƯƠNG 1. LÀM QUEN VỚI ĐỒ THỊ **11**

Bài 1. Đồ thị là gì? (bài tập 1.1, 1.2, 1.3) 12
Bài 2. Biểu diễn đồ thị trong máy tính (bài tập 2.1, 2.2, 2.3) 19
Bài 3. Duyệt theo chiều sâu (DFS) (bài tập 3.1, 3.2, 3.3) 26
Bài 4. Duyệt theo chiều rộng (BFS) (bài tập 4.1, 4.2, 4.3) 32

CHƯƠNG 2. ĐƯỜNG ĐI NGẮN NHẤT **38**

Bài 5. Thuật toán Dijkstra (bài tập 5.1, 5.2, 5.3) 39
Bài 6. Thuật toán Bellman-Ford và chu trình âm (bài tập 6.1, 6.2, 6.3) 46
Bài 7. Thuật toán Floyd (bài tập 7.1, 7.2, 7.3) 53
Bài 8. Đồ thị trạng thái và BFS 0-1 (bài tập 8.1, 8.2, 8.3) 59

CHƯƠNG 3. CÂY VÀ CÂY KHUNG **66**

Bài 9. Cấu trúc các tập rời nhau (DSU) (bài tập 9.1, 9.2, 9.3) 67
Bài 10. Cây khung nhỏ nhất: Kruskal và Prim (bài tập 10.1, 10.2, 10.3) 74
Bài 11. Cây có gốc, tổ tiên chung gần nhất (LCA) (bài tập 11.1, 11.2, 11.3) 80

CHƯƠNG 4. CẤU TRÚC CỦA ĐỒ THỊ **86**

Bài 12. Sắp xếp tô pô và quy hoạch động trên DAG (bài tập 12.1, 12.2, 12.3) 87
Bài 13. Cầu và khớp (bài tập 13.1, 13.2, 13.3) 94
Bài 14. Thành phần liên thông mạnh (bài tập 14.1, 14.2, 14.3) 100
Bài 15. Chu trình và đường đi Euler (bài tập 15.1, 15.2, 15.3) 106

CHƯƠNG 5. GHÉP CẶP VÀ LUỒNG **113**

Bài 16. Đồ thị hai phía và bộ ghép cực đại (bài tập 16.1, 16.2, 16.3) 114
Bài 17. Luồng cực đại và lát cắt nhỏ nhất (bài tập 17.1, 17.2, 17.3) 120

PHỤ LỤC A. KIỂM TRA TRƯỚC KHI NỘP BÀI **127**

PHỤ LỤC B. TỰ KIỂM TRA BẰNG CÁCH ĐỐI CHIẾU **128**

PHỤ LỤC C. BẢNG THEO DÕI TIẾN ĐỘ **130**

BẢNG TỔNG HỢP 51 BÀI TẬP

Bảng dưới đây liệt kê toàn bộ các bài tập của sách, kèm tên tệp, mức độ, kiến thức chính và số trang, để em tiện chọn bài theo chủ đề hoặc theo mức độ.

Chương 1. LÀM QUEN VỚI ĐỒ THỊ

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
1.1	**BẮT TAY**	`BATTAY`	★☆☆ CƠ BẢN	Dùng tổng bậc để kiểm tra nhanh một dãy số liệu.	12
1.2	**NGƯỜI NỔI TIẾNG**	`NOITIENG`	★★☆ TRUNG BÌNH	Đếm bán bậc; xử lí dữ liệu bị lặp.	15
1.3	**TAM GIÁC BẠN BÈ**	`TAMGIAC`	★★★ NÂNG CAO	Đếm cấu trúc nhỏ trong đồ thị lớn; đánh giá độ phức tạp theo bậc.	17
2.1	**CHUYỂN ĐỔI**	`CHUYENDOI`	★☆☆ CƠ BẢN	Thành thạo cả ba cách biểu diễn đồ thị.	19
2.2	**BẠN CHUNG**	`BANCHUNG`	★★☆ TRUNG BÌNH	Chọn cách biểu diễn phù hợp với loại truy vấn.	22
2.3	**BẢN ĐỒ KHO BÁU**	`BANDO`	★★★ NÂNG CAO	Nhận ra mỗi thay đổi chỉ ảnh hưởng tới vài ô xung quanh.	24
3.1	**ĐẾM ĐẢO**	`DEMDAO`	★☆☆ CƠ BẢN	Loang trên lưới không đệ quy.	26
3.2	**CÂY HAY KHÔNG**	`CAYKHONG`	★★☆ TRUNG BÌNH	Đưa bài toán tối ưu về đếm số thành phần liên thông.	28
3.3	**MẠNG LƯỚI DỰ PHÒNG**	`DUPHONG`	★★★ NÂNG CAO	Đếm theo thành phần; lựa chọn tối ưu từ kích thước các thành phần.	30
4.1	**MÃ ĐI TUẦN**	`MADITUAN`	★☆☆ CƠ BẢN	Một lần BFS trả lời mọi câu hỏi có chung điểm xuất phát.	32
4.2	**CHẠY TRỐN ĐÁM CHÁY**	`CHAYTRON`	★★☆ TRUNG BÌNH	Tách bài toán "hai đối tượng cùng di chuyển" thành hai lần BFS.	34
4.3	**CHIA ĐỘI**	`CHIADOI`	★★★ NÂNG CAO	Kết hợp tô màu hai phía với bài toán chia tập.	36

Chương 2. ĐƯỜNG ĐI NGẮN NHẤT

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
5.1	**ĐƯỜNG TỚI TRƯỜNG**	`DUONGTRUONG`	★☆☆ CƠ BẢN	Cài đặt Dijkstra chuẩn với hàng đợi ưu tiên.	39
5.2	**NÚT GIAO QUAN TRỌNG**	`NUTGIAO`	★★☆ TRUNG BÌNH	Phân tích cấu trúc của tập mọi đường đi ngắn nhất.	42

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
5.3	**XE BUÝT ĐÊM**	`XEBUYT`	★★★ NÂNG CAO	Nhận ra khi nào Dijkstra vẫn đúng với trọng số phụ thuộc thời gian.	44
6.1	**ĐỔI TIỀN**	`DOITIEN`	★☆☆ CƠ BẢN	Mô hình hóa: biến "tích lớn hơn 1" thành "tổng âm".	46
6.2	**CHUYẾN BAY GIÁ RẺ**	`BAYRE`	★★☆ TRUNG BÌNH	Hiểu ý nghĩa "sau lượt thứ i" của Bellman-Ford.	49
6.3	**LỊCH THI ĐẤU**	`LICHTHIDAU`	★★★ NÂNG CAO	Biến hệ bất phương trình thành đồ thị.	51
7.1	**THỦ ĐÔ MỚI**	`THUDO`	★☆☆ CƠ BẢN	Cài đặt Floyd; xử lí trường hợp bằng nhau.	53
7.2	**XẾP HẠNG KÌ THỦ**	`XEPHANG`	★★☆ TRUNG BÌNH	Lập luận trên quan hệ "mạnh hơn" có tính bắc cầu.	55
7.3	**XE TẢI QUÁ KHỔ**	`XETAI`	★★★ NÂNG CAO	Tổng quát hóa khung Floyd cho một "phép đo" khác.	57
8.1	**ĐÈN GIAO THÔNG**	`DENGIAOTHONG`	★☆☆ CƠ BẢN	Thêm thời gian (theo chu kì) vào trạng thái.	59
8.2	**PHÁ TƯỜNG**	`PHATUONG`	★★☆ TRUNG BÌNH	Thêm "tài nguyên đã dùng" vào trạng thái.	62
8.3	**ROBOT XOAY**	`ROBOTXOAY`	★★★ NÂNG CAO	Đồ thị có trọng số 0 và 1: dùng hàng đợi hai đầu.	64

Chương 3. CÂY VÀ CÂY KHUNG

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
9.1	**MẠNG BẠN BÈ**	`MANGBANBE`	★☆☆ CƠ BẢN	Cài đặt DSU chuẩn, duy trì thông tin theo từng bước.	67
9.2	**PHÁ CẦU**	`PHACAU`	★★☆ TRUNG BÌNH	Đảo ngược thứ tự thao tác để biến "xóa" thành "thêm".	70
9.3	**HAI PHE**	`HAIPHE`	★★★ NÂNG CAO	Gắn thông tin quan hệ vào cạnh của cây DSU.	72
10.1	**NỐI MẠNG TRƯỜNG HỌC**	`NOIMANG`	★☆☆ CƠ BẢN	Khởi tạo DSU từ hiện trạng trước khi chạy Kruskal.	74
10.2	**ĐƯỜNG ÍT DỐC**	`ITDOC`	★★☆ TRUNG BÌNH	Biến câu hỏi trên đồ thị thành câu hỏi trên cây khung.	76
10.3	**GIẾNG VÀ ỐNG**	`GIENGONG`	★★★ NÂNG CAO	Mô hình hóa lựa chọn "tự làm" bằng một đỉnh ảo.	78
11.1	**HỌ HÀNG**	`HOHANG`	★☆☆ CƠ BẢN	Cài đặt LCA chuẩn; tìm gốc của cây.	80

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
11.2	**CÁP QUANG YẾU NHẤT**	`CAPQUANG`	★★☆ TRUNG BÌNH	Gắn thêm thông tin tổng hợp vào bảng nhảy nhị phân.	82
11.3	**NHÂN VIÊN CẤP K**	`CAPK`	★★★ NÂNG CAO	Đổi cách trả lời câu hỏi khi bộ nhớ không đủ cho bảng nhảy nhị phân.	84

Chương 4. CẤU TRÚC CỦA ĐỒ THỊ

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
12.1	**XẾP HẠNG GIẢI ĐẤU**	`XHGIAIDAU`	★☆☆ CƠ BẢN	Đọc thêm thông tin từ quá trình chạy Kahn.	87
12.2	**DỰ ÁN XÂY NHÀ**	`XAYNHA`	★★☆ TRUNG BÌNH	Quy hoạch động hai chiều: xuôi và ngược thứ tự tô pô.	90
12.3	**ĐẾM LỘ TRÌNH**	`DEMLOTRINH`	★★★ NÂNG CAO	Chỉ giữ lại phần đồ thị "có ích" trước khi quy hoạch động.	92
13.1	**ĐƯỜNG HUYẾT MẠCH**	`HUYETMACH`	★☆☆ CƠ BẢN	Cài đặt tìm cầu không đệ quy, đúng với cạnh lặp.	94
13.2	**TRẠM BIẾN ÁP**	`BIENAP`	★★☆ TRUNG BÌNH	Không chỉ tìm khớp mà còn đo mức độ "quan trọng" của khớp.	96
13.3	**CÁP DỰ PHÒNG**	`CAPDUPHONG`	★★★ NÂNG CAO	Nén đồ thị thành cây rồi lập luận trên cây.	98
14.1	**LAN TRUYỀN TIN ĐỒN**	`TINDON`	★☆☆ CƠ BẢN	Đếm thành phần nguồn trên đồ thị nén.	100
14.2	**KHO BÁU THỊ TRẤN**	`KHOBAU`	★★☆ TRUNG BÌNH	Kết hợp TPLTM với quy hoạch động.	102
14.3	**HỘI ĐỒNG TUYỂN CHỌN**	`TUYENCHON`	★★★ NÂNG CAO	Mô hình hóa ràng buộc logic thành đồ thị kéo theo.	104
15.1	**NÉT VẼ**	`NETVE`	★☆☆ CƠ BẢN	Kiểm tra cả hai điều kiện: bậc và tính liên thông.	106
15.2	**GHÉP DOMINO**	`DOMINO`	★★☆ TRUNG BÌNH	Mô hình hóa: mỗi đối tượng là một cạnh, không phải một đỉnh.	109
15.3	**MẬT KHẨU KÉT SẮT**	`KETSAT`	★★★ NÂNG CAO	Xây đồ thị trừu tượng trên các xâu.	111

Chương 5. GHÉP CẶP VÀ LUỒNG

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
16.1	**XẾP PHÒNG THI**	`XEPPHONG`	★☆☆ CƠ BẢN	Cài đặt Kuhn chuẩn.	114
16.2	**ĐẶT DOMINO**	`DATDOMINO`	★★☆ TRUNG BÌNH	Nhận ra cấu trúc hai phía ẩn trong lưới ô.	116

Bài	Tên bài	Tệp	Mức độ	Kĩ năng rèn luyện	Trang
16.3	**CAMERA GIÁM SÁT**	`CAMERA`	★★★ NÂNG CAO	Không chỉ tính kích thước mà còn dựng được đáp án.	118
17.1	**ĐƯỜNG ỐNG**	`DUONGONG`	★☆☆ CƠ BẢN	Cài đặt luồng cực đại và trích ra lát cắt nhỏ nhất.	120
17.2	**LỘ TRÌNH RỜI NHAU**	`ROINHAU`	★★☆ TRUNG BÌNH	Không chỉ đếm mà còn truy vết các đường đi từ luồng.	123
17.3	**PHÂN CÔNG TRỰC NHẬT**	`TRUCNHAT`	★★★ NÂNG CAO	Dựng mạng luồng từ các giới hạn "nhiều nhất" và "đúng bằng".	125

CHƯƠNG 1. LÀM QUEN VỚI ĐỒ THỊ

Các bài tập của chương này rèn luyện kĩ năng nền tảng nhất: nhìn ra đồ thị ẩn trong đề bài, cất đồ thị vào máy tính một cách hợp lí, và duyệt đồ thị bằng DFS, BFS. Đừng xem nhẹ chúng: rất nhiều bài khó ở các chương sau, khi bóc hết lớp vỏ, chỉ còn lại một lần duyệt đồ thị được đặt đúng chỗ.

TRONG CHƯƠNG NÀY

Bài 1. Đồ thị là gì?

- 1.1 BẮT TAY — ★☆☆ CƠ BẢN
- 1.2 NGƯỜI NỔI TIẾNG — ★★☆ TRUNG BÌNH
- 1.3 TAM GIÁC BẠN BÈ — ★★★ NÂNG CAO

Bài 2. Biểu diễn đồ thị trong máy tính

- 2.1 CHUYỂN ĐỔI — ★☆☆ CƠ BẢN
- 2.2 BẠN CHUNG — ★★☆ TRUNG BÌNH
- 2.3 BẢN ĐỒ KHO BÁU — ★★★ NÂNG CAO

Bài 3. Duyệt theo chiều sâu (DFS)

- 3.1 ĐẾM ĐẢO — ★☆☆ CƠ BẢN
- 3.2 CÂY HAY KHÔNG — ★★☆ TRUNG BÌNH
- 3.3 MẠNG LƯỚI DỰ PHÒNG — ★★★ NÂNG CAO

Bài 4. Duyệt theo chiều rộng (BFS)

- 4.1 MÃ ĐI TUẦN — ★☆☆ CƠ BẢN
- 4.2 CHẠY TRỐN ĐÁM CHÁY — ★★☆ TRUNG BÌNH
- 4.3 CHIA ĐỘI — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi viết code, luôn trả lời bốn câu hỏi: **Đỉnh là gì? Cạnh là gì? Có hướng không? Có trọng số không?**
- Với lưới R × C, đừng vội dựng danh sách kề: hãy dùng hai mảng dx, dy để đi sang ô kề.
- Đồ thị tới 10^5–10^6 đỉnh có thể làm DFS đệ quy bị tràn ngăn xếp. Hãy tập viết DFS bằng ngăn xếp tự quản lí hoặc dùng BFS.

BÀI 1 Đồ thị là gì?

ÔN NHANH KIẾN THỨC · Tập 1, Bài 1

- **Định lí bắt tay:** Trong mọi đồ thị vô hướng, deg(1) + deg(2) + ... + deg(n) = 2m. Hệ quả: số đỉnh bậc lẻ luôn là số chẵn.
- Đồ thị có hướng: mỗi cung u → v góp 1 vào **bán bậc ra** của u và 1 vào **bán bậc vào** của v; tổng bán bậc ra = tổng bán bậc vào = m.
- Tính bậc mọi đỉnh trong O(n + m): duyệt từng **cạnh** một lần, cộng 1 cho cả hai đầu mút, thay vì với mỗi đỉnh lại duyệt toàn bộ danh sách cạnh.
- Khi tổng có thể vượt $2 \cdot 10^9$, hãy dùng kiểu `long long`.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
1.1	**BẮT TAY**	★☆☆ CƠ BẢN	Dùng tổng bậc để kiểm tra nhanh một dãy số liệu.
1.2	**NGƯỜI NỔI TIẾNG**	★★☆ TRUNG BÌNH	Đếm bán bậc; xử lí dữ liệu bị lặp.
1.3	**TAM GIÁC BẠN BÈ**	★★★ NÂNG CAO	Đếm cấu trúc nhỏ trong đồ thị lớn; đánh giá độ phức tạp theo bậc.

BÀI 1.1 BẮT TAY ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`BATTAY.CPP`	`BATTAY.INP`	`BATTAY.OUT`	1 giây · 256 MB

Kiến thức: *Định lí bắt tay, bậc của đỉnh (Tập 1, Bài 1).*

Cuối buổi giao lưu của câu lạc bộ Tin học có n bạn tham gia, đánh số từ 1 tới n. Ban tổ chức hỏi từng bạn: *"Hôm nay em đã bắt tay bao nhiêu lần?"* và ghi lại câu trả lời: bạn thứ i khai là đã bắt tay d_i lần.

Mỗi lần bắt tay luôn diễn ra giữa **hai bạn khác nhau**, và được tính là một lần cho mỗi bạn trong cặp. Hai bạn có thể bắt tay với nhau **nhiều lần** (mỗi lần đều được tính).

Thầy phụ trách muốn biết các câu trả lời có thể cùng đúng hay không. Nếu có thể, hãy cho biết trong buổi giao lưu đã có tất cả bao nhiêu cái bắt tay; nếu không thể, nghĩa là chắc chắn có bạn khai sai.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ hai chứa n số nguyên $d_1, d_2, ..., d_n$.

Kết quả

Một số nguyên duy nhất là tổng số cái bắt tay, hoặc −1 nếu các câu trả lời không thể cùng đúng.

Ràng buộc

$1 \le n \le 10^5$; $0 \le d_i \le 10^9$.

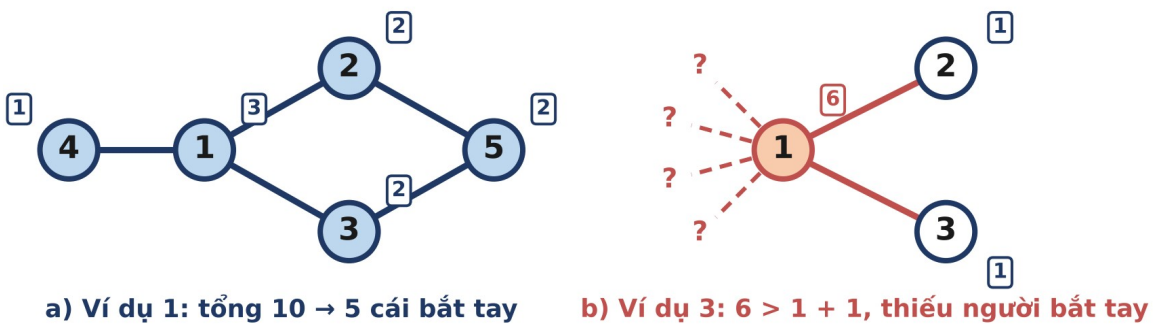
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 100$, $d_i \le 100$ và mỗi d_i không vượt quá tổng các số còn lại.
2	30%	$n \le 100$, $d_i \le 100$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

BATTAY.INP	BATTAY.OUT	Giải thích
5 3 2 2 1 2	5	Tổng các số là 10, nên có 10 / 2 = **5** cái bắt tay. Hình 1.1a là một cách bắt tay thỏa mãn.

BATTAY.INP	BATTAY.OUT	Giải thích
4 3 3 1 2	-1	Tổng các số là 9, là số **lẻ**. Theo định lí bắt tay, tổng phải chẵn, nên chắc chắn có bạn khai sai.

BATTAY.INP	BATTAY.OUT	Giải thích
3 6 1 1	-1	Tổng bằng 8 là số chẵn, nhưng bạn 1 khai 6 lần trong khi hai bạn còn lại chỉ có tổng cộng 2 lần để bắt tay với bạn 1 (Hình 1.1b).



Hình 1.1. *Mỗi bạn là một đỉnh, mỗi lần bắt tay là một cạnh; số trong ô vuông là bậc của đỉnh.*

CHÚ Ý

Với $n = 10^5$ và $d_i = 10^9$, tổng có thể lên tới 10^{14}, vượt quá phạm vi của kiểu `int`.

Tổng chẵn là **chưa đủ** (xem ví dụ 3). Hãy suy nghĩ: một bạn bắt tay nhiều lần nhất thì những cái bắt tay ấy phải "tiêu" vào ai?

HƯỚNG SUY NGHĨ

- Đặt $S = d_1 + ... + d_n$ và M là số lớn nhất. Điều kiện nào về S là bắt buộc? Điều kiện nào về M và $S - M$ là bắt buộc?
- Thử tự thuyết phục rằng hai điều kiện ấy cũng là **đủ**: luôn cho bạn có nhiều lượt bắt tay còn lại nhất bắt tay với bạn có nhiều lượt thứ hai.

BÀI 1.2	NGƯỜI NỔI TIẾNG	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NOITIENG.CPP`	`NOITIENG.INP`	`NOITIENG.OUT`	1 giây · 256 MB

Kiến thức: *Đồ thị có hướng, bán bậc vào và bán bậc ra (Tập 1, Bài 1).*

Trên mạng xã hội của trường có n tài khoản, đánh số từ 1 tới n. Tài khoản u có thể **theo dõi** tài khoản v ($u \neq v$); quan hệ này có hướng: u theo dõi v không có nghĩa là v theo dõi u.

Một tài khoản được gọi là **người nổi tiếng** nếu nó được **mọi** tài khoản khác theo dõi, nhưng bản thân nó **không theo dõi ai**. Dễ thấy có nhiều nhất một người nổi tiếng (vì sao?).

Dữ liệu được trích xuất từ nhật kí hệ thống gồm m dòng, mỗi dòng "u v" cho biết u theo dõi v. Do hệ thống ghi nhật kí không cẩn thận, **một quan hệ có thể bị ghi nhiều lần**. Hãy tìm người nổi tiếng.

Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên u, v ($1 \le u, v \le n, u \neq v$): tài khoản u theo dõi tài khoản v.

Kết quả

Số hiệu của người nổi tiếng, hoặc -1 nếu không có.

Ràng buộc

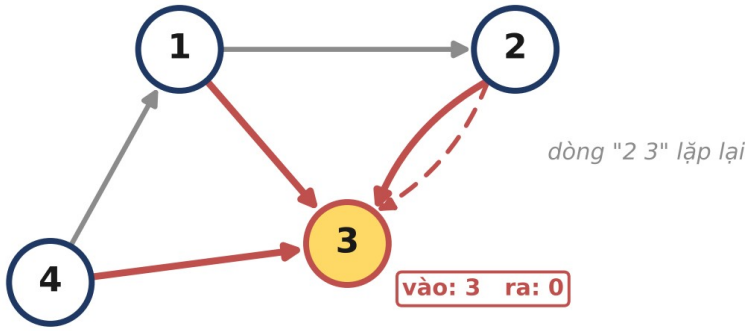
$2 \le n \le 2 \cdot 10^5$; $0 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 1000$, không có dòng nào bị lặp.
2	30%	Không có dòng nào bị lặp.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NOITIENG.INP	NOITIENG.OUT	Giải thích
4 6 1 3 2 3 4 3 1 2 4 1 2 3	3	Tài khoản 3 được 1, 2, 4 theo dõi và không theo dõi ai. Dòng "2 3" xuất hiện hai lần nhưng chỉ là **một** quan hệ.

NOITIENG.INP	NOITIENG.OUT	Giải thích
3 3 1 2 3 2 2 1	-1	Tài khoản 2 được 1 và 3 theo dõi, nhưng lại theo dõi 1 nên không phải người nổi tiếng. Không có ai khác thỏa mãn.



Hình 1.2. *Ví dụ 1: đỉnh 3 có bán bậc vào bằng n − 1 = 3 và bán bậc ra bằng 0.*

CHÚ Ý

Nếu chỉ đếm bán bậc vào mà không loại các dòng lặp, ở ví dụ 1 đỉnh 3 sẽ có "bán bậc vào" bằng $4 \neq n - 1$ và em sẽ trả lời sai. Cách gọn nhất: sắp xếp các cặp (u, v) rồi bỏ các cặp trùng nhau.

HƯỚNG SUY NGHĨ

- Người nổi tiếng chính là đỉnh có bán bậc vào bằng $n - 1$ và bán bậc ra bằng 0 trong đồ thị **đã loại cung lặp**.
- Vì sao không thể có hai người nổi tiếng? Hãy xét xem người này có theo dõi người kia không.

BÀI 1.3	TAM GIÁC BẠN BÈ	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TAMGIAC.CPP`	`TAMGIAC.INP`	`TAMGIAC.OUT`	2 giây · 256 MB

Kiến thức: *Bậc của đỉnh, duyệt theo cạnh, định hướng cạnh theo bậc (Tập 1, Bài 1–2).*

Một lớp học có n học sinh và m cặp bạn thân. Quan hệ bạn thân là hai chiều. Ba học sinh tạo thành một **tam giác bạn bè** nếu ba người **đôi một** là bạn thân của nhau.

Cô giáo muốn biết trong lớp có bao nhiêu tam giác bạn bè (hai tam giác khác nhau nếu tập ba người khác nhau).

Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên u, v ($1 \le u, v \le n$, $u \ne v$) cho biết u và v là bạn thân. Không có cặp nào được cho hai lần.

Kết quả

Một số nguyên duy nhất là số tam giác bạn bè.

Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 100$.
2	30%	$n \le 2000$.
3	50%	Không có ràng buộc gì thêm.

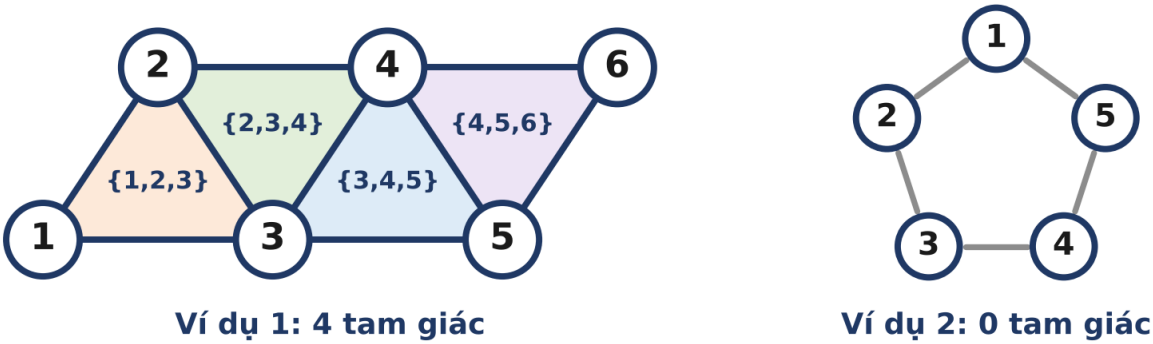
Ví dụ

TAMGIAC.INP	TAMGIAC.OUT	Giải thích

`6 9`
`1 2`
`1 3`
`2 3`
`2 4`
`3 4`
`3 5`
`4 5`
`4 6`
`5 6`
	`4`	Bốn tam giác: {1, 2, 3}, {2, 3, 4}, {3, 4, 5} và {4, 5, 6} (Hình 1.3, bên trái).

TAMGIAC.INP	TAMGIAC.OUT	Giải thích

`5 5`
`1 2`
`2 3`
`3 4`
`4 5`
`5 1`
	`0`	Năm bạn xếp thành một vòng tròn, không có ba bạn nào đôi một quen nhau.



Hình 1.3. *Mỗi tam giác bạn bè là một chu trình độ dài 3.*

CHÚ Ý

Số tam giác có thể rất lớn: chỉ cần khoảng 630 bạn đôi một quen nhau ($m \approx 2 \cdot 10^5$) đã có hơn $4 \cdot 10^7$ tam giác. Không thể liệt kê từng tam giác bằng ba vòng lặp lồng nhau; hãy dùng `long long` cho biến đếm để yên tâm.

Cách "với mỗi cạnh (u, v), duyệt mọi bạn của u và kiểm tra có là bạn của v không" có thể rất chậm nếu có một học sinh quen gần như cả lớp.

HƯỚNG SUY NGHĨ

- Subtask 1: thử mọi bộ ba. Subtask 2: dùng ma trận kề, với mỗi cạnh (u, v) đếm số w kề cả u lẫn v, rồi chia 3.
- Subtask 3: hướng mỗi cạnh từ đỉnh có bậc nhỏ hơn sang đỉnh có bậc lớn hơn (bằng nhau thì so số hiệu). Khi đó mỗi đỉnh có không quá khoảng $\sqrt{(2m)}$ cung đi ra. Mỗi tam giác được đếm đúng một lần ở đỉnh "thấp" nhất của nó.

BÀI 2 Biểu diễn đồ thị trong máy tính

ÔN NHANH KIẾN THỨC · Tập 1, Bài 2

- **Ma trận kề** a[u][v]: kiểm tra hai đỉnh có kề nhau trong O(1), nhưng tốn O(n^2) bộ nhớ và O(n) để liệt kê đỉnh kề. Hợp với n vài nghìn.
- **Danh sách cạnh**: lưu m cặp (u, v), tiện cho sắp xếp cạnh (Kruskal) và lặp qua mọi cạnh (Bellman-Ford).
- **Danh sách kề** `vector<int> g[N]`: tổng kích thước 2m với đồ thị vô hướng, liệt kê đỉnh kề của u trong O(deg(u)). Đây là cách mặc định.
- **Lưới R × C**: không cần dựng đồ thị; ô (x, y) đi sang (x + dx[k], y + dy[k]) với k = 0..3, nhớ kiểm tra còn trong bảng.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
2.1	**CHUYỂN ĐỔI**	★☆☆ CƠ BẢN	Thành thạo cả ba cách biểu diễn đồ thị.
2.2	**BẠN CHUNG**	★★☆ TRUNG BÌNH	Chọn cách biểu diễn phù hợp với loại truy vấn.
2.3	**BẢN ĐỒ KHO BÁU**	★★★ NÂNG CAO	Nhận ra mỗi thay đổi chỉ ảnh hưởng tới vài ô xung quanh.

BÀI 2.1 CHUYỂN ĐỔI ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CHUYENDOI.CPP`	`CHUYENDOI.INP`	`CHUYENDOI.OUT`	1 giây · 256 MB

Kiến thức: *Ma trận kề, danh sách cạnh, danh sách kề (Tập 1, Bài 2).*

Phần mềm vẽ sơ đồ mạng của trường xuất dữ liệu dưới dạng **ma trận kề**: mạng có n máy, đánh số từ 1 tới n; phần tử ở hàng i, cột j bằng 1 nếu máy i và máy j được nối trực tiếp bằng một sợi cáp, bằng 0 nếu không. Ma trận đối xứng và các phần tử trên đường chéo chính đều bằng 0.

Để nạp vào một phần mềm khác, em cần chuyển dữ liệu sang hai dạng: **danh sách cạnh** và **danh sách kề**.

Dữ liệu vào

Dòng đầu chứa số nguyên n. Dòng thứ i trong n dòng tiếp theo chứa n số 0 hoặc 1, là hàng thứ i của ma trận kề.

Kết quả

- Dòng đầu: số cạnh m.
- m dòng tiếp theo: mỗi dòng hai số u < v mô tả một cạnh. Các cạnh được in theo thứ tự tăng dần của u, cùng u thì tăng dần của v.
- n dòng cuối: dòng thứ i bắt đầu bằng deg(i), tiếp theo là các đỉnh kề với i theo thứ tự tăng dần.

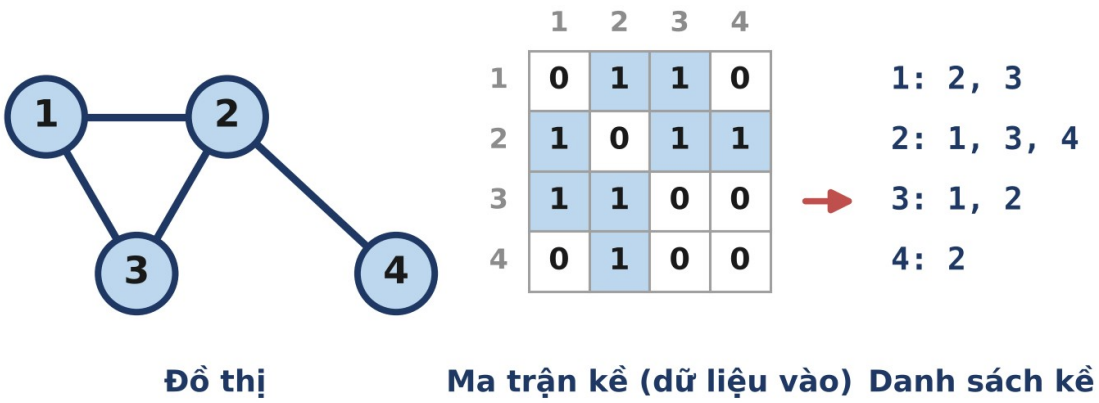
Ràng buộc

$1 \le n \le 1000$.

Subtask	Điểm	Ràng buộc thêm
1	40%	$n \le 10$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

CHUYENDOI.INP	CHUYENDOI.OUT	Giải thích
4 0 1 1 0 1 0 1 1 1 1 0 0 0 1 0 0	4 1 2 1 3 2 3 2 4 2 2 3 3 1 3 4 2 1 2 1 2	Có 4 cạnh: (1, 2), (1, 3), (2, 3), (2, 4). Đỉnh 2 kề với 1, 3, 4 nên dòng của đỉnh 2 là "3 1 3 4". Hình 2.1 vẽ cả ba cách biểu diễn.



Hình 2.1. *Cùng một đồ thị, ba cách cất vào máy tính.*

CHÚ Ý

Với n = 1000, dữ liệu vào có 10^6 số. Dùng `scanf` hoặc `ios::sync_with_stdio(false)` để đọc cho nhanh.

Chỉ in cạnh khi i < j, nếu không mỗi cạnh sẽ bị in hai lần (vì ma trận đối xứng).

HƯỚNG SUY NGHĨ

- Duyệt i từ 1 tới n, j từ 1 tới n theo đúng thứ tự: các cạnh (i, j) với i < j tự nhiên xuất hiện theo thứ tự yêu cầu, và danh sách kề của i tự nhiên tăng dần. Không cần sắp xếp gì cả!

BÀI 2.2	BẠN CHUNG	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BANCHUNG.CPP	**BANCHUNG.INP**	**BANCHUNG.OUT**	**1 giây · 256 MB**

Kiến thức: *Danh sách kề đã sắp xếp, tìm kiếm nhị phân, phân tích theo bậc (Tập 1, Bài 2).*

Mạng xã hội của trường có n học sinh và m quan hệ bạn bè (hai chiều). Khi bạn u xem trang cá nhân của bạn v, mạng xã hội hiện dòng chữ *"Hai bạn có k bạn chung"*.

Có q lượt xem trang. Với mỗi lượt xem (u, v), hãy cho biết số bạn chung của u và v, tức là số học sinh w là bạn của cả u lẫn v.

Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, q. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$) cho biết u và v là bạn; không có cặp nào được cho hai lần. Mỗi dòng trong q dòng cuối chứa hai số u, v ($u \neq v$) là một lượt xem.

Kết quả

Gồm q dòng, dòng thứ i là số bạn chung của lượt xem thứ i.

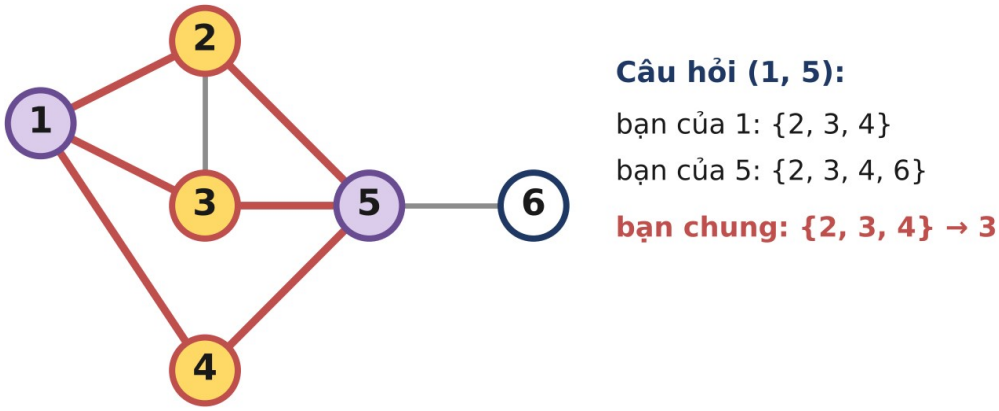
Ràng buộc

$2 \le n \le 10^5$; $0 \le m \le 10^5$; $1 \le q \le 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 500$.
2	30%	Mỗi học sinh có không quá 300 bạn.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

BANCHUNG.INP	BANCHUNG.OUT	Giải thích
6 8 4 1 2 1 3 1 4 2 3 2 5 3 5 4 5 5 6 1 5 2 3 1 6 4 2	3 2 0 2	(1, 5): bạn chung là 2, 3, 4 (Hình 2.2). (2, 3): bạn chung là 1 và 5. (1, 6): bạn duy nhất của 6 là 5, không phải bạn của 1. (4, 2): bạn chung là 1 và 5.



Hình 2.2. *Lượt xem (1, 5): ba bạn chung được tô vàng.*

CHÚ Ý

Subtask 3 có thể có hai "người nổi tiếng" cùng có $5 \cdot 10^4$ bạn, và cả 10^5 lượt xem đều hỏi về đúng cặp này. Duyệt lại từ đầu mỗi lần sẽ tốn $5 \cdot 10^9$ phép tính. Hãy nghĩ cách để một cặp được hỏi lại không phải tính lại.

HƯỚNG SUY NGHĨ

- Subtask 1: ma trận kề, mỗi câu hỏi duyệt w từ 1 tới n.
- Sắp xếp danh sách kề của từng đỉnh. Với câu hỏi (u, v), duyệt danh sách **ngắn hơn** và tìm nhị phân trong danh sách còn lại: O(min(deg u, deg v) · log).
- Ghi nhớ kết quả của các cặp đã hỏi (ví dụ dùng `map`). Có thể chứng minh tổng chi phí trên các cặp **khác nhau** không vượt quá cỡ $m \cdot \sqrt{q}$.

BÀI 2.3	BẢN ĐỒ KHO BÁU	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BANDO.CPP	**BANDO.INP**	**BANDO.OUT**	**1 giây · 256 MB**

Kiến thức: *Biểu diễn lưới bằng dx, dy; cập nhật cục bộ (Tập 1, Bài 2).*

Tấm bản đồ kho báu là một lưới R hàng, C cột. Ô ở hàng x, cột y kí hiệu là (x, y); mỗi ô hoặc là **đất trống** (kí tự `.`), hoặc là **đá** (kí tự `#`). Hai ô **kề nhau** nếu chúng chung một cạnh.

Theo lời truyền lại, kho báu được chôn ở một ô đất trống có **đúng k ô đất trống kề với nó**. Ta gọi những ô như vậy là ô **khả nghi**.

Đoàn thám hiểm muốn biết số ô khả nghi. Nhưng địa hình liên tục thay đổi: có q sự kiện xảy ra lần lượt, sự kiện (x, y) làm ô (x, y) **đổi trạng thái**: nếu đang là đất trống thì bị đá lở lấp thành đá, nếu đang là đá thì được đào thành đất trống. Hãy cho biết số ô khả nghi lúc ban đầu và sau mỗi sự kiện.

Dữ liệu vào

Dòng đầu chứa bốn số nguyên R, C, k, q. R dòng tiếp theo, mỗi dòng là một xâu độ dài C chỉ gồm các kí tự `.` và `#` mô tả bản đồ. Mỗi dòng trong q dòng cuối chứa hai số x, y ($1 \le x \le R$, $1 \le y \le C$) mô tả một sự kiện.

Kết quả

Gồm q + 1 dòng: dòng đầu là số ô khả nghi lúc ban đầu, dòng thứ i + 1 là số ô khả nghi sau sự kiện thứ i.

Ràng buộc

$1 \le R, C \le 1000$; $0 \le k \le 4$; $0 \le q \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	q = 0.
2	30%	R, C ≤ 100 và q ≤ 100.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

BANDO.INP	BANDO.OUT	Giải thích
3 4 2 2 ..#. .... #..# 1 3 2 2	6 6 5	Ban đầu có 6 ô khả nghi (tô vàng trong Hình 2.3; số trong mỗi ô trống là số ô trống kề với nó). Đào ô (1, 3): ô (1, 2) có thêm một ô trống kề nên không còn khả nghi, còn ô (1, 4) trở thành khả nghi; vẫn là 6 ô. Lấp ô (2, 2): còn 5 ô khả nghi là (1, 1), (1, 2), (1, 4), (2, 4), (3, 3).



Hình 2.3. *Số trong ô trống là số ô trống kề cạnh với nó; ô vàng là ô khả nghi (k = 2).*

CHÚ Ý

Đếm lại toàn bộ bản đồ sau mỗi sự kiện tốn $R \cdot C \cdot q = 2 \cdot 10^{11}$ phép tính, quá chậm.

HƯỚNG SUY NGHĨ

- Khi ô (x, y) đổi trạng thái, "số ô trống kề" chỉ thay đổi ở chính nó và **bốn ô kề**. Những ô khác không bị ảnh hưởng.
- Trước khi đổi: trừ khỏi đáp số các ô khả nghi trong năm ô ấy. Đổi trạng thái. Sau đó cộng lại các ô khả nghi trong năm ô ấy. Mỗi sự kiện chỉ tốn O(1).

BÀI 3 Duyệt theo chiều sâu (DFS)

ÔN NHANH KIẾN THỨC · Tập 1, Bài 3

- DFS từ một đỉnh thăm đúng các đỉnh cùng **thành phần liên thông** với nó; gọi DFS từ mọi đỉnh chưa thăm để đếm số thành phần, tổng thời gian O(n + m).
- Một **rừng** có n đỉnh, k cây thì có đúng n − k cạnh. Đồ thị là **cây** khi và chỉ khi liên thông và có đúng n − 1 cạnh.
- DFS đệ quy sâu tới 10^5–10^6 tầng có thể tràn ngăn xếp: hãy viết bằng `vector` làm ngăn xếp, hoặc dùng BFS, hoặc dùng DSU (Bài 9).
- Loang trên lưới: mỗi ô là một đỉnh, kề với tối đa 4 ô; đánh dấu ô đã thăm **ngay khi đẩy vào ngăn xếp** để không đẩy một ô nhiều lần.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
3.1	**ĐẾM ĐẢO**	★☆☆ CƠ BẢN	Loang trên lưới không đệ quy.
3.2	**CÂY HAY KHÔNG**	★★☆ TRUNG BÌNH	Đưa bài toán tối ưu về đếm số thành phần liên thông.
3.3	**MẠNG LƯỚI DỰ PHÒNG**	★★★ NÂNG CAO	Đếm theo thành phần; lựa chọn tối ưu từ kích thước các thành phần.

BÀI 3.1 ĐẾM ĐẢO ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DEMDAO.CPP`	`DEMDAO.INP`	`DEMDAO.OUT`	1 giây · 256 MB

Kiến thức: *Loang (DFS/BFS) trên lưới, thành phần liên thông (Tập 1, Bài 3).*

Ảnh vệ tinh một vùng biển được chia thành lưới R hàng, C cột. Mỗi ô là **đất** (kí tự `#`) hoặc **nước** (kí tự `.`). Hai ô đất **chung cạnh** thuộc cùng một hòn đảo; hai ô chỉ chung một góc thì không được coi là nối với nhau trực tiếp.

Một **hòn đảo** là một nhóm ô đất liên thông lớn nhất có thể. Diện tích hòn đảo là số ô của nó. Hãy đếm số hòn đảo và tìm diện tích hòn đảo lớn nhất.

Dữ liệu vào

Dòng đầu chứa hai số nguyên R, C. R dòng tiếp theo, mỗi dòng là một xâu độ dài C chỉ gồm `#` và `.`.

Kết quả

Hai số nguyên: số hòn đảo và diện tích hòn đảo lớn nhất (in 0 0 nếu không có ô đất nào).

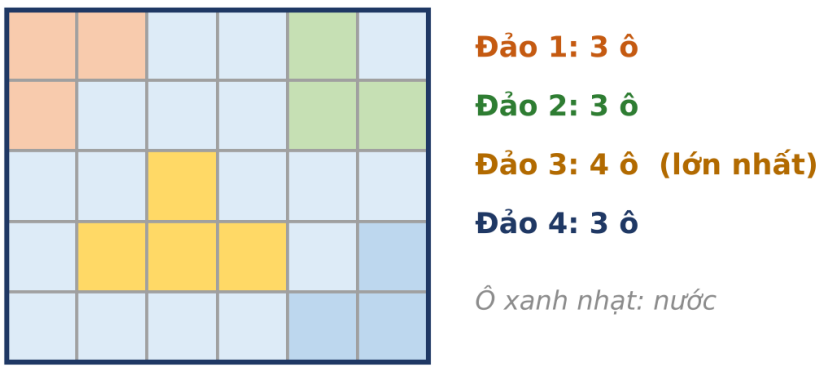
Ràng buộc

$1 \le R, C \le 1000$.

Subtask	Điểm	Ràng buộc thêm
1	40%	$R, C \le 50$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

DEMDAO.INP	DEMDAO.OUT	Giải thích
5 6 ##..#. #...## ..#... .###.# ....##	4 4	Có 4 hòn đảo với diện tích 3, 3, 4, 3 (Hình 3.1). Đảo lớn nhất rộng 4 ô. Ô (1, 5) và ô (2, 6) chỉ chung góc, nhưng cùng thuộc đảo 2 nhờ ô (2, 5).



Hình 3.1. *Mỗi hòn đảo một màu; ô xanh nhạt là nước.*

CHÚ Ý

Khi cả bảng 1000×1000 là đất, DFS đệ quy sẽ đi sâu tới 10^6 tầng và thường làm chương trình bị lỗi tràn ngăn xếp (Runtime Error). Hãy dùng ngăn xếp hoặc hàng đợi tự quản lí.

HƯỚNG SUY NGHĨ

- Duyệt từng ô; gặp ô đất chưa thăm thì tăng số đảo và loang từ ô đó, vừa loang vừa đếm số ô.

BÀI 3.2	CÂY HAY KHÔNG	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CAYKHONG.CPP`	`CAYKHONG.INP`	`CAYKHONG.OUT`	1 giây · 256 MB

Kiến thức: *Thành phần liên thông, tính chất của cây và rừng (Tập 1, Bài 3).*

Thầy giáo vẽ lên bảng một đồ thị vô hướng n đỉnh, m cạnh. Đồ thị có thể có **khuyên** (cạnh nối một đỉnh với chính nó) và **cạnh lặp** (nhiều cạnh nối cùng một cặp đỉnh).

Mỗi **thao tác**, một học sinh được xóa đi một cạnh đang có, hoặc vẽ thêm một cạnh nối hai đỉnh bất kì. Hỏi cần ít nhất bao nhiêu thao tác để đồ thị trở thành một **cây** (liên thông, không có chu trình)? Đặc biệt, nếu đồ thị đã là cây thì đáp số là 0.

Thầy hỏi T lần với T đồ thị khác nhau.

Dữ liệu vào

Dòng đầu chứa số T. Mỗi bộ dữ liệu gồm: một dòng chứa n, m; tiếp theo là m dòng, mỗi dòng hai số u, v ($1 \le u, v \le n$) mô tả một cạnh.

Kết quả

Gồm T dòng, mỗi dòng là số thao tác ít nhất cho bộ dữ liệu tương ứng.

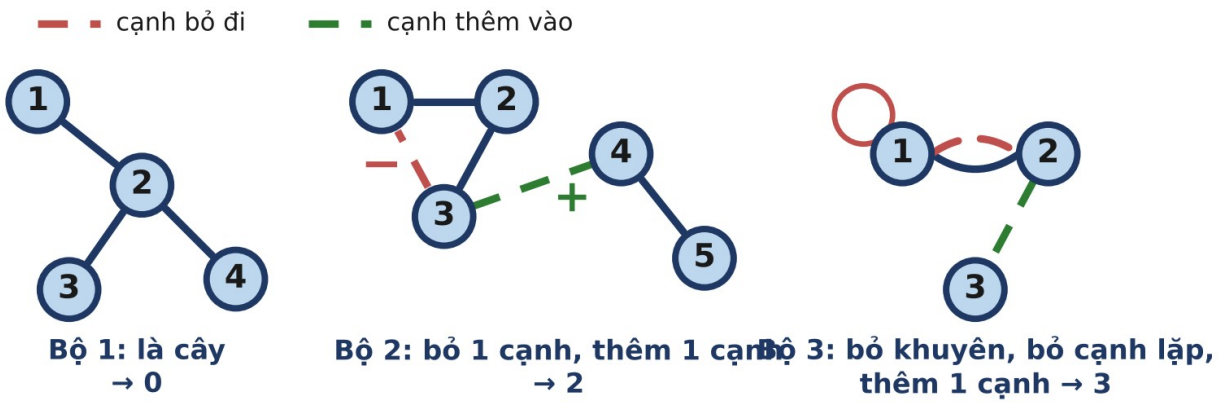
Ràng buộc

$1 \le T \le 10^4$; $n \ge 1, m \ge 0$; tổng các n và tổng các m trong một tệp đều không vượt quá $2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Không có khuyên và cạnh lặp, đồ thị liên thông.
2	30%	Không có khuyên và cạnh lặp.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

CAYKHONG.INP	CAYKHONG.OUT	Giải thích
3 4 3 1 2 2 3 2 4 5 4 1 2 2 3 3 1 4 5 3 3 1 1 1 2 1 2	0 2 3	Bộ 1: đã là cây. Bộ 2: xóa một cạnh của tam giác 1-2-3 và nối hai thành phần lại: 2 thao tác. Bộ 3: xóa khuyên tại 1, xóa một trong hai cạnh (1, 2), nối đỉnh 3 vào: 3 thao tác (Hình 3.2).



Hình 3.2. Ba bộ dữ liệu của ví dụ: cạnh đỏ nét đứt bị xóa, cạnh xanh nét đứt được thêm.

CHÚ Ý

Khuyên và cạnh lặp đều nằm trên chu trình, nên chúng luôn phải bị xóa. Ví dụ: đồ thị 2 đỉnh có đúng một cạnh là khuyên tại đỉnh 1 có $m = n - 1$ nhưng **không** phải cây (đáp số là 2: xóa khuyên, thêm cạnh (1, 2)).

HƯỚNG SUY NGHĨ

- Gọi k là số thành phần liên thông. Mỗi lần thêm cạnh giảm k đi nhiều nhất 1, mỗi lần xóa cạnh không làm giảm k. Vậy cần ít nhất bao nhiêu lần thêm?
- Đồ thị cuối cùng có đúng $n - 1$ cạnh. Từ số cạnh ban đầu m và số lần thêm, suy ra số lần xóa. Đáp số là một công thức theo n, m, k.

BÀI 3.3	MẠNG LƯỚI DỰ PHÒNG	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DUPHONG.CPP	**DUPHONG.INP**	**DUPHONG.OUT**	**1 giây · 256 MB**

Kiến thức: *Kích thước thành phần liên thông, đếm cặp (Tập 1, Bài 3).*

Mạng nội bộ của một công ti gồm n máy tính, đánh số từ 1 tới n, và m sợi cáp, mỗi sợi nối trực tiếp hai máy. Hai máy **liên lạc được** với nhau nếu có một dãy cáp nối chúng (có thể qua các máy trung gian).

Bộ phận kĩ thuật cần một bản báo cáo gồm ba thông tin:

1) Với mỗi máy, số máy liên lạc được với nó (tính cả chính nó).

2) Số cặp máy (không kể thứ tự) liên lạc được với nhau.

3) Nếu được lắp thêm **đúng một** sợi cáp dự phòng nối hai máy tùy chọn, số cặp máy liên lạc được với nhau lớn nhất có thể là bao nhiêu.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$) mô tả một sợi cáp.

Kết quả

Dòng 1 gồm n số, số thứ i là câu trả lời của ý 1) cho máy i. Dòng 2 và dòng 3 lần lượt là câu trả lời ý 2) và ý 3).

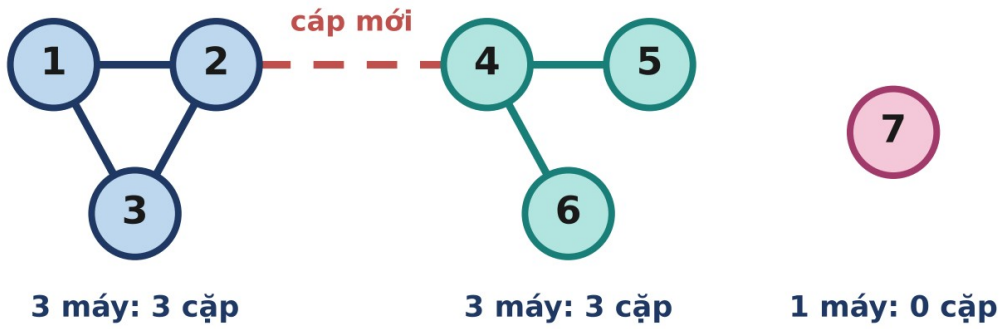
Ràng buộc

$1 \le n \le 5 \cdot 10^5$; $0 \le m \le 5 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n, m \le 1000$.
2	30%	$n, m \le 10^5$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DUPHONG.INP	DUPHONG.OUT	Giải thích
7 5 1 2 2 3 4 5 6 4 3 1	3 3 3 3 3 3 1 6 15	Ba nhóm máy: {1, 2, 3}, {4, 5, 6} và {7}. Số cặp liên lạc được: 3 + 3 + 0 = 6. Nối hai nhóm 3 máy lại (Hình 3.3): thêm 3 · 3 = 9 cặp, tổng 15.



Hình 3.3. *Nối hai nhóm lớn nhất tạo ra nhiều cặp mới nhất.*

CHÚ Ý

Số cặp có thể tới khoảng $1{,}25 \cdot 10^{11}$: hãy dùng `long long`.

Mạng có thể là một "đường thẳng" $5 \cdot 10^5$ máy, DFS đệ quy sẽ tràn ngăn xếp.

HƯỚNG SUY NGHĨ

- Một nhóm (thành phần liên thông) có s máy đóng góp $s(s-1)/2$ cặp.
- Một sợi cáp mới chỉ có ích khi nối hai nhóm khác nhau có kích thước a và b: số cặp tăng thêm $a \cdot b$. Chọn a, b thế nào?

BÀI 4 Duyệt theo chiều rộng (BFS)

ÔN NHANH KIẾN THỨC · Tập 1, Bài 4

- BFS thăm các đỉnh theo thứ tự khoảng cách (số cạnh) tăng dần; d[v] là **số cạnh ít nhất** từ đỉnh xuất phát tới v. Độ phức tạp O(n + m).
- **BFS nhiều nguồn**: đẩy tất cả các nguồn vào hàng đợi với d = 0 từ đầu; d[v] là khoảng cách tới nguồn **gần nhất**.
- Đồ thị **hai phía** khi và chỉ khi không có chu trình lẻ; kiểm tra bằng cách tô hai màu theo BFS: đỉnh kề phải khác màu.
- Trên lưới, BFS cho đường đi ít bước nhất; đánh dấu ô ngay khi đẩy vào hàng đợi.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
4.1	**MÃ ĐI TUẦN**	★☆☆ CƠ BẢN	Một lần BFS trả lời mọi câu hỏi có chung điểm xuất phát.
4.2	**CHẠY TRỐN ĐÁM CHÁY**	★★☆ TRUNG BÌNH	Tách bài toán "hai đối tượng cùng di chuyển" thành hai lần BFS.
4.3	**CHIA ĐỘI**	★★★ NÂNG CAO	Kết hợp tô màu hai phía với bài toán chia tập.

BÀI 4.1 MÃ ĐI TUẦN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`MADITUAN.CPP`	`MADITUAN.INP`	`MADITUAN.OUT`	1 giây · 256 MB

Kiến thức: *BFS trên lưới với 8 hướng đi của quân mã (Tập 1, Bài 4).*

Trên bàn cờ N × N (hàng và cột đánh số từ 1 tới N), một quân mã đang đứng ở ô (x_0, y_0). Mỗi nước, quân mã đi theo hình chữ L: sang ô (x + a, y + b) với (a, b) là một trong 8 cặp (±1, ±2), (±2, ±1), miễn là ô mới còn nằm trong bàn cờ.

Có Q câu hỏi, mỗi câu hỏi là một ô đích (x, y). Với mỗi câu hỏi, hãy cho biết quân mã cần ít nhất bao nhiêu nước để đi từ ô xuất phát tới ô đích, hoặc −1 nếu không thể tới được.

Dữ liệu vào

Dòng đầu chứa bốn số nguyên N, x_0, y_0, Q. Mỗi dòng trong Q dòng tiếp theo chứa hai số x, y là một ô đích.

Kết quả

Gồm Q dòng, dòng thứ i là đáp số của câu hỏi thứ i.

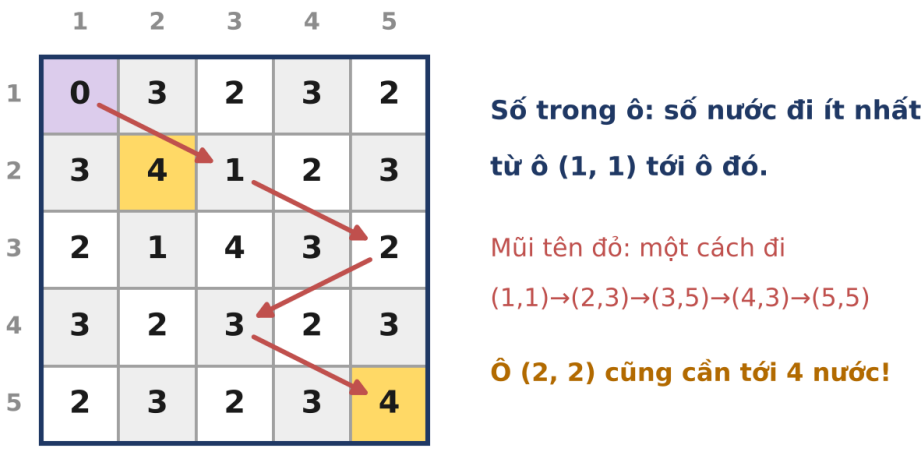
Ràng buộc

$1 \le N \le 1000$; $1 \le Q \le 10^5$; mọi ô đều nằm trong bàn cờ.

Subtask	Điểm	Ràng buộc thêm
1	30%	$N \le 8$.
2	30%	$Q = 1$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

MADITUAN.INP	MADITUAN.OUT	Giải thích
5 1 1 3 5 5 2 2 1 1	4 4 0	Ô (5, 5) cần 4 nước (Hình 4.1). Ô (2, 2) tuy ở ngay chéo ô xuất phát nhưng cũng cần 4 nước! Ô (1, 1) chính là ô xuất phát: 0 nước.



Hình 4.1. *Kết quả BFS từ ô (1, 1) trên bàn cờ 5 × 5.*

CHÚ Ý

Chạy BFS riêng cho từng câu hỏi tốn $Q \cdot N^2 = 10^{11}$ phép tính. Mọi câu hỏi có **chung** ô xuất phát!

Với $N \le 3$ có những ô không thể tới (ví dụ ô giữa bàn cờ 3 × 3).

HƯỚNG SUY NGHĨ

- Chạy BFS **một lần** từ (x_0, y_0) trên N^2 ô, lưu d cho mọi ô, rồi trả lời mỗi câu hỏi trong O(1).

BÀI 4.2	CHẠY TRỐN ĐÁM CHÁY	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CHAYTRON.CPP`	`CHAYTRON.INP`	`CHAYTRON.OUT`	1 giây · 256 MB

Kiến thức: *BFS nhiều nguồn, so sánh hai lần BFS (Tập 1, Bài 4).*

An đang ở trong một tòa nhà có mặt bằng là lưới R × C thì đám cháy bùng lên. Mỗi ô là tường (`#`), lối đi (`.`), vị trí của An (`J`, có đúng một ô) hoặc một ô đang cháy (`F`, có thể có nhiều ô hoặc không có ô nào).

Mỗi phút, **cùng một lúc**: An di chuyển sang một ô kề cạnh không phải tường; và lửa từ mỗi ô đang cháy lan sang mọi ô kề cạnh không phải tường. An **không được** bước vào ô mà lửa tới vào cùng phút đó hoặc sớm hơn. Lửa và An đều không đi xuyên tường.

Nếu An đang đứng ở một ô thuộc **hàng hoặc cột ngoài cùng** của lưới, phút tiếp theo An có thể bước ra khỏi tòa nhà. Hỏi An cần ít nhất bao nhiêu phút để thoát ra ngoài?

Dữ liệu vào

Dòng đầu chứa hai số nguyên R, C. R dòng tiếp theo, mỗi dòng là một xâu độ dài C mô tả một hàng của lưới.

Kết quả

Số phút ít nhất để An thoát ra ngoài, hoặc −1 nếu An không thể thoát.

Ràng buộc

$1 \le R, C \le 1000$.

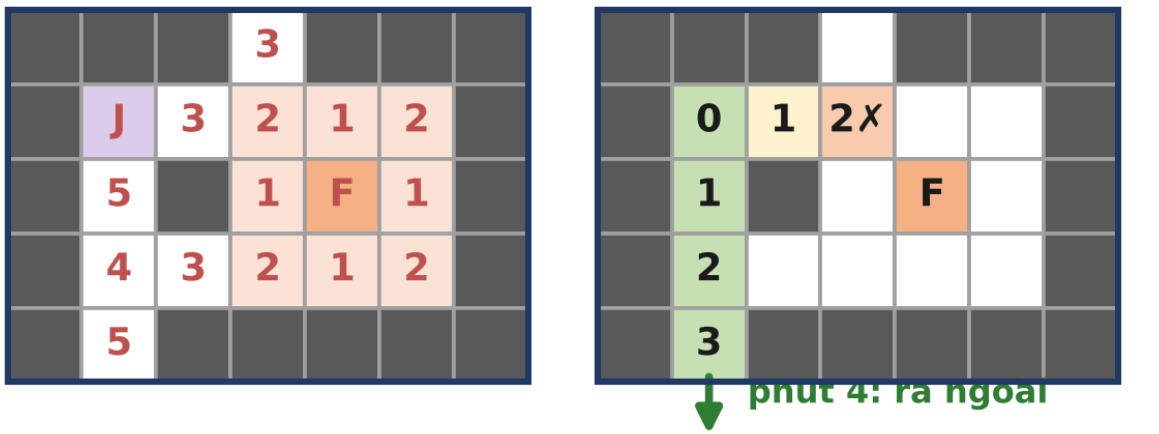
Subtask	Điểm	Ràng buộc thêm
1	30%	Không có ô nào đang cháy.
2	30%	Có đúng một ô đang cháy, $R, C \le 100$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

CHAYTRON.INP	CHAYTRON.OUT	Giải thích

`5 7`
`###.###`
`#J....#`
`#.#.F.#`
`#.....#`
`#.#####`
	4	Lối ra phía trên rất gần, nhưng An tới ô (2, 4) ở phút 2, đúng lúc lửa cũng tới: không được phép (Hình 4.2).
		An chạy xuống lối ra phía dưới: có mặt ở ô (5, 2) ở phút 3 và ra ngoài ở phút 4.

CHAYTRON.INP	CHAYTRON.OUT	Giải thích
4 4 #### #J.# #.F. ####	-1	Ô duy nhất dẫn ra ngoài là ô (3, 4), nhưng lửa ở ngay cạnh và tới đó ở phút 1. An không thể thoát.



Hình 4.2. *Ví dụ 1. Trái: phút lửa tới từng ô. Phải: phút An có mặt ở từng ô trên đường chạy (ô đỏ: không được bước vào).*

CHÚ Ý

Không mô phỏng lại toàn bộ đám cháy sau mỗi bước chạy của An: cách đó tốn R · C cho mỗi phút.

Điều kiện là "lửa tới **sau** An", tức thời điểm An có mặt phải **nhỏ hơn hẳn** thời điểm lửa tới ô đó.

HƯỚNG SUY NGHĨ

- BFS nhiều nguồn từ mọi ô `F` cho fire[x][y]: phút lửa tới ô (x, y) (vô cùng nếu không tới).
- BFS từ ô `J` cho An, chỉ bước vào ô (x, y) ở phút t nếu t < fire[x][y]. Ô biên đầu tiên được lấy ra khỏi hàng đợi cho đáp số t + 1.

BÀI 4.3	CHIA ĐỘI	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
CHIADOI.CPP	CHIADOI.INP	CHIADOI.OUT	1 giây · 256 MB

Kiến thức: *Kiểm tra đồ thị hai phía bằng BFS; quy hoạch động tổng tập con (Tập 1, Bài 4).*

Lớp có n bạn, đánh số từ 1 tới n, cần chia thành **hai đội** để chơi kéo co; mỗi bạn thuộc đúng một đội, và một đội có thể không có ai. Có m cặp bạn **ghét nhau**; hai bạn ghét nhau phải ở **khác đội**.

Trong các cách chia hợp lệ, cô giáo muốn hai đội có số người **chênh lệch ít nhất**. Hãy tìm độ chênh lệch nhỏ nhất ấy.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$) cho biết u và v ghét nhau. Không có cặp nào được cho hai lần.

Kết quả

Độ chênh lệch nhỏ nhất giữa số người của hai đội, hoặc −1 nếu không có cách chia nào hợp lệ.

Ràng buộc

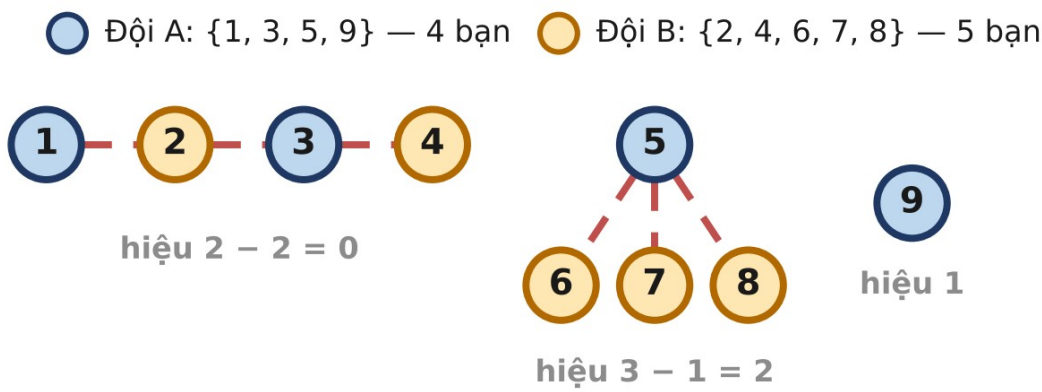
$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 20$.
2	30%	Với mọi cặp bạn, có một dãy quan hệ "ghét nhau" nối họ (đồ thị liên thông).
3	20%	$n \le 5000$.
4	30%	Không có ràng buộc gì thêm.

Ví dụ

CHIADOI.INP	CHIADOI.OUT	Giải thích
9 6 1 2 2 3 3 4 5 6 5 7 5 8	1	Ba nhóm: dãy 1-2-3-4 (chia 2 và 2), nhóm 5-6-7-8 (bạn 5 một bên, ba bạn kia một bên) và bạn 9 đứng riêng. Cách tốt nhất (Hình 4.3): đội A = {1, 3, 5, 9}, đội B = {2, 4, 6, 7, 8}, chênh lệch 1.

CHIADOI.INP	CHIADOI.OUT	Giải thích
3 3 1 2 2 3 3 1	-1	Ba bạn đôi một ghét nhau: không thể chia vào hai đội.



Hình 4.3. *Ví dụ 1: đường nét đứt nối hai bạn ghét nhau. Mỗi nhóm chỉ có hai cách tô màu, ta chọn bên nào vào đội A.*

CHÚ Ý

Mỗi thành phần liên thông được tô hai màu theo **đúng một** cách (sai khác đổi màu toàn bộ). Vì thế ta chỉ được chọn, cho mỗi thành phần, phía nào vào đội A.

HƯỚNG SUY NGHĨ

- Với thành phần thứ i, gọi a_i, b_i là số đỉnh hai màu. Độ chênh lệch cuối cùng là $|\pm(a_1 - b_1) \pm (a_2 - b_2) \pm \ldots|$.
- Đây là bài toán chia các số $c_i = |a_i - b_i|$ thành hai nhóm có tổng gần nhau nhất: quy hoạch động tổng tập con với `bitset`. Tổng các c_i không vượt quá n.
- Để nhanh hơn nữa: các c_i bằng nhau có thể gộp lại theo lũy thừa của 2 (1, 2, 4, ...).

CHƯƠNG 2. ĐƯỜNG ĐI NGẮN NHẤT

Tìm đường đi ngắn nhất là dạng bài xuất hiện nhiều nhất trong các kì thi. Các bài tập của chương rèn cho em ba kĩ năng: chọn đúng thuật toán theo loại trọng số và kích thước đồ thị (Dijkstra, Bellman-Ford hay Floyd); biến đổi bài toán về dạng đường đi ngắn nhất (logarit, ràng buộc hiệu, phép toán max-min); và xây dựng đồ thị trạng thái khi "vị trí" thôi là chưa đủ.

TRONG CHƯƠNG NÀY

Bài 5. Thuật toán Dijkstra

- **5.1 ĐƯỜNG TỚI TRƯỜNG** ★☆☆ CƠ BẢN
- **5.2 NÚT GIAO QUAN TRỌNG** ★★☆ TRUNG BÌNH
- **5.3 XE BUÝT ĐÊM** ★★★ NÂNG CAO

Bài 6. Thuật toán Bellman-Ford và chu trình âm

- **6.1 ĐỔI TIỀN** ★☆☆ CƠ BẢN
- **6.2 CHUYẾN BAY GIÁ RẺ** ★★☆ TRUNG BÌNH
- **6.3 LỊCH THI ĐẤU** ★★★ NÂNG CAO

Bài 7. Thuật toán Floyd

- **7.1 THỦ ĐÔ MỚI** ★☆☆ CƠ BẢN
- **7.2 XẾP HẠNG KÌ THỦ** ★★☆ TRUNG BÌNH
- **7.3 XE TẢI QUÁ KHỔ** ★★★ NÂNG CAO

Bài 8. Đồ thị trạng thái và BFS 0-1

- **8.1 ĐÈN GIAO THÔNG** ★☆☆ CƠ BẢN
- **8.2 PHÁ TƯỜNG** ★★☆ TRUNG BÌNH
- **8.3 ROBOT XOAY** ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trọng số không âm và một nguồn: **Dijkstra**. Có trọng số âm hoặc giới hạn số cạnh: **Bellman-Ford**. Cần mọi cặp và $n \le 500$: **Floyd**.
- Gần như mọi bài đường đi ngắn nhất đều cần `long long`. Hãy tạo thói quen dùng nó ngay từ đầu.
- Với đồ thị trạng thái, hãy viết ra: trạng thái gồm những gì, có bao nhiêu trạng thái, từ một trạng thái đi được tới đâu và tốn bao nhiêu.

BÀI 5 Thuật toán Dijkstra

ÔN NHANH KIẾN THỨC · Tập 1, Bài 5

- Dijkstra tìm đường đi ngắn nhất từ một đỉnh khi **mọi trọng số không âm**. Mỗi lần chốt đỉnh u có nhãn nhỏ nhất, rồi nới lỏng d[v] = min(d[v], d[u] + w(u, v)).
- Cài đặt bằng `priority_queue` các cặp (d, u); khi lấy ra cặp có d ≠ d[u] thì bỏ qua (bản cũ). Độ phức tạp O(m log n).
- Đỉnh v nằm trên **một** đường đi ngắn nhất từ s tới t khi $d_s[v] + d_t[v] = d_s[t]$; cạnh (u, v) nằm trên một đường đi ngắn nhất khi $d_s[u] + w + d_t[v] = d_s[t]$.
- Khoảng cách có thể vượt $2 \cdot 10^9$: dùng `long long`, và chọn giá trị "vô cùng" đủ lớn.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
5.1	**ĐƯỜNG TỚI TRƯỜNG**	★☆☆ CƠ BẢN	Cài đặt Dijkstra chuẩn với hàng đợi ưu tiên.
5.2	**NÚT GIAO QUAN TRỌNG**	★★☆ TRUNG BÌNH	Phân tích cấu trúc của tập mọi đường đi ngắn nhất.
5.3	**XE BUÝT ĐÊM**	★★★ NÂNG CAO	Nhận ra khi nào Dijkstra vẫn đúng với trọng số phụ thuộc thời gian.

BÀI 5.1 ĐƯỜNG TỚI TRƯỜNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DUONGTRUONG.CPP`	`DUONGTRUONG.INP`	`DUONGTRUONG.OUT`	1 giây · 256 MB

Kiến thức: *Thuật toán Dijkstra trên đồ thị có hướng (Tập 1, Bài 5).*

Thị trấn có n nút giao thông, đánh số từ 1 tới n, và m con đường **một chiều**. Con đường thứ i đi từ nút u_i tới nút v_i và mất w_i phút.

Nhà An ở nút s. Sáng nay An muốn tính trước thời gian ngắn nhất để đi từ nhà tới **mọi** nút giao thông trong thị trấn, để chọn hiệu sách, nhà bạn, trường học... cho tiện đường.

Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, s. Mỗi dòng trong m dòng tiếp theo chứa ba số nguyên u, v, w (u ≠ v) mô tả một con đường một chiều từ u tới v mất w phút.

Kết quả

Một dòng gồm n số, số thứ v là thời gian ngắn nhất để đi từ s tới v, hoặc −1 nếu không đi tới được.

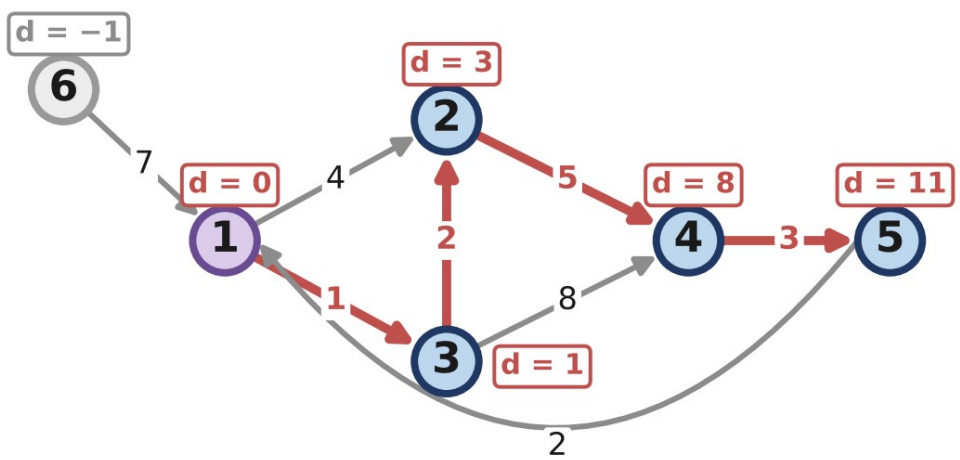
Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $0 \le w \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	40%	$n \le 1000$, $m \le 5000$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

DUONGTRUONG.INP	DUONGTRUONG.OUT	Giải thích
6 8 1 1 2 4 1 3 1 3 2 2 2 4 5 3 4 8 4 5 3 5 1 2 6 1 7	0 3 1 8 11 -1	Tới nút 2: đi 1 → 3 → 2 mất 1 + 2 = 3 phút, nhanh hơn đường thẳng 1 → 2 (4 phút). Tới nút 4: 1 → 3 → 2 → 4 mất 8 phút; tới nút 5 mất 11 phút. Nút 6 chỉ có đường **đi ra** tới 1, không có đường đi vào: in −1.



Hình 5.1. *Cạnh đỏ tạo thành cây đường đi ngắn nhất từ nút 1; nhãn d là đáp số.*

CHÚ Ý

Một đường đi có thể dài tới $(10^5 - 1) \cdot 10^9 \approx 10^{14}$ phút: phải dùng `long long`. Với các nút không tới được, nhớ in −1 chứ không in giá trị "vô cùng".

Trọng số có thể bằng 0; Dijkstra vẫn đúng.

HƯỚNG SUY NGHĨ

- Subtask 1: Dijkstra $O(n^2)$ (mỗi lượt tìm đỉnh có nhãn nhỏ nhất bằng vòng lặp).
- Subtask 2: Dijkstra với `priority_queue`.

BÀI 5.2	NÚT GIAO QUAN TRỌNG	★★☆ TRUNG BÌNH	
Chương trình **NUTGIAO.CPP**	Dữ liệu vào **NUTGIAO.INP**	Kết quả **NUTGIAO.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Dijkstra từ hai đầu, cạnh và đỉnh trên đường đi ngắn nhất (Tập 1, Bài 5).*

Thành phố có n nút giao, đánh số từ 1 tới n, và m con đường **hai chiều**; con đường thứ i nối u_i với v_i và dài w_i mét. Mỗi sáng, hàng nghìn xe đi từ nút 1 (khu dân cư) tới nút n (khu công nghiệp), và mỗi xe luôn chọn **một** đường đi ngắn nhất (nếu có nhiều đường ngắn nhất, xe có thể chọn bất kì đường nào).

Một nút giao được gọi là **quan trọng** nếu nó nằm trên **mọi** đường đi ngắn nhất từ 1 tới n: dù xe chọn đường nào cũng phải đi qua nó. Hiển nhiên nút 1 và nút n là quan trọng. Hãy tìm tất cả các nút quan trọng để thành phố bố trí cảnh sát giao thông.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa ba số u, v, w ($u \neq v$) mô tả một con đường. Dữ liệu đảm bảo có đường đi từ 1 tới n.

Kết quả

Dòng đầu là số nút quan trọng k. Dòng thứ hai gồm k số là các nút quan trọng theo thứ tự tăng dần.

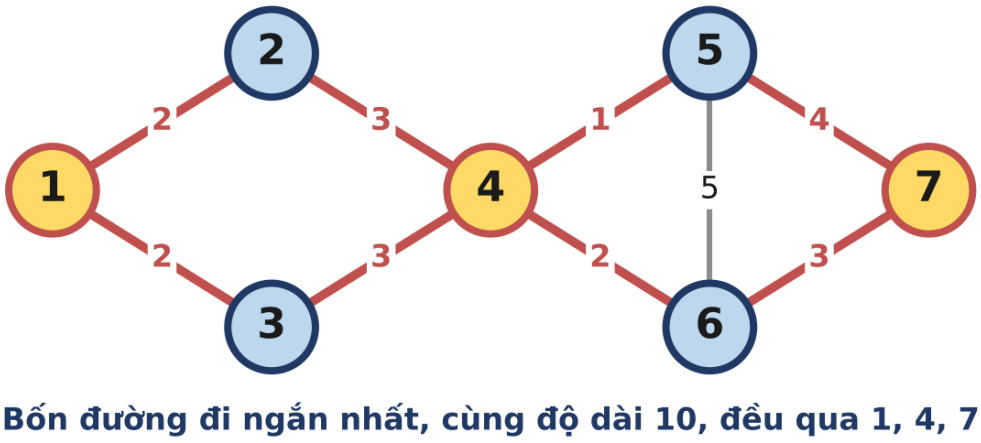
Ràng buộc

$2 \le n \le 10^5$; $1 \le m \le 2 \cdot 10^5$; $1 \le w \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 1000$, $m \le 2000$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

NUTGIAO.INP	NUTGIAO.OUT	Giải thích
7 9 1 2 2 1 3 2 2 4 3 3 4 3 4 5 1 5 7 4 4 6 2 6 7 3 5 6 5	3 1 4 7	Khoảng cách từ 1 tới 7 là 10, có bốn đường ngắn nhất: 1-2-4-5-7, 1-2-4-6-7, 1-3-4-5-7, 1-3-4-6-7. Cả bốn đường đều qua 1, 4, 7 (Hình 5.2). Con đường 5-6 dài 5 không nằm trên đường ngắn nhất nào.



Hình 5.2. *Cạnh đỏ nằm trên ít nhất một đường đi ngắn nhất; đỉnh vàng là nút quan trọng.*

CHÚ Ý

Đỉnh v nằm trên **một** đường đi ngắn nhất chưa chắc đã nằm trên **mọi** đường đi ngắn nhất (ví dụ đỉnh 2).

Số đường đi ngắn nhất có thể lớn khủng khiếp (tới $2^{n/2}$), không thể liệt kê.

HƯỚNG SUY NGHĨ

- Subtask 1: với mỗi đỉnh v, xóa v rồi chạy lại Dijkstra; v quan trọng khi khoảng cách từ 1 tới n tăng lên (hoặc không còn đường).
- Chạy Dijkstra từ 1 (được d_1) và từ n (được d_n). Cách 1: đếm số đường đi ngắn nhất $cnt_1[v]$, $cnt_n[v]$ theo mô-đun một số nguyên tố lớn; v quan trọng khi $cnt_1[v] \cdot cnt_n[v] \equiv cnt_1[n]$.
- Cách 2 (không dùng mô-đun): mỗi đường đi ngắn nhất đi qua các "mốc" d_1 tăng dần từ 0 tới D. Đỉnh v quan trọng khi **không** có đỉnh nào khác trên đường ngắn nhất cùng mốc $d_1[v]$, và không có cạnh đường-ngắn-nhất nào "nhảy qua" mốc ấy.

BÀI 5.3	XE BUÝT ĐÊM	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
XEBUYT.CPP	**XEBUYT.INP**	**XEBUYT.OUT**	**1 giây · 256 MB**

Kiến thức: *Dijkstra theo thời điểm tới, nới lỏng có điều kiện (Tập 1, Bài 5).*

Mạng xe buýt đêm của thành phố có n bến, đánh số từ 1 tới n, và m tuyến. Tuyến thứ i chạy **một chiều** từ bến u_i tới bến v_i và mất w_i phút. Chuyến đầu tiên của tuyến xuất bến lúc a_i, sau đó cứ p_i phút lại có một chuyến, và chuyến cuối cùng xuất bến **không muộn hơn** b_i. Nói cách khác, tuyến i có các chuyến xuất bến tại các thời điểm a_i, $a_i + p_i$, $a_i + 2p_i$, ... không vượt quá b_i.

Bình đang ở bến 1 tại thời điểm 0 và muốn về bến n càng sớm càng tốt. Bình có thể chờ ở bến bao lâu cũng được; nếu tới một bến lúc t thì có thể lên ngay chuyến xuất bến đúng lúc t. Hỏi Bình về tới bến n sớm nhất lúc nào?

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa sáu số nguyên u, v, w, a, p, b mô tả một tuyến xe buýt ($u \neq v$).

Kết quả

Thời điểm sớm nhất Bình tới bến n, hoặc −1 nếu không thể tới.

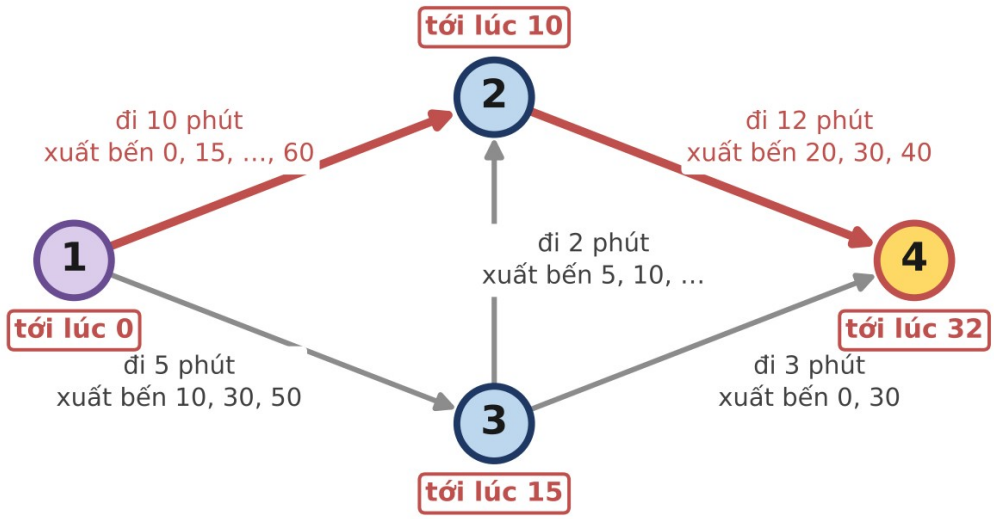
Ràng buộc

$2 \le n \le 10^5$; $1 \le m \le 2 \cdot 10^5$; $1 \le w, p \le 10^9$; $0 \le a \le b \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Với mọi tuyến, a = 0 và p = 1 (xe xuất bến bất cứ lúc nào trước giờ chót b).
2	30%	n, m ≤ 1000.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

XEBUYT.INP	XEBUYT.OUT	Giải thích
4 5 1 2 10 0 15 60 1 3 5 10 20 50 2 4 12 20 10 40 3 4 3 0 30 30 3 2 2 5 5 100	32	Bình lên tuyến 1 lúc 0, tới bến 2 lúc 10; chờ chuyến 20 của tuyến 3, tới bến 4 lúc **32** (Hình 5.3). Đi qua bến 3: tới bến 3 lúc 15 nhưng tuyến 3 → 4 phải chờ tới chuyến lúc 30, tới bến 4 lúc 33. Còn 3 → 2 tới bến 2 lúc 17, rồi vẫn phải chờ chuyến lúc 20.



Hình 5.3. *Nhãn dưới mỗi bến là thời điểm sớm nhất Bình có mặt ở bến đó.*

CHÚ Ý

Thời điểm tới có thể lên tới $2 \cdot 10^9$, sát giới hạn của kiểu `int`; còn biểu thức $a + \lceil (t - a)/p \rceil \cdot p$ có thể vượt xa hơn trước khi được so sánh với b. Hãy dùng `long long` cho mọi phép tính thời gian.

Tính chuyến xuất bến đầu tiên không sớm hơn t bằng công thức, đừng lặp từng chuyến: p có thể bằng 1 và $b - a$ tới 10^9.

HƯỚNG SUY NGHĨ

- Nếu tới bến u lúc t thì chuyến sớm nhất của tuyến i có thể lên là lúc a_i (nếu $t \le a_i$), hoặc $a_i + \lceil (t - a_i)/p_i \rceil \cdot p_i$; nếu thời điểm đó vượt b_i thì không đi được tuyến này nữa.
- Tới bến **sớm hơn** không bao giờ thiệt (luôn có thể chờ). Vì vậy Dijkstra với nhãn "thời điểm tới sớm nhất" vẫn đúng: khi chốt bến u với thời điểm t nhỏ nhất, mọi cách tới u khác đều không tốt hơn.

BÀI 6 Thuật toán Bellman-Ford và chu trình âm

ÔN NHANH KIẾN THỨC · Tập 1, Bài 6

- Bellman-Ford lặp n − 1 lượt, mỗi lượt nới lỏng **mọi** cạnh: O(n · m). Chạy được với trọng số âm.
- Nếu ở lượt thứ n vẫn còn nhãn giảm được thì đồ thị có **chu trình âm** tới được từ nguồn.
- Muốn đường đi dùng **không quá k cạnh**: chạy đúng k lượt, và mỗi lượt nới lỏng từ **bản sao** của mảng d ở lượt trước.
- **Hệ ràng buộc hiệu** $x_v - x_u \le c$ biến thành cạnh u → v trọng số c; hệ có nghiệm khi và chỉ khi đồ thị không có chu trình âm.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
6.1	**ĐỔI TIỀN**	★☆☆ CƠ BẢN	Mô hình hóa: biến "tích lớn hơn 1" thành "tổng âm".
6.2	**CHUYẾN BAY GIÁ RẺ**	★★☆ TRUNG BÌNH	Hiểu ý nghĩa "sau lượt thứ i" của Bellman-Ford.
6.3	**LỊCH THI ĐẤU**	★★★ NÂNG CAO	Biến hệ bất phương trình thành đồ thị.

BÀI 6.1 ĐỔI TIỀN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DOITIEN.CPP`	`DOITIEN.INP`	`DOITIEN.OUT`	1 giây · 256 MB

Kiến thức: *Chu trình âm, biến tích thành tổng bằng logarit (Tập 1, Bài 6).*

Ở một khu chợ quốc tế có n loại tiền, đánh số từ 1 tới n, và m quầy đổi tiền. Quầy thứ i nhận tiền loại u_i và đổi **mỗi đơn vị** thành r_i đơn vị tiền loại v_i (chỉ đổi theo một chiều).

Một người lanh lợi muốn tìm một **vòng đổi**: bắt đầu với một ít tiền loại nào đó, đổi lần lượt qua một số quầy rồi quay về đúng loại tiền ban đầu, và nhận được **nhiều tiền hơn** lúc đầu. Hãy cho biết vòng đổi như vậy có tồn tại không.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên u, v ($u \ne v$) và một số thực r (có không quá 4 chữ số sau dấu chấm thập phân) mô tả một quầy.

Kết quả

In `YES` nếu tồn tại vòng đổi có lãi, ngược lại in `NO`.

Ràng buộc

$1 \le n \le 100$; $0 \le m \le 10^4$; $0{,}0001 \le r \le 100$.

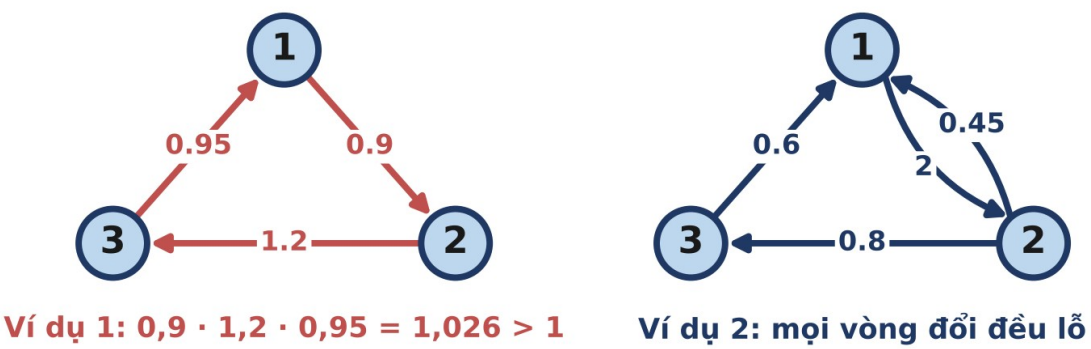
Dữ liệu đảm bảo: tích các tỉ giá dọc theo một vòng đổi bất kì hoặc không vượt quá 1, hoặc lớn hơn $1 + 10^{-6}$ (để tránh rắc rối do sai số).

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 3$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

DOITIEN.INP	DOITIEN.OUT	Giải thích
3 3 1 2 0.9 2 3 1.2 3 1 0.95	YES	Đổi 1 → 2 → 3 → 1: một đơn vị tiền 1 thành 0,9 · 1,2 · 0,95 = **1,026** đơn vị, có lãi (Hình 6.1).

DOITIEN.INP	DOITIEN.OUT	Giải thích
3 4 1 2 2 2 1 0.45 2 3 0.8 3 1 0.6	NO	Vòng 1 → 2 → 1 cho 2 · 0,45 = 0,9; vòng 1 → 2 → 3 → 1 cho 2 · 0,8 · 0,6 = 0,96. Không có vòng nào có lãi.



Hình 6.1. *Mỗi loại tiền là một đỉnh, mỗi quầy là một cung có trọng số là tỉ giá.*

CHÚ Ý

Tỉ giá là số thực: không so sánh bằng ==, và luôn dùng một sai số nhỏ (chẳng hạn 10^{-9}) khi kiểm tra "nhãn còn giảm".

Vòng đổi có lãi có thể không đi qua loại tiền 1; hãy coi như **mọi** đỉnh đều là nguồn (khởi tạo d[v] = 0 cho mọi v).

HƯỚNG SUY NGHĨ

- Tích $r_1 \cdot r_2 \cdot ... \cdot r_k > 1$ tương đương $(-\log r_1) + (-\log r_2) + ... + (-\log r_k) < 0$.
- Đặt trọng số cung là $-\log r$ rồi tìm chu trình âm bằng Bellman-Ford.

BÀI 6.2	CHUYẾN BAY GIÁ RẺ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
BAYRE.CPP	**BAYRE.INP**	**BAYRE.OUT**	**1 giây · 256 MB**

Kiến thức: *Bellman-Ford giới hạn số cạnh, sao chép mảng nhãn (Tập 1, Bài 6).*

Hãng hàng không giá rẻ có m chuyến bay giữa n thành phố; chuyến thứ i bay từ thành phố u_i tới thành phố v_i với giá vé c_i (bay một chiều).

Chi đang ở thành phố 1 và muốn tới thành phố n. Vì mỗi lần đổi máy bay rất mệt, Chi chỉ chấp nhận đi **không quá k chuyến bay**. Hãy tìm tổng giá vé ít nhất.

Dữ liệu vào

Dòng đầu chứa n, m, k. Mỗi dòng trong m dòng tiếp theo chứa ba số nguyên u, v, c ($u \neq v$) mô tả một chuyến bay.

Kết quả

Tổng giá vé ít nhất, hoặc −1 nếu không có cách đi nào dùng không quá k chuyến bay.

Ràng buộc

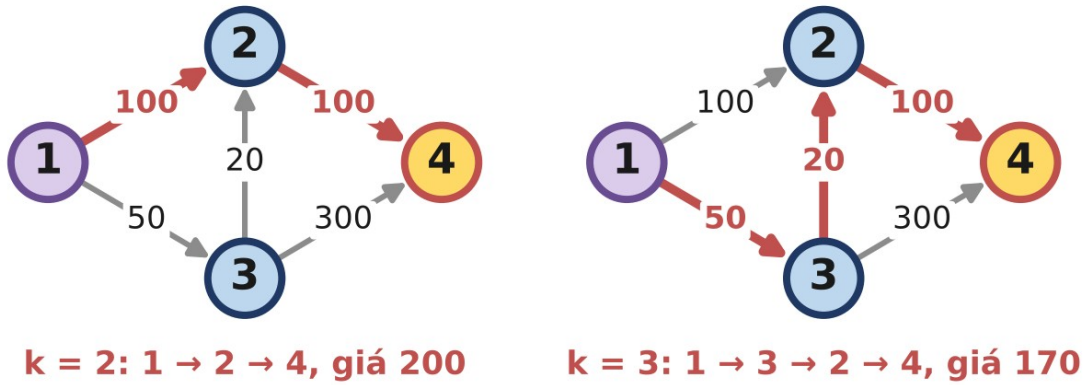
$2 \le n \le 1000$; $0 \le m \le 10^4$; $1 \le k \le 1000$; $1 \le c \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$k \ge n - 1$ (giới hạn số chuyến không có tác dụng).
2	30%	$n \le 100$, $m \le 1000$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

BAYRE.INP	BAYRE.OUT	Giải thích
4 5 2 1 2 100 2 4 100 1 3 50 3 2 20 3 4 300	200	Với k = 2, chỉ có hai cách: 1 → 2 → 4 (200) và 1 → 3 → 4 (350). Chọn **200**.

BAYRE.INP	BAYRE.OUT	Giải thích
4 5 3 1 2 100 2 4 100 1 3 50 3 2 20 3 4 300	170	Cùng mạng bay nhưng k = 3: đi 1 → 3 → 2 → 4 chỉ tốn 50 + 20 + 100 = **170** (Hình 6.2).



Hình 6.2. *Giới hạn số chuyến bay làm thay đổi đáp số.*

CHÚ Ý

Nếu trong một lượt em nới lỏng **ngay trên** mảng d (không sao chép), một lượt có thể "đi" được nhiều cạnh liền nhau. Với ví dụ 1, chương trình như vậy có thể in 170 thay vì 200!

HƯỚNG SUY NGHĨ

- Gọi $d_i[v]$ là giá rẻ nhất tới v dùng không quá i chuyến. Khi đó $d_i[v] = \min(d_{i-1}[v], \min\{d_{i-1}[u] + c : \text{chuyến } u \to v\})$.
- Chỉ cần hai mảng: mảng của lượt trước và mảng của lượt hiện tại. Độ phức tạp $O(k \cdot m)$.

BÀI 6.3	LỊCH THI ĐẤU	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`LICHTHIDAU.CPP`	`LICHTHIDAU.INP`	`LICHTHIDAU.OUT`	1 giây · 256 MB

Kiến thức: *Hệ ràng buộc hiệu, đường đi dài nhất, chu trình dương (Tập 1, Bài 6).*

Ban tổ chức giải bóng đá học sinh cần xếp lịch cho n trận đấu, đánh số từ 1 tới n. Mỗi trận diễn ra vào một ngày là số nguyên dương (ngày 1, ngày 2, ...); nhiều trận có thể cùng một ngày. Có m ràng buộc, mỗi ràng buộc thuộc một trong hai loại:

- **Loại 1** "1 A B c": trận A phải diễn ra sau trận B **ít nhất** c ngày, tức ngày[A] ≥ ngày[B] + c.
- **Loại 2** "2 A B c": trận A **không được muộn hơn** trận B quá c ngày, tức ngày[A] ≤ ngày[B] + c.

Ban tổ chức muốn mỗi trận diễn ra **sớm nhất có thể**. Người ta chứng minh được rằng nếu có lịch hợp lệ thì có một lịch hợp lệ mà **mọi** trận đều đồng thời diễn ra sớm nhất có thể. Hãy in lịch đó.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa bốn số nguyên t, A, B, c ($t \in \{1, 2\}$, $A \neq B$) mô tả một ràng buộc.

Kết quả

Một dòng gồm n số, số thứ i là ngày diễn ra trận i; hoặc −1 nếu không có lịch nào thỏa mãn mọi ràng buộc.

Ràng buộc

$1 \le n \le 1000$; $0 \le m \le 5000$; $0 \le c \le 10^6$.

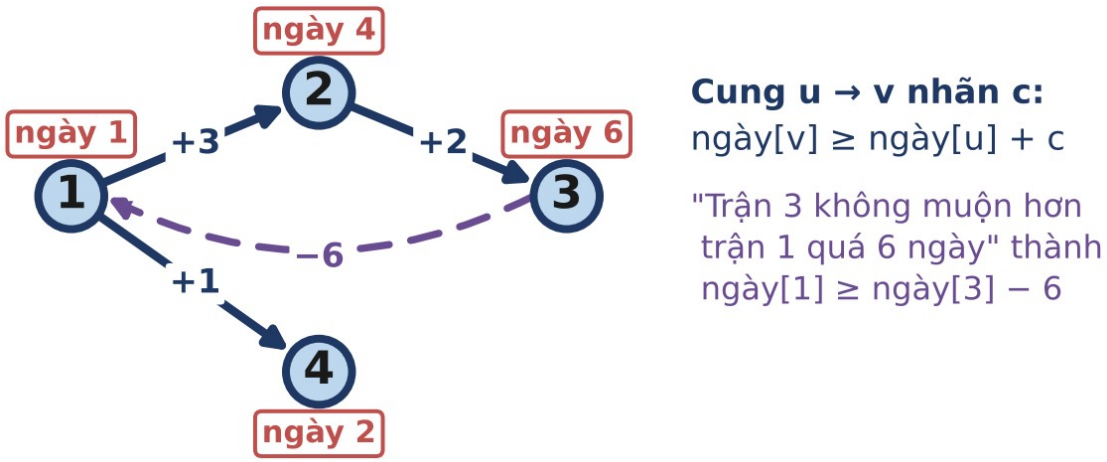
Subtask	Điểm	Ràng buộc thêm
1	40%	Chỉ có ràng buộc loại 1.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

LICHTHIDAU.INP	LICHTHIDAU.OUT	Giải thích

`4 4`
`1 2 1 3`
`1 3 2 2`
`2 3 1 6`
`1 4 1 1`
	`1 4 6 2`	Trận 1 vào ngày 1; trận 2 sau trận 1 ít nhất 3 ngày: ngày 4; trận 3 sau trận 2 ít nhất 2 ngày: ngày 6, và 6 ≤ 1 + 6 thỏa ràng buộc loại 2; trận 4 vào ngày 2.

LICHTHIDAU.INP	LICHTHIDAU.OUT	Giải thích
2 2 1 2 1 3 2 2 1 2	-1	Trận 2 phải sau trận 1 ít nhất 3 ngày nhưng không được muộn hơn trận 1 quá 2 ngày: mâu thuẫn.



Hình 6.3. *Ví dụ 1 dưới dạng đồ thị ràng buộc; ngày của mỗi trận là độ dài đường đi dài nhất (cộng 1).*

CHÚ Ý

Trong đồ thị ràng buộc, một **chu trình có tổng dương** nghĩa là ngày[A] > ngày[A]: vô nghiệm. Bellman-Ford tìm đường đi **dài nhất** cũng phát hiện được chu trình dương sau n lượt.

Ngày có thể lên tới khoảng 10^9; nên dùng `long long` cho an toàn.

HƯỚNG SUY NGHĨ

- Mỗi ràng buộc là một bất đẳng thức dạng ngày[y] ≥ ngày[x] + c: loại 1 cho cung B → A trọng số c, loại 2 cho cung A → B trọng số −c.
- Khởi tạo ngày[v] = 1 cho mọi v rồi lặp: nếu ngày[x] + c > ngày[y] thì tăng ngày[y]. Nếu sau n lượt vẫn còn tăng được thì in −1.
- Vì sao kết quả là "sớm nhất có thể"? Mỗi lần tăng đều bị **ép buộc** bởi một ràng buộc, nên mọi lịch hợp lệ đều phải muộn ít nhất như vậy.

BÀI 7 Thuật toán Floyd

ÔN NHANH KIẾN THỨC · Tập 1, Bài 7

- Floyd: for k, for i, for j: d[i][j] = min(d[i][j], d[i][k] + d[k][j]). Vòng k phải ở **ngoài cùng**. Độ phức tạp $O(n^3)$, hợp với n vài trăm.
- **Bao đóng bắc cầu**: r[i][j] = r[i][j] or (r[i][k] and r[k][j]); dùng `bitset` để nhanh gấp 64 lần.
- Cùng khung vòng lặp, thay (min, +) bằng (max, min) ta được **đường đi có cạnh yếu nhất lớn nhất** giữa mọi cặp đỉnh.
- Khởi tạo: d[i][i] = 0; nhiều cạnh nối cùng một cặp thì giữ cạnh tốt nhất.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
7.1	**THỦ ĐÔ MỚI**	★☆☆ CƠ BẢN	Cài đặt Floyd; xử lí trường hợp bằng nhau.
7.2	**XẾP HẠNG KÌ THỦ**	★★☆ TRUNG BÌNH	Lập luận trên quan hệ "mạnh hơn" có tính bắc cầu.
7.3	**XE TẢI QUÁ KHỔ**	★★★ NÂNG CAO	Tổng quát hóa khung Floyd cho một "phép đo" khác.

BÀI 7.1 THỦ ĐÔ MỚI ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`THUDO.CPP`	`THUDO.INP`	`THUDO.OUT`	1 giây · 256 MB

Kiến thức: *Thuật toán Floyd, khoảng cách giữa mọi cặp đỉnh (Tập 1, Bài 7).*

Một quốc gia có n thành phố, đánh số từ 1 tới n, nối với nhau bởi m con đường hai chiều; con đường thứ i nối u_i với v_i và dài w_i km. Từ một thành phố bất kì luôn đi được tới mọi thành phố khác.

Nhà vua muốn chọn thủ đô mới là thành phố có **tổng khoảng cách** (đường đi ngắn nhất) tới tất cả các thành phố khác **nhỏ nhất**. Nếu có nhiều thành phố như vậy, chọn thành phố có số hiệu nhỏ nhất.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa ba số u, v, w ($u \neq v$) mô tả một con đường; có thể có nhiều con đường nối cùng một cặp thành phố.

Kết quả

Hai số nguyên: số hiệu thành phố được chọn và tổng khoảng cách tương ứng.

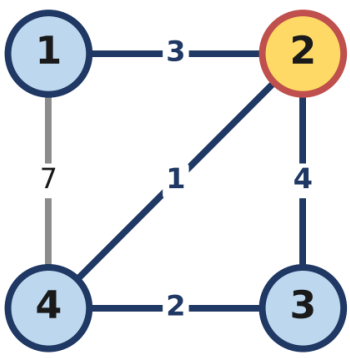
Ràng buộc

$1 \le n \le 400$; $0 \le m \le 10^5$; $1 \le w \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 50$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

THUDO.INP	THUDO.OUT	Giải thích
4 5 1 2 3 2 3 4 3 4 2 1 4 7 2 4 1	2 7	Bảng khoảng cách ở Hình 7.1. Thành phố 2 và 4 cùng có tổng nhỏ nhất bằng 7; chọn thành phố có số hiệu nhỏ hơn là **2**.



	1	2	3	4	Tổng
1	0	3	6	4	13
2	3	0	3	1	7
3	6	3	0	2	11
4	4	1	2	0	7

Hình 7.1. *Ma trận khoảng cách sau khi chạy Floyd và tổng của từng hàng.*

CHÚ Ý

Khi nhiều con đường nối cùng một cặp, chỉ giữ con đường ngắn nhất khi khởi tạo ma trận (dùng `min`, đừng gán đè).

Tổng khoảng cách có thể tới $399 \cdot 399 \cdot 10^6 \approx 1{,}6 \cdot 10^{11}$: dùng `long long`.

HƯỚNG SUY NGHĨ

- Floyd cho ma trận d; đáp số là hàng có tổng nhỏ nhất. Duyệt i tăng dần và chỉ cập nhật khi tổng **nhỏ hơn hẳn** thì tự động chọn được số hiệu nhỏ nhất.

BÀI 7.2	XẾP HẠNG KÌ THỦ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`XEPHANG.CPP`	`XEPHANG.INP`	`XEPHANG.OUT`	1 giây · 256 MB

Kiến thức: *Bao đóng bắc cầu,* `bitset` *(Tập 1, Bài 7).*

Một giải cờ vua có n kì thủ với trình độ **đôi một khác nhau**, đánh số từ 1 tới n. Trong mọi ván, người mạnh hơn luôn thắng. Ban tổ chức chỉ ghi lại được kết quả của m ván đấu, mỗi ván dạng "a b": kì thủ a thắng kì thủ b.

Từ các kết quả này, có những kì thủ mà ta **xác định chắc chắn** được thứ hạng (hạng 1 là người mạnh nhất). Hãy tìm tất cả những kì thủ như vậy.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số a, b ($a \neq b$): a thắng b. Dữ liệu đảm bảo các kết quả không mâu thuẫn nhau.

Kết quả

Dòng đầu là số kì thủ xác định được thứ hạng. Dòng thứ hai liệt kê các kì thủ đó theo thứ tự tăng dần của số hiệu (để trống nếu không có).

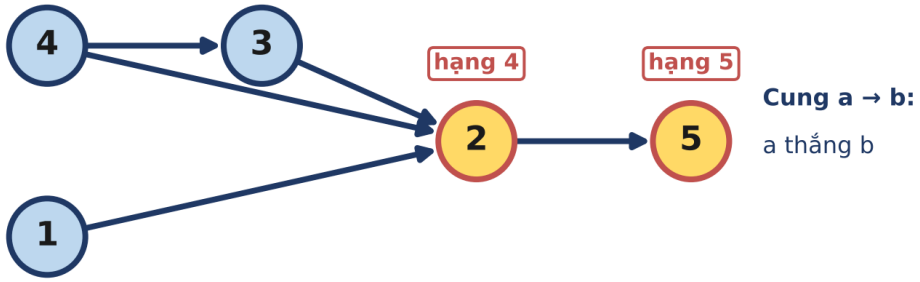
Ràng buộc

$1 \le n \le 2000$; $0 \le m \le 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	40%	$n \le 100$.
2	40%	$n \le 500$.
3	20%	Không có ràng buộc gì thêm.

Ví dụ

XEPHANG.INP	XEPHANG.OUT	Giải thích
5 5 4 3 4 2 3 2 1 2 2 5	2 2 5	Kì thủ 2 thua 1, 3, 4 và thắng 5: biết chắc có 3 người mạnh hơn, 1 người yếu hơn, nên xếp **hạng 4**. Kì thủ 5 thua 2, nên yếu hơn cả 1, 3, 4: **hạng 5**. Không biết 1 với 3, 4 ai mạnh hơn.



Hình 7.2. *Cung a → b nghĩa là a thắng b; chỉ kì thủ 2 và 5 xác định được thứ hạng.*

CHÚ Ý

Floyd với n = 2000 cần $8 \cdot 10^9$ phép tính, quá chậm. Với `bitset<2001>`, phép "hàng i |= hàng k" chỉ tốn khoảng 2000/64 phép tính.

HƯỚNG SUY NGHĨ

- Kì thủ v xác định được thứ hạng khi và chỉ khi với **mọi** kì thủ u khác, ta biết chắc u mạnh hơn hay yếu hơn v, tức là (số người chắc chắn yếu hơn v) + (số người chắc chắn mạnh hơn v) = n − 1.
- "u chắc chắn mạnh hơn v" khi có một dãy kết quả u thắng ... thắng v: đó là bao đóng bắc cầu.

BÀI 7.3	XE TẢI QUÁ KHỔ	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
XETAI.CPP	XETAI.INP	XETAI.OUT	1 giây · 256 MB

Kiến thức: *Floyd với phép toán (max, min) (Tập 1, Bài 7).*

Mạng giao thông có n thị trấn và m con đường hai chiều. Con đường thứ i nối u_i với v_i và chỉ chịu được xe có tải trọng **không quá** c_i tấn.

Công ti vận tải nhận q đơn hàng. Đơn hàng thứ j cần chở từ thị trấn s_j tới thị trấn t_j bằng **một** xe tải, xe chỉ được đi trên những con đường chịu được tải trọng của nó. Với mỗi đơn hàng, hãy cho biết tổng tải trọng (xe + hàng) lớn nhất có thể chở.

Dữ liệu vào

Dòng đầu chứa n, m, q. Mỗi dòng trong m dòng tiếp theo chứa ba số u, v, c ($u \neq v$). Mỗi dòng trong q dòng cuối chứa hai số s, t ($s \neq t$).

Kết quả

Gồm q dòng, mỗi dòng là tải trọng lớn nhất cho đơn hàng tương ứng, hoặc 0 nếu không có đường đi.

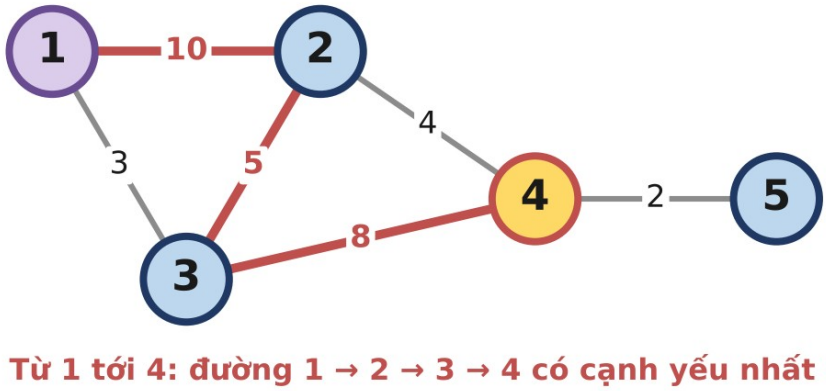
Ràng buộc

$2 \le n \le 500$; $0 \le m \le 10^5$; $1 \le q \le 10^5$; $1 \le c \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 50$, $q \le 50$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

XETAI.INP	XETAI.OUT	Giải thích
5 6 3 1 2 10 2 3 5 1 3 3 3 4 8 4 5 2 2 4 4 1 4 1 5 3 1	5 2 5	Từ 1 tới 4: đường 1 → 2 → 3 → 4 có cạnh yếu nhất là 5 tấn, tốt hơn đường 1 → 2 → 4 (4 tấn) và 1 → 3 → 4 (3 tấn). Muốn tới 5 phải qua con đường 4-5 chỉ chịu 2 tấn. Từ 3 tới 1: đi 3 → 2 → 1, chở được 5 tấn.



Hình 7.3. *Tải trọng của một đường đi bằng tải trọng của con đường yếu nhất trên nó.*

CHÚ Ý

Floyd chạy một lần $O(n^3) = 1{,}25 \cdot 10^8$ phép tính đơn giản, sau đó mỗi đơn hàng trả lời trong O(1). Chạy BFS/Dijkstra lại cho từng đơn hàng sẽ quá chậm.

HƯỚNG SUY NGHĨ

- Gọi b[i][j] là tải trọng lớn nhất từ i tới j. Đi từ i tới j qua k: tải trọng là min(b[i][k], b[k][j]).
- Thay phép "min của tổng" trong Floyd bằng phép "max của min": b[i][j] = max(b[i][j], min(b[i][k], b[k][j])).
- Một lời giải khác (Bài 10): đáp số bằng cạnh nhỏ nhất trên đường đi giữa s và t trong **cây khung lớn nhất**.

BÀI 8 Đồ thị trạng thái và BFS 0-1

ÔN NHANH KIẾN THỨC · Tập 1, Bài 8

- Khi "vị trí" chưa đủ mô tả tình huống, hãy đưa thêm thông tin vào đỉnh: **trạng thái** = (vị trí, thông tin phụ). Số trạng thái = số vị trí × số giá trị của thông tin phụ.
- Mọi cạnh trọng số 1: BFS trên đồ thị trạng thái.
- **BFS 0-1**: cạnh trọng số 0 thì đẩy vào **đầu** hàng đợi hai đầu (`deque`), trọng số 1 thì đẩy vào **cuối**. Độ phức tạp O(số trạng thái + số cạnh).
- Luôn ước lượng số trạng thái trước khi code để chắc chắn đủ bộ nhớ và thời gian.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
8.1	**ĐÈN GIAO THÔNG**	★☆☆ CƠ BẢN	Thêm thời gian (theo chu kì) vào trạng thái.
8.2	**PHÁ TƯỜNG**	★★☆ TRUNG BÌNH	Thêm "tài nguyên đã dùng" vào trạng thái.
8.3	**ROBOT XOAY**	★★★ NÂNG CAO	Đồ thị có trọng số 0 và 1: dùng hàng đợi hai đầu.

BÀI 8.1 ĐÈN GIAO THÔNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DENGIAOTHONG.CPP`	`DENGIAOTHONG.INP`	`DENGIAOTHONG.OUT`	1 giây · 256 MB

Kiến thức: *BFS trên trạng thái (nút, thời điểm mod chu kì) (Tập 1, Bài 8).*

Thành phố có n nút giao và m con đường hai chiều; đi hết mỗi con đường mất đúng 1 phút. Mỗi nút giao có một đèn tín hiệu. Mọi đèn có chung chu kì P phút: đèn ở nút i được mô tả bởi xâu s độ dài P gồm các kí tự `0` (đỏ) và `1` (xanh); ở phút t (t = 0, 1, 2, ...), đèn của nút i có màu ứng với kí tự thứ (t mod P) của s_i (đánh số từ 0).

Xe chỉ được **rời** nút i ở phút t nếu đèn của nút i đang xanh ở phút t; khi đèn đỏ, xe có thể đứng chờ tại chỗ (mỗi lần chờ 1 phút). Xe tới một nút vào lúc nào cũng được, không phụ thuộc màu đèn.

Một xe cứu thương ở nút 1 tại phút 0 cần tới nút n sớm nhất có thể. Hãy cho biết thời điểm sớm nhất.

Dữ liệu vào

Dòng đầu chứa n, m, P. Dòng thứ i trong n dòng tiếp theo là xâu s_i. Mỗi dòng trong m dòng cuối chứa hai số u, v ($u \neq v$) mô tả một con đường.

Kết quả

Thời điểm sớm nhất xe tới nút n, hoặc −1 nếu không thể tới.

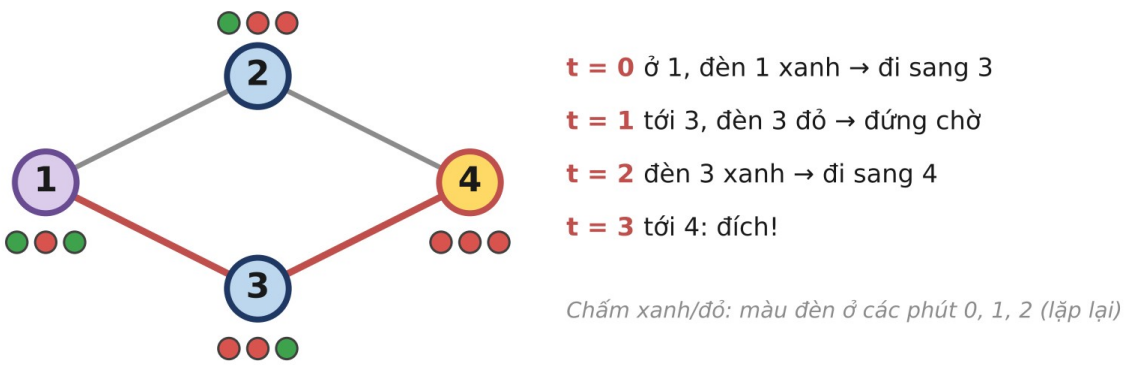
Ràng buộc

$1 \le n \le 10^4$; $0 \le m \le 2 \cdot 10^4$; $1 \le P \le 60$.

Subtask	Điểm	Ràng buộc thêm
1	30%	P = 1 (mỗi đèn luôn xanh hoặc luôn đỏ).
2	30%	n ≤ 100.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DENGIAOTHONG.INP	DENGIAOTHONG.OUT	Giải thích
4 4 3 101 100 001 000 1 2 2 4 1 3 3 4	3	Phút 0 đèn nút 1 xanh, xe đi sang nút 3, tới lúc 1. Đèn nút 3 đỏ ở phút 1, xe chờ; phút 2 đèn xanh, xe đi sang nút 4, tới lúc **3** (Hình 8.1). Nếu đi qua nút 2: tới lúc 1 nhưng đèn nút 2 chỉ xanh ở các phút 0, 3, 6, ... nên tới nút 4 lúc 4.



Hình 8.1. *Trạng thái của xe là (nút, phút mod P); đứng chờ cũng là một bước đi.*

> **CHÚ Ý**
>
> Nếu chỉ lưu "đã thăm nút u" thì sai: tới nút u ở hai thời điểm khác nhau (theo mod P) là hai tình huống khác nhau.

HƯỚNG SUY NGHĨ

- Trạng thái (u, r) với r = t mod P; có n · P ≤ $6 \cdot 10^5$ trạng thái. Từ (u, r): chờ sang (u, (r + 1) mod P); nếu $s_u[r] = 1$ thì đi sang (v, (r + 1) mod P) với mọi v kề u.
- Mọi bước đều tốn 1 phút nên BFS cho thời điểm sớm nhất. Trạng thái đầu tiên có u = n được lấy ra là đáp số.

BÀI 8.2	PHÁ TƯỜNG	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
PHATUONG.CPP	**PHATUONG.INP**	**PHATUONG.OUT**	**1 giây · 256 MB**

Kiến thức: *BFS trên trạng thái (ô, số tường đã phá) (Tập 1, Bài 8).*

Một mê cung là lưới R × C gồm các ô trống (`.`), ô tường (`#`), một ô xuất phát (`S`) và một ô đích (`T`); ô `S` và ô `T` đều là ô trống. Mỗi bước, Minh đi sang một ô kề cạnh.

Minh mang theo một chiếc búa có thể dùng **không quá k lần**: mỗi lần dùng búa, Minh phá một bức tường ở ô kề rồi bước vào ô đó (bước này cũng tính là một bước). Hỏi Minh cần ít nhất bao nhiêu bước để đi từ `S` tới `T`?

Dữ liệu vào

Dòng đầu chứa R, C, k. R dòng tiếp theo mô tả mê cung.

Kết quả

Số bước ít nhất, hoặc −1 nếu không thể tới `T`.

Ràng buộc

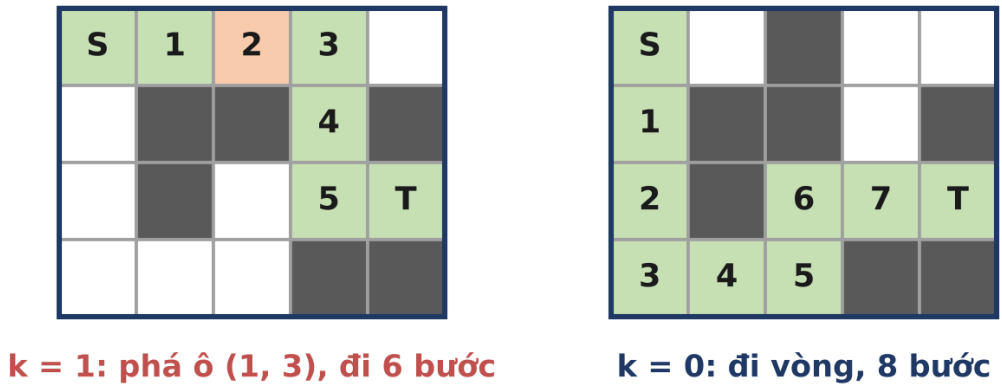
$1 \le R, C \le 100$; $0 \le k \le 10$; có đúng một ô `S` và một ô `T`.

Subtask	Điểm	Ràng buộc thêm
1	30%	k = 0.
2	30%	k = 1.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

PHATUONG.INP	PHATUONG.OUT	Giải thích
4 5 1 S.#.. .##.# .#..T ...##	6	Phá bức tường ở ô (1, 3) rồi đi thẳng xuống: **6** bước (Hình 8.2, trái).

PHATUONG.INP	PHATUONG.OUT	Giải thích
4 5 0 S.#.. .##.# .#..T ...##	8	Cùng mê cung nhưng không có búa: phải đi vòng xuống dưới, mất **8** bước (Hình 8.2, phải).



Hình 8.2. *Số trong ô là thứ tự bước đi; ô cam là bức tường bị phá.*

CHÚ Ý

Đánh dấu "đã thăm" theo ô là **sai**: có thể tới một ô bằng đường ngắn nhưng đã dùng hết búa, trong khi một đường dài hơn tới cùng ô ấy còn búa để phá tường phía trước.

HƯỚNG SUY NGHĨ

- Trạng thái (x, y, j): đang ở ô (x, y) và đã phá j bức tường. Có $R \cdot C \cdot (k + 1) \le 1{,}1 \cdot 10^5$ trạng thái.
- Bước vào ô trống: giữ nguyên j; bước vào ô tường: j tăng 1 (chỉ khi $j < k$). Mọi bước tốn 1 nên dùng BFS.

BÀI 8.3	ROBOT XOAY	★★★ NÂNG CAO	
Chương trình **ROBOTXOAY.CPP**	Dữ liệu vào **ROBOTXOAY.INP**	Kết quả **ROBOTXOAY.OUT**	Giới hạn **2 giây · 256 MB**

Kiến thức: *BFS 0-1 trên trạng thái (ô, hướng) (Tập 1, Bài 8).*

Một robot hút bụi được đặt trong căn phòng là lưới R × C gồm ô trống (`.`), ô có đồ đạc (`#`), ô xuất phát (`S`) và ô đích (`T`). Robot luôn quay mặt về một trong bốn hướng: Bắc `N` (lên trên), Đông `E` (sang phải), Nam `S` (xuống dưới), Tây `W` (sang trái).

Robot có hai loại thao tác: **tiến** một ô theo hướng đang quay mặt (ô đó phải trống và nằm trong phòng) và **xoay** 90° sang trái hoặc sang phải tại chỗ. Tiến thì không tốn pin, nhưng mỗi lần xoay tốn 1 đơn vị pin. Hãy tìm số đơn vị pin ít nhất để robot đi từ `S` tới `T` (tới `T` theo hướng nào cũng được).

Dữ liệu vào

Dòng đầu chứa R, C và một kí tự h là hướng ban đầu của robot. R dòng tiếp theo mô tả căn phòng.

Kết quả

Số lần xoay ít nhất, hoặc −1 nếu robot không thể tới `T`.

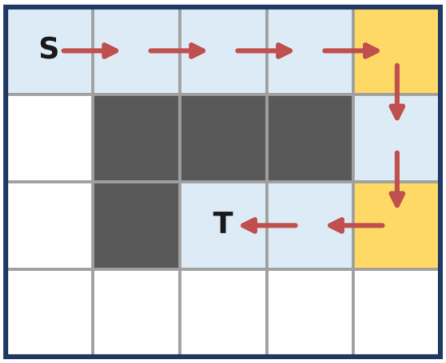
Ràng buộc

$1 \le R, C \le 1000$; có đúng một ô `S` và một ô `T`.

Subtask	Điểm	Ràng buộc thêm
1	30%	$R, C \le 50$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

ROBOTXOAY.INP	ROBOTXOAY.OUT	Giải thích
4 5 E S.... .###. .#T..	2	Robot hướng Đông, tiến tới ô (1, 5), xoay sang Nam, tiến tới ô (3, 5), xoay sang Tây và tiến tới T. Tổng cộng **2** lần xoay (Hình 8.3).



Hình 8.3. *Đường đi của robot; ô vàng là nơi robot xoay.*

CHÚ Ý

BFS thường (coi mọi thao tác tốn 1) cho số **thao tác** ít nhất, không phải số lần xoay ít nhất.

Có tới $4 \cdot 10^6$ trạng thái: dùng mảng một chiều thay cho `map`, và dùng `deque`, không dùng `priority_queue` nếu muốn thật nhanh.

HƯỚNG SUY NGHĨ

- Trạng thái (x, y, hướng). Tiến: cạnh trọng số 0 tới (x + dx, y + dy, hướng). Xoay: cạnh trọng số 1 tới (x, y, hướng ± 90°).
- BFS 0-1: cạnh 0 đẩy vào đầu `deque`, cạnh 1 đẩy vào cuối. Khi lấy ra một trạng thái ở ô T, đó là đáp số.

CHƯƠNG 3. CÂY VÀ CÂY KHUNG

Cây là đồ thị "tiết kiệm" nhất: liên thông với số cạnh ít nhất. Chương này luyện ba công cụ đi kèm với cây: DSU để quản lí các nhóm đang lớn dần, cây khung nhỏ nhất để chọn ra bộ khung rẻ nhất, và nhảy nhị phân để trả lời nhanh các câu hỏi về đường đi trên cây.

TRONG CHƯƠNG NÀY

Bài 9. Cấu trúc các tập rời nhau (DSU)

- **9.1 MẠNG BẠN BÈ** — ★☆☆ CƠ BẢN
- **9.2 PHÁ CẦU** — ★★☆ TRUNG BÌNH
- **9.3 HAI PHE** — ★★★ NÂNG CAO

Bài 10. Cây khung nhỏ nhất: Kruskal và Prim

- **10.1 NỐI MẠNG TRƯỜNG HỌC** — ★☆☆ CƠ BẢN
- **10.2 ĐƯỜNG ÍT DỐC** — ★★☆ TRUNG BÌNH
- **10.3 GIẾNG VÀ ỐNG** — ★★★ NÂNG CAO

Bài 11. Cây có gốc, tổ tiên chung gần nhất (LCA)

- **11.1 HỌ HÀNG** — ★☆☆ CƠ BẢN
- **11.2 CÁP QUANG YẾU NHẤT** — ★★☆ TRUNG BÌNH
- **11.3 NHÂN VIÊN CẤP K** — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Khi đề bài có các thao tác "gộp nhóm", "kết nối", hãy nghĩ ngay tới DSU. Khi có thao tác "xóa", thử đi ngược thời gian.
- Cây khung nhỏ nhất không chỉ tối ưu tổng trọng số mà còn tối ưu **cạnh lớn nhất** trên mọi đường đi.
- Bảng nhảy nhị phân có thể mang thêm thông tin (min, max, tổng) của đoạn đường 2^j bước.

BÀI 9 Cấu trúc các tập rời nhau (DSU)

ÔN NHANH KIẾN THỨC · Tập 1, Bài 9

- DSU quản lí các nhóm rời nhau: `find(x)` tìm đại diện (gốc) của nhóm chứa x, `unite(a, b)` gộp hai nhóm. Với **nén đường đi** và **gộp theo kích thước**, mỗi thao tác gần như O(1).
- Có thể gắn thêm thông tin vào gốc: kích thước nhóm sz, số nhóm cnt (giảm 1 mỗi lần gộp thành công)...
- DSU chỉ **gộp** được, không **tách** được. Bài toán xóa dần thường được giải bằng cách **đi ngược thời gian**: xóa thành thêm.
- **DSU có chẵn lẻ**: lưu thêm w[x] = "x cùng phe hay khác phe với cha"; khi tìm gốc thì cộng dồn (XOR) dọc đường đi.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
9.1	**MẠNG BẠN BÈ**	★☆☆ CƠ BẢN	Cài đặt DSU chuẩn, duy trì thông tin theo từng bước.
9.2	**PHÁ CẦU**	★★☆ TRUNG BÌNH	Đảo ngược thứ tự thao tác để biến "xóa" thành "thêm".
9.3	**HAI PHE**	★★★ NÂNG CAO	Gắn thông tin quan hệ vào cạnh của cây DSU.

BÀI 9.1 MẠNG BẠN BÈ ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`MANGBANBE.CPP`	`MANGBANBE.INP`	`MANGBANBE.OUT`	1 giây · 256 MB

Kiến thức: *DSU với kích thước nhóm và số nhóm (Tập 1, Bài 9).*

Một trường học mới có n học sinh, đánh số từ 1 tới n; lúc đầu chưa ai quen ai. Trong năm học lần lượt diễn ra m sự kiện, sự kiện thứ i là hai bạn u_i và v_i **kết bạn**. Hai bạn thuộc cùng một **nhóm bạn** nếu có một dãy quan hệ bạn bè nối họ (bạn của bạn cũng cùng nhóm).

Sau mỗi sự kiện, nhà trường muốn biết lúc đó có bao nhiêu nhóm bạn, và nhóm đông nhất có bao nhiêu người.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$) mô tả một sự kiện; một cặp có thể kết bạn nhiều lần.

Kết quả

Gồm m dòng, dòng thứ i chứa hai số: số nhóm bạn và số người của nhóm đông nhất sau sự kiện thứ i.

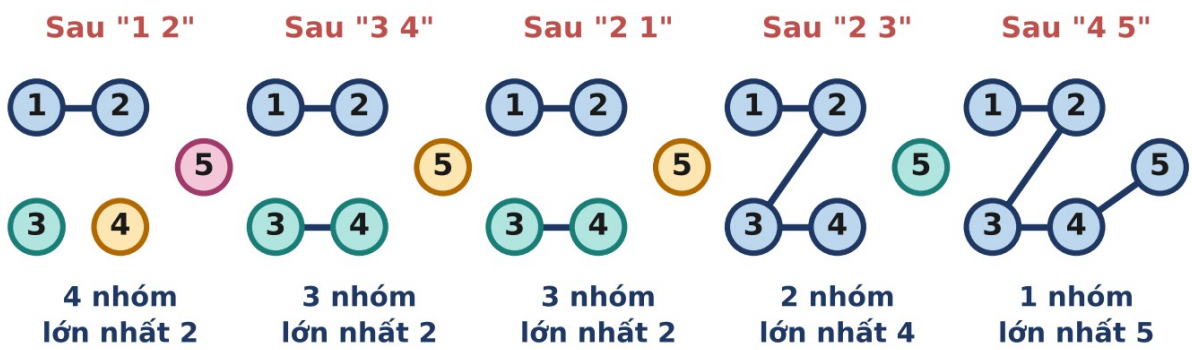
Ràng buộc

$1 \le n, m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	40%	n, m ≤ 1000.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

MANGBANBE.INP	MANGBANBE.OUT	Giải thích
5 5 1 2 3 4 2 1 2 3 4 5	4 2 3 2 3 2 2 4 1 5	Diễn biến được vẽ ở Hình 9.1. Sự kiện "2 1": hai bạn đã cùng nhóm nên không có gì thay đổi. Sự kiện "2 3" gộp hai nhóm 2 người thành nhóm 4 người.



Hình 9.1. *Các nhóm bạn sau từng sự kiện; mỗi nhóm một màu.*

CHÚ Ý

Đừng chạy lại BFS/DFS sau mỗi sự kiện: tổng cộng $O(m \cdot (n + m))$ là quá chậm.

Khi gộp, chỉ giảm số nhóm nếu hai bạn **chưa** cùng nhóm.

HƯỚNG SUY NGHĨ

- Lưu cnt (số nhóm, ban đầu bằng n) và best (kích thước lớn nhất, ban đầu bằng 1). Mỗi lần gộp thành công hai gốc a, b: cnt giảm 1, sz[a] += sz[b], best = max(best, sz[a]).
- Vì nhóm chỉ lớn lên chứ không bao giờ bị tách, best không bao giờ giảm.

BÀI 9.2	PHÁ CẦU	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
PHACAU.CPP	**PHACAU.INP**	**PHACAU.OUT**	**1 giây · 256 MB**

Kiến thức: *DSU, xử lí ngoại tuyến bằng cách đi ngược thời gian (Tập 1, Bài 9).*

Một quần đảo có n hòn đảo, đánh số từ 1 tới n, và m cây cầu, cây cầu thứ i nối hai đảo u_i và v_i. Hai đảo **cùng nhóm** nếu đi được từ đảo này tới đảo kia qua các cây cầu.

Một cơn bão lớn sắp đổ bộ. Theo dự báo, q cây cầu sẽ lần lượt bị sập, cây cầu sập thứ j là cây cầu số k_j (các k_j đôi một khác nhau). Đội cứu hộ cần biết ngay sau mỗi lần sập cầu, quần đảo bị chia thành bao nhiêu nhóm.

Dữ liệu vào

Dòng đầu chứa n, m, q. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v mô tả một cây cầu. Dòng cuối chứa q số k_1, k_2, ..., k_q.

Kết quả

Gồm q dòng, dòng thứ j là số nhóm đảo ngay sau khi cây cầu k_j sập.

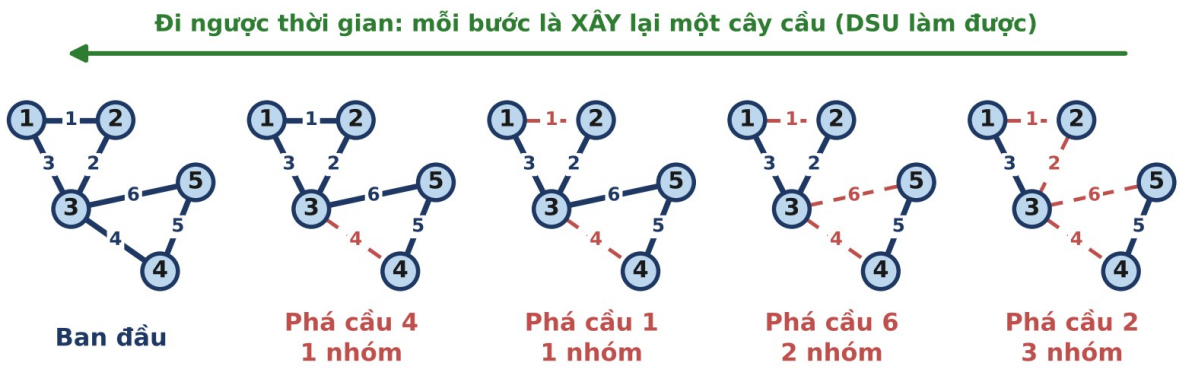
Ràng buộc

$1 \le n, m \le 2 \cdot 10^5$; $1 \le q \le m$.

Subtask	Điểm	Ràng buộc thêm
1	30%	n, m ≤ 1000.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

PHACAU.INP	PHACAU.OUT	Giải thích
5 6 4 1 2 2 3 3 1 3 4 4 5 5 3 4 1 6 2	1 1 2 3	Cầu 4 sập: đảo 4 vẫn nối với 3 qua đảo 5. Cầu 1 sập: đảo 1 vẫn nối với 2 qua đảo 3. Cầu 6 sập: nhóm {4, 5} bị tách ra. Cầu 2 sập: đảo 2 bị cô lập. Kết quả 1, 1, 2, 3 (Hình 9.2).



Hình 9.2. *Cầu nét đứt đỏ đã sập; số trên mỗi cây cầu là số hiệu của nó.*

CHÚ Ý

Trạng thái sau khi sập **cả q** cây cầu là điểm xuất phát tốt nhất: dựng DSU với các cây cầu không bao giờ sập.

Nhớ rằng đáp số cần in là số nhóm **sau** mỗi lần sập, nên khi đi ngược phải ghi đáp số **trước** khi xây lại cây cầu.

HƯỚNG SUY NGHĨ

- Đi ngược: bắt đầu từ trạng thái cuối cùng (đã sập hết q cầu). Với j = q, q − 1, ..., 1: ghi lại số nhóm hiện tại làm đáp số thứ j, rồi **xây lại** cây cầu k_j.
- Cuối cùng in các đáp số theo thứ tự j = 1, 2, ..., q.

BÀI 9.3	HAI PHE	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
HAIPHE.CPP	**HAIPHE.INP**	**HAIPHE.OUT**	**1 giây · 256 MB**

Kiến thức: *DSU có tính chẵn lẻ (Tập 1, Bài 9).*

Trong một trò chơi nhập vai, n nhân vật được chia thành **hai phe** đối địch, nhưng người chơi không biết ai thuộc phe nào. Người chơi lần lượt thu thập m mẩu thông tin; mẩu thứ i có dạng "a b t": nếu t = 0 thì a và b **cùng phe**, nếu t = 1 thì a và b **khác phe**.

Có thể có một mẩu thông tin là giả, mâu thuẫn với các thông tin trước đó. Hãy tìm mẩu thông tin **đầu tiên** mà khi kết hợp với tất cả các mẩu trước nó thì không còn cách chia phe nào phù hợp.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa ba số a, b, t ($a \neq b$, $t \in \{0, 1\}$).

Kết quả

Số thứ tự của mẩu thông tin mâu thuẫn đầu tiên, hoặc 0 nếu mọi thông tin đều phù hợp.

Ràng buộc

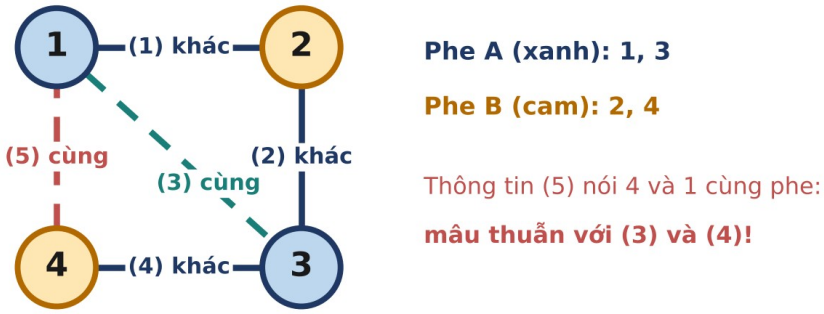
$2 \le n \le 2 \cdot 10^5$; $1 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Mọi mẩu thông tin đều có t = 1.
2	30%	n, m ≤ 2000.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

HAIPHE.INP	HAIPHE.OUT	Giải thích
4 5 1 2 1 2 3 1 1 3 0 3 4 1 4 1 0	5	Bốn mẩu đầu xác định phe: {1, 3} và {2, 4}. Mẩu 5 nói 4 và 1 cùng phe: mâu thuẫn (Hình 9.3).

HAIPHE.INP	HAIPHE.OUT	Giải thích
3 3 1 2 1 2 3 1 1 3 1	3	1 khác 2, 2 khác 3 nên 1 cùng phe 3; mẩu 3 lại nói 1 khác 3.



Hình 9.3. *Ví dụ 1. Nét liền: khác phe; nét đứt: cùng phe.*

CHÚ Ý

Chỉ quan tâm tới mâu thuẫn **đầu tiên**; các mẩu thông tin sau đó không cần đọc kĩ (nhưng vẫn phải đọc hết dữ liệu nếu chương trình của em cần).

DSU có chẵn lẻ cài đặt bằng đệ quy có thể bị tràn ngăn xếp nếu không gộp theo kích thước; hãy cẩn thận.

HƯỚNG SUY NGHĨ

- Subtask 1: mọi thông tin đều là "khác phe"; mâu thuẫn xuất hiện khi tạo thành chu trình lẻ. Có thể làm bằng DSU trên 2n đỉnh: "a ở phe A" và "a ở phe B".
- Tổng quát: lưu p[x] (cha) và w[x] $\in \{0, 1\}$ (x có khác phe với cha không). Chẵn lẻ từ x tới gốc là XOR các w trên đường đi.
- Mẩu "a b t": nếu a, b cùng gốc thì kiểm tra par(a) XOR par(b) = t; nếu khác gốc ra, rb thì nối ra vào rb với w[ra] = par(a) XOR par(b) XOR t.

BÀI 10 Cây khung nhỏ nhất: Kruskal và Prim

ÔN NHANH KIẾN THỨC · Tập 1, Bài 10

- **Kruskal**: sắp xếp cạnh tăng dần, lần lượt nhận cạnh nếu hai đầu chưa cùng nhóm (dùng DSU). O(m log m).
- **Tính chất lát cắt**: cạnh nhẹ nhất đi qua một lát cắt luôn thuộc một cây khung nhỏ nhất.
- Trong cây khung nhỏ nhất, đường đi giữa u và v có **cạnh lớn nhất nhỏ nhất** trong mọi đường đi giữa u và v của đồ thị ban đầu.
- Kĩ thuật **đỉnh ảo**: thêm một đỉnh đặc biệt nối tới mọi đỉnh để mô hình hóa lựa chọn "tự làm lấy".

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
10.1	**NỐI MẠNG TRƯỜNG HỌC**	★☆☆ CƠ BẢN	Khởi tạo DSU từ hiện trạng trước khi chạy Kruskal.
10.2	**ĐƯỜNG ÍT DỐC**	★★☆ TRUNG BÌNH	Biến câu hỏi trên đồ thị thành câu hỏi trên cây khung.
10.3	**GIẾNG VÀ ỐNG**	★★★ NÂNG CAO	Mô hình hóa lựa chọn "tự làm" bằng một đỉnh ảo.

BÀI 10.1 NỐI MẠNG TRƯỜNG HỌC ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NOIMANG.CPP`	`NOIMANG.INP`	`NOIMANG.OUT`	1 giây · 256 MB

Kiến thức: *Thuật toán Kruskal với các cạnh có sẵn (Tập 1, Bài 10).*

Sở Giáo dục muốn nối mạng cho n trường học, đánh số từ 1 tới n, sao cho hai trường bất kì đều liên lạc được với nhau (trực tiếp hoặc qua các trường khác). Hiện đã có k đường cáp **có sẵn**, không mất tiền.

Có m phương án lắp thêm cáp, phương án thứ i nối trường u_i với trường v_i và tốn c_i triệu đồng. Hãy tính chi phí lắp thêm ít nhất.

Dữ liệu vào

Dòng đầu chứa n, k, m. Mỗi dòng trong k dòng tiếp theo chứa hai số u, v là một đường cáp có sẵn. Mỗi dòng trong m dòng cuối chứa ba số u, v, c là một phương án.

Kết quả

Chi phí ít nhất, hoặc −1 nếu không thể nối mạng cho mọi trường.

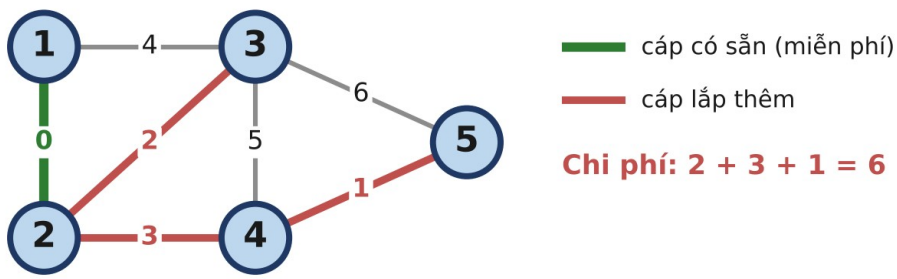
Ràng buộc

$1 \le n \le 10^5$; $0 \le k \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $1 \le c \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	k = 0, n ≤ 1000, m ≤ 5000.
2	30%	k = 0.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

NOIMANG.INP	NOIMANG.OUT	Giải thích
5 1 6 1 2 1 3 4 2 3 2 3 4 5 2 4 3 4 5 1 3 5 6	6	Trường 1 và 2 đã nối sẵn. Kruskal nhận lần lượt các cáp 4-5 (1), 2-3 (2), 2-4 (3); cáp 1-3 (4) bị bỏ vì 1 và 3 đã liên lạc được. Tổng **6** (Hình 10.1).



Hình 10.1. *Cáp xanh có sẵn được "nhận" trước, với chi phí 0.*

CHÚ Ý

Chi phí có thể tới khoảng 10^{14}: dùng `long long`.

Đừng quên kiểm tra mạng cuối cùng có liên thông không (số nhóm bằng 1).

HƯỚNG SUY NGHĨ

- Gộp các cặp trường nối bằng cáp có sẵn vào DSU trước, rồi chạy Kruskal bình thường trên m phương án.
- Cách nhìn khác: coi mỗi cáp có sẵn là một phương án có giá 0.

BÀI 10.2	ĐƯỜNG ÍT DỐC	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`ITDOC.CPP`	`ITDOC.INP`	`ITDOC.OUT`	1 giây · 256 MB

Kiến thức: *Tính chất minimax của cây khung nhỏ nhất, đường đi trên cây (Tập 1, Bài 10–11).*

Vùng núi có n bản làng và m con đường mòn hai chiều; con đường thứ i nối u_i với v_i và có độ dốc h_i. Với một lộ trình, độ khó của lộ trình là độ dốc **lớn nhất** trong các con đường nó đi qua.

Có q đoàn khách du lịch, đoàn thứ j muốn đi từ bản s_j tới bản t_j theo lộ trình có độ khó **nhỏ nhất**. Hãy cho biết độ khó ấy với mỗi đoàn.

Dữ liệu vào

Dòng đầu chứa n, m, q. Mỗi dòng trong m dòng tiếp theo chứa ba số u, v, h ($u \neq v$). Mỗi dòng trong q dòng cuối chứa hai số s, t.

Kết quả

Gồm q dòng; dòng thứ j là độ khó nhỏ nhất của lộ trình cho đoàn thứ j, hoặc −1 nếu không có lộ trình. Nếu s = t thì in 0.

Ràng buộc

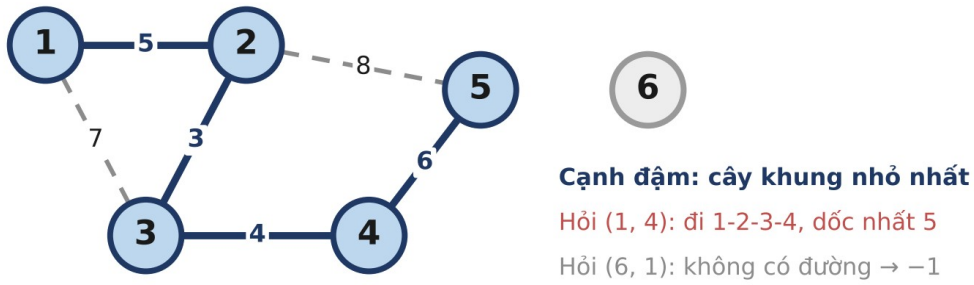
$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $1 \le q \le 10^5$; $1 \le h \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	n, m, q ≤ 1000.
2	30%	q ≤ 1000.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

ITDOC.INP	ITDOC.OUT	Giải thích

`6 6 4`
`1 2 5`
`2 3 3`
`1 3 7`
`3 4 4`
`4 5 6`
`2 5 8`
`1 4`
`1 5`
`2 3`
`6 1`
	`5`
	`6`
	`3`
	`-1`
		(1, 4): đi 1-2-3-4 có độ dốc 5, 3, 4, độ khó **5**; đường 1-3-4 khó 7.
		(1, 5): đi 1-2-3-4-5, độ khó 6 (đường 1-2-5 khó 8).
		(2, 3): con đường trực tiếp, độ khó 3.
		(6, 1): bản 6 không có đường mòn nào: −1.



Hình 10.2. *Mọi lộ trình tốt nhất đều nằm trên cây khung nhỏ nhất (cạnh đậm).*

CHÚ Ý

Đừng chạy lại thuật toán cho từng đoàn khách nếu q lớn.

Đồ thị có thể không liên thông; khi đó ta có **rừng** khung nhỏ nhất.

HƯỚNG SUY NGHĨ

- Subtask 1: với mỗi câu hỏi, sắp xếp cạnh và thêm dần vào DSU cho tới khi s, t cùng nhóm; cạnh vừa thêm là đáp số.
- Dựng cây khung nhỏ nhất. Đáp số cho (s, t) là cạnh lớn nhất trên **đường đi trong cây** từ s tới t. Subtask 2: DFS từ s cho mỗi câu hỏi, O(n · q).
- Subtask 3: nhảy nhị phân như ở Bài 11, lưu thêm mx[v][j] là cạnh lớn nhất trên đoạn đường nhảy 2^j bước.

BÀI 10.3	GIẾNG VÀ ỐNG	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`GIENGONG.CPP`	`GIENGONG.INP`	`GIENGONG.OUT`	1 giây · 256 MB

Kiến thức: *Cây khung nhỏ nhất với đỉnh ảo (Tập 1, Bài 10).*

Một ngôi làng có n ngôi nhà chưa có nước sạch. Với nhà thứ i, có thể **đào giếng** ngay trong nhà với chi phí a_i. Ngoài ra có m phương án **nối ống**: phương án thứ j nối nhà u_j với nhà v_j với chi phí c_j; nước chảy được qua ống theo cả hai chiều.

Một nhà có nước nếu nó có giếng, hoặc được nối (qua một dãy ống) tới một nhà có giếng. Hãy tính tổng chi phí ít nhất để mọi nhà đều có nước.

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ hai chứa n số a_1, ..., a_n. Mỗi dòng trong m dòng cuối chứa ba số u, v, c ($u \neq v$).

Kết quả

Tổng chi phí ít nhất.

Ràng buộc

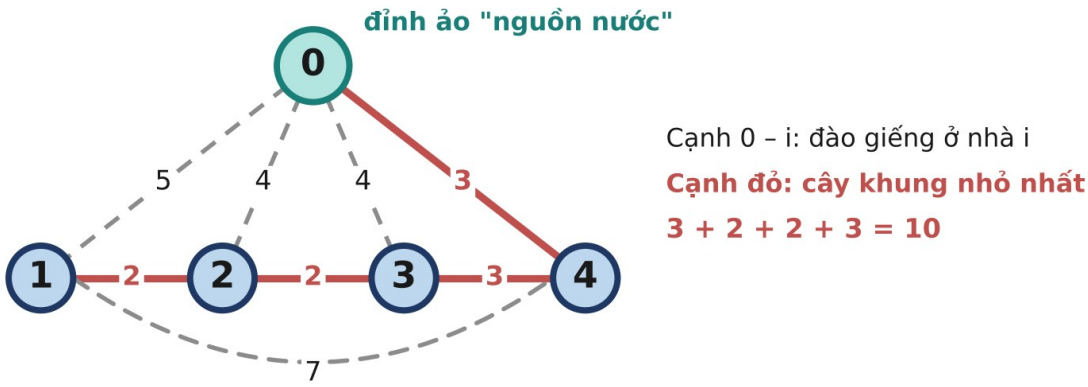
$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $1 \le a_i, c_j \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 10$, $m \le 10$.
2	30%	$n \le 1000$, $m \le 5000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

GIENGONG.INP	GIENGONG.OUT	Giải thích

`4 4`
`5 4 4 3`
`1 2 2`
`2 3 2`
`3 4 3`
`1 4 7`
	`10`	Đào một giếng ở nhà 4 (chi phí 3) và nối ống 1-2, 2-3, 3-4 (chi phí 2 + 2 + 3). Tổng **10** (Hình 10.3).
		Đào giếng ở cả 4 nhà tốn 16; nối ống 1-4 thì quá đắt.



Hình 10.3. *Thêm đỉnh ảo 0; đào giếng ở nhà i tương đương với cạnh (0, i) có trọng số a_i.*

CHÚ Ý

Cần **ít nhất một** giếng, nhưng có thể cần nhiều giếng (khi các nhà xa nhau hoặc không có ống nối).

Tổng chi phí có thể tới khoảng 10^{14}: dùng `long long`.

HƯỚNG SUY NGHĨ

- Thêm đỉnh 0 tượng trưng cho "nguồn nước", nối 0 với nhà i bằng cạnh trọng số a_i.
- Mọi nhà đều có nước khi và chỉ khi đồ thị gồm n + 1 đỉnh (kể cả đỉnh 0) liên thông. Đáp số là trọng số cây khung nhỏ nhất của đồ thị mới.

BÀI 11 Cây có gốc, tổ tiên chung gần nhất (LCA)

ÔN NHANH KIẾN THỨC · Tập 1, Bài 11

- Với cây có gốc: depth[v] là độ sâu, up[v][j] là tổ tiên thứ 2^j của v; up[v][j] = up[up[v][j − 1]][j − 1]. Bảng có n · log n phần tử.
- **LCA(u, v)**: nâng đỉnh sâu hơn lên cùng độ sâu, rồi nhảy cả hai lên với j giảm dần khi up khác nhau. O(log n) mỗi câu hỏi.
- Khoảng cách trên cây: dist(u, v) = depth[u] + depth[v] − 2 · depth[LCA(u, v)].
- Tổ tiên thứ k: phân tích k thành tổng các lũy thừa của 2 và nhảy theo từng bit.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
11.1	**HỌ HÀNG**	★☆☆ CƠ BẢN	Cài đặt LCA chuẩn; tìm gốc của cây.
11.2	**CÁP QUANG YẾU NHẤT**	★★☆ TRUNG BÌNH	Gắn thêm thông tin tổng hợp vào bảng nhảy nhị phân.
11.3	**NHÂN VIÊN CẤP K**	★★★ NÂNG CAO	Đổi cách trả lời câu hỏi khi bộ nhớ không đủ cho bảng nhảy nhị phân.

BÀI 11.1 HỌ HÀNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`HOHANG.CPP`	`HOHANG.INP`	`HOHANG.OUT`	1 giây · 256 MB

Kiến thức: *LCA bằng nhảy nhị phân, độ sâu trên cây (Tập 1, Bài 11).*

Gia phả của một dòng họ có n người, đánh số từ 1 tới n. Mỗi người, trừ **ông tổ** của cả dòng họ, có đúng một người cha cũng thuộc gia phả. Ông tổ ở **đời 0**, con của ông ở đời 1, cháu ở đời 2, và cứ thế tiếp tục.

Có q câu hỏi, mỗi câu hỏi gồm hai người u, v. Hãy cho biết **ông tổ chung gần nhất** của u và v (người có đời lớn nhất mà vừa là tổ tiên của u, vừa là tổ tiên của v; mỗi người được coi là tổ tiên của chính mình), và hai người chênh nhau bao nhiêu đời.

Dữ liệu vào

Dòng đầu chứa n. Dòng thứ hai chứa n số p_1, ..., p_n: p_i là cha của người i, riêng ông tổ có $p_i = 0$. Dòng thứ ba chứa q. Mỗi dòng trong q dòng cuối chứa hai số u, v.

Kết quả

Gồm q dòng, mỗi dòng hai số: ông tổ chung gần nhất và số đời chênh lệch.

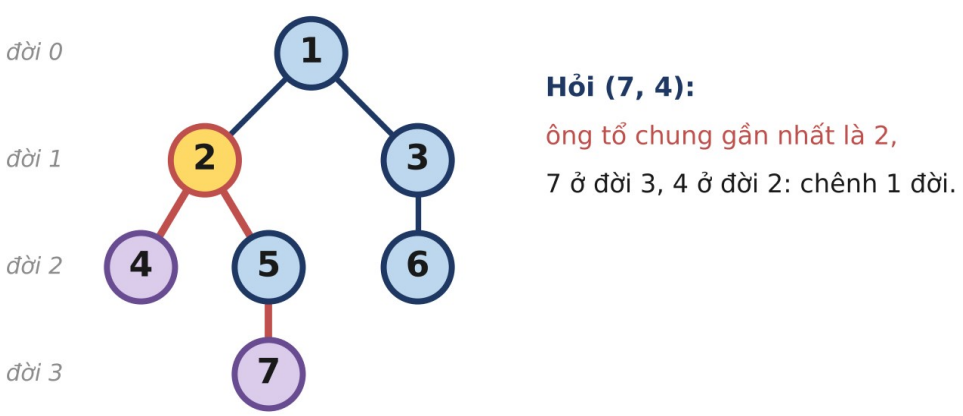
Ràng buộc

$1 \le n, q \le 2 \cdot 10^5$. Ông tổ không nhất thiết là người số 1.

Subtask	Điểm	Ràng buộc thêm
1	40%	$n, q \le 1000$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

HOHANG.INP	HOHANG.OUT	Giải thích
7 0 1 1 2 2 3 5 4 7 4 7 6 5 7 6 6	2 1 1 1 5 1 6 0	(7, 4): tổ tiên chung gần nhất là 2; 7 ở đời 3, 4 ở đời 2 (Hình 11.1). (7, 6): chỉ gặp nhau ở ông tổ 1. (5, 7): 5 là cha của 7. (6, 6): chính là 6, chênh 0 đời.



Hình 11.1. *Gia phả của ví dụ; đỉnh vàng là ông tổ chung gần nhất của 7 và 4.*

CHÚ Ý

Gia phả có thể là một "đường thẳng" $2 \cdot 10^5$ đời: DFS đệ quy để tính độ sâu có thể tràn ngăn xếp. Hãy dùng BFS hoặc ngăn xếp tự quản lí.

HƯỚNG SUY NGHĨ

- Tìm ông tổ ($p_i = 0$), dựng danh sách con, BFS để tính depth và up[v][0] = p_v.
- Subtask 1: cho u, v lần lượt leo lên cha tới khi gặp nhau, O(n) mỗi câu hỏi.

BÀI 11.2	CÁP QUANG YẾU NHẤT	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
CAPQUANG.CPP	CAPQUANG.INP	CAPQUANG.OUT	1 giây · 256 MB

Kiến thức: *Nhảy nhị phân kèm giá trị nhỏ nhất trên đoạn (Tập 1, Bài 11).*

Mạng cáp quang của một tỉnh gồm n trạm và n − 1 đoạn cáp, tạo thành một **cây** (giữa hai trạm bất kì có đúng một đường truyền). Đoạn cáp thứ i nối u_i với v_i và có băng thông c_i.

Tốc độ truyền giữa hai trạm bị giới hạn bởi đoạn cáp **yếu nhất** (băng thông nhỏ nhất) trên đường truyền giữa chúng. Có q yêu cầu, mỗi yêu cầu hỏi tốc độ truyền giữa hai trạm u, v.

Dữ liệu vào

Dòng đầu chứa n, q. Mỗi dòng trong n − 1 dòng tiếp theo chứa ba số u, v, c. Mỗi dòng trong q dòng cuối chứa hai số u, v ($u \neq v$).

Kết quả

Gồm q dòng, mỗi dòng là băng thông nhỏ nhất trên đường truyền tương ứng.

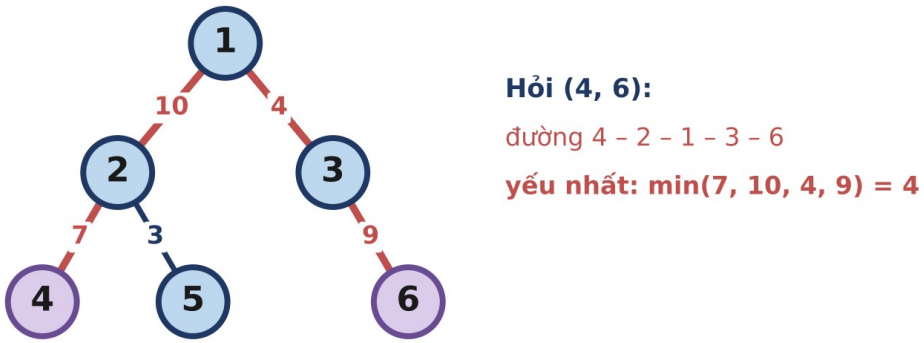
Ràng buộc

$2 \le n \le 10^5$; $1 \le q \le 10^5$; $1 \le c \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	n, q ≤ 1000.
2	20%	Đoạn cáp thứ i nối trạm i với trạm i + 1.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CAPQUANG.INP	CAPQUANG.OUT	Giải thích
6 3 1 2 10 1 3 4 2 4 7 2 5 3 3 6 9 4 5 4 6 2 4	3 4 7	(4, 5): đường 4-2-5, băng thông 7 và 3: đáp số 3. (4, 6): đường 4-2-1-3-6, đáp số min(7, 10, 4, 9) = **4** (Hình 11.2). (2, 4): một đoạn cáp duy nhất, băng thông 7.



Hình 11.2. *Đường truyền giữa hai trạm đi qua tổ tiên chung gần nhất của chúng.*

CHÚ Ý

Subtask 2 là bài toán "giá trị nhỏ nhất trên đoạn" của một dãy số; có thể giải bằng bảng thưa (sparse table), cùng ý tưởng nhảy 2^j bước.

HƯỚNG SUY NGHĨ

- Lưu mn[v][j] là băng thông nhỏ nhất trên đoạn đường từ v nhảy lên 2^j bước: mn[v][j] = min(mn[v][j − 1], mn[up[v][j − 1]][j − 1]).
- Khi tìm LCA(u, v), mỗi lần nhảy u (hoặc v) lên 2^j bước thì cập nhật đáp số bằng mn tương ứng.

BÀI 11.3	NHÂN VIÊN CẤP K	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CAPK.CPP`	`CAPK.INP`	`CAPK.OUT`	2 giây · 64 MB

Kiến thức: *Tổ tiên thứ k; trả lời ngoại tuyến bằng DFS và ngăn xếp đường đi (Tập 1, Bài 11).*

Một tập đoàn có n nhân viên, đánh số từ 1 tới n; nhân viên 1 là tổng giám đốc. Mỗi nhân viên v khác có đúng một **cấp trên trực tiếp** p_v. Cấp trên thứ 1 của v là p_v, cấp trên thứ 2 của v là cấp trên trực tiếp của p_v, v.v.; cấp trên thứ 0 của v là chính v.

Phòng nhân sự có q câu hỏi, mỗi câu hỏi (v, k): ai là cấp trên thứ k của nhân viên v? Nếu v không có cấp trên thứ k thì trả lời −1.

Dữ liệu vào

Dòng đầu chứa n, q. Dòng thứ hai chứa n − 1 số $p_2, p_3, ..., p_n$. Mỗi dòng trong q dòng cuối chứa hai số v, k.

Kết quả

Gồm q dòng, mỗi dòng là câu trả lời cho câu hỏi tương ứng.

Ràng buộc

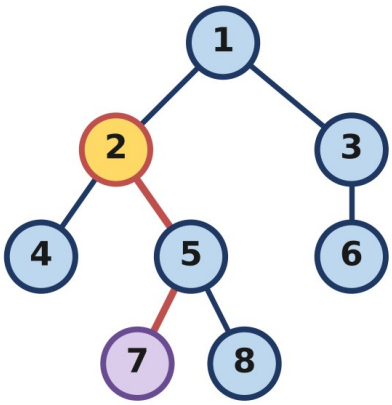
$1 \le n, q \le 10^6$; $0 \le k \le 10^9$; dữ liệu đảm bảo các quan hệ cấp trên tạo thành một cây gốc 1.

Giới hạn bộ nhớ **64 MB**.

Subtask	Điểm	Ràng buộc thêm
1	20%	n, q ≤ 1000.
2	40%	n, q ≤ $2 \cdot 10^5$.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

CAPK.INP	CAPK.OUT	Giải thích
8 4 1 1 2 2 3 5 5 7 2 8 0 6 3 4 1	2 8 -1 2	(7, 2): cấp trên của 7 là 5, của 5 là **2**. (8, 0): chính là 8. (6, 3): 6 → 3 → 1, tổng giám đốc không có cấp trên: −1. (4, 1): cấp trên trực tiếp của 4 là 2.



(7, 2): cấp trên trực tiếp của 7 là 5, của 5 là 2 → in 2

(8, 0): chính là 8

(6, 3): 6 → 3 → 1 → ? → in −1

(4, 1): in 2

Hình 11.3. *Cây quản lí của ví dụ.*

CHÚ Ý

Bảng up[v][j] với n = 10^6 và j tới 20 chiếm 80 MB, vượt giới hạn bộ nhớ!

Với $2 \cdot 10^6$ số cần đọc và 10^6 số cần in, hãy dùng `scanf`/`printf` hoặc đọc ghi nhanh.

k có thể lớn hơn nhiều so với độ sâu của cây.

HƯỚNG SUY NGHĨ

- Subtask 2: nhảy nhị phân, phân tích k theo các bit (nếu $k \geq 2^{20}$ thì chắc chắn trả lời −1).
- Subtask 3: đọc **hết** câu hỏi trước, gom câu hỏi theo đỉnh v. DFS (không đệ quy) từ gốc và duy trì mảng path là dãy các đỉnh từ gốc tới đỉnh đang đứng. Khi đứng ở v, cấp trên thứ k của v chính là path[size − 1 − k] (nếu chỉ số không âm).
- Bộ nhớ chỉ còn O(n + q).

CHƯƠNG 4. CẤU TRÚC CỦA ĐỒ THỊ

*Chương này nhìn đồ thị "từ trên cao": thứ tự giữa các đỉnh (sắp xếp tô pô), những chỗ yếu của mạng lưới (cầu và khớp), các cụm đi qua lại được với nhau (thành phần liên thông mạnh) và những hành trình đi qua mọi cạnh (Euler). Điểm chung của các bài tập là: hãy **nén** đồ thị thành một cấu trúc đơn giản hơn (DAG, cây) rồi mới giải.*

TRONG CHƯƠNG NÀY

Bài 12. Sắp xếp tô pô và quy hoạch động trên DAG
- 12.1 XẾP HẠNG GIẢI ĐẤU — ★☆☆ CƠ BẢN
- 12.2 DỰ ÁN XÂY NHÀ — ★★☆ TRUNG BÌNH
- 12.3 ĐẾM LỘ TRÌNH — ★★★ NÂNG CAO

Bài 13. Cầu và khớp
- 13.1 ĐƯỜNG HUYẾT MẠCH — ★☆☆ CƠ BẢN
- 13.2 TRẠM BIẾN ÁP — ★★☆ TRUNG BÌNH
- 13.3 CÁP DỰ PHÒNG — ★★★ NÂNG CAO

Bài 14. Thành phần liên thông mạnh
- 14.1 LAN TRUYỀN TIN ĐỒN — ★☆☆ CƠ BẢN
- 14.2 KHO BÁU THỊ TRẤN — ★★☆ TRUNG BÌNH
- 14.3 HỘI ĐỒNG TUYỂN CHỌN — ★★★ NÂNG CAO

Bài 15. Chu trình và đường đi Euler
- 15.1 NÉT VẼ — ★☆☆ CƠ BẢN
- 15.2 GHÉP DOMINO — ★★☆ TRUNG BÌNH
- 15.3 MẬT KHẨU KÉT SẮT — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Bài toán trên đồ thị có hướng có chu trình: thử nén thành phần liên thông mạnh để được DAG.
- Các thuật toán num/low và Hierholzer rất dễ bị tràn ngăn xếp: hãy luyện viết phiên bản không đệ quy.
- Với bài có nhiều đáp án, tự viết chương trình kiểm tra đáp án để thử trên nhiều bộ dữ liệu.

BÀI 12 Sắp xếp tô pô và quy hoạch động trên DAG

ÔN NHANH KIẾN THỨC · Tập 1, Bài 12

- **Thuật toán Kahn**: lặp lại việc lấy ra một đỉnh có bán bậc vào bằng 0 rồi xóa các cung đi ra của nó. Nếu lấy ra được đủ n đỉnh thì đó là một thứ tự tô pô; ngược lại đồ thị có chu trình.
- Thứ tự tô pô **duy nhất** khi và chỉ khi trong suốt quá trình, tập đỉnh sẵn sàng (bán bậc vào bằng 0) luôn chỉ có đúng một đỉnh.
- Quy hoạch động trên DAG: xử lí các đỉnh theo thứ tự tô pô; mọi đỉnh "đứng trước" đều đã có kết quả. Dùng cho đường đi dài nhất, đếm số đường đi...
- Đếm số đường đi thường phải lấy theo mô-đun $10^9 + 7$.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
12.1	**XẾP HẠNG GIẢI ĐẤU**	★☆☆ CƠ BẢN	Đọc thêm thông tin từ quá trình chạy Kahn.
12.2	**DỰ ÁN XÂY NHÀ**	★★☆ TRUNG BÌNH	Quy hoạch động hai chiều: xuôi và ngược thứ tự tô pô.
12.3	**ĐẾM LỘ TRÌNH**	★★★ NÂNG CAO	Chỉ giữ lại phần đồ thị "có ích" trước khi quy hoạch động.

BÀI 12.1 XẾP HẠNG GIẢI ĐẤU ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`XHGIAIDAU.CPP`	`XHGIAIDAU.INP`	`XHGIAIDAU.OUT`	1 giây · 256 MB

Kiến thức: *Thuật toán Kahn, kiểm tra chu trình và tính duy nhất (Tập 1, Bài 12).*

Một giải đấu có n đội, đánh số từ 1 tới n. Ban tổ chức nhận được m kết quả dạng "a b": đội a **xếp trên** đội b trong bảng xếp hạng cuối cùng. Bảng xếp hạng là một thứ tự của cả n đội từ hạng 1 tới hạng n, không có hai đội đồng hạng.

Hãy cho biết từ m kết quả ấy có xác định được **duy nhất** một bảng xếp hạng hay không:

- Nếu các kết quả **mâu thuẫn** (không có bảng xếp hạng nào thỏa mãn tất cả), in −1.
- Nếu có **nhiều** bảng xếp hạng thỏa mãn, in 0.
- Nếu có **đúng một** bảng xếp hạng thỏa mãn, in bảng xếp hạng đó.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số a, b ($a \neq b$).

Kết quả

Một số −1 hoặc 0, hoặc n số là các đội theo thứ tự từ hạng 1 tới hạng n.

Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$.

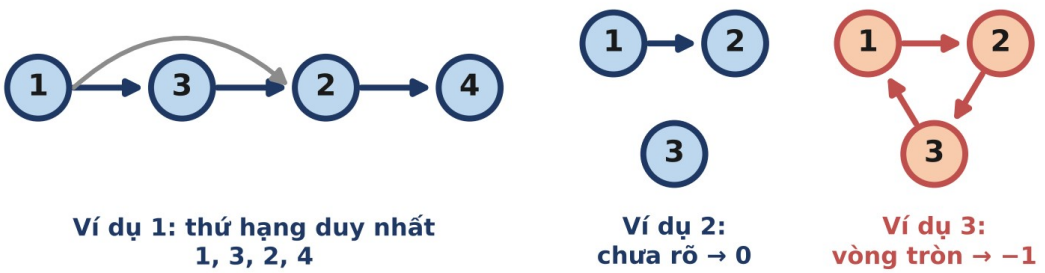
Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 8$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

XHGIAIDAU.INP	XHGIAIDAU.OUT	Giải thích
4 4 1 3 3 2 2 4 1 2	1 3 2 4	1 trên 3, 3 trên 2, 2 trên 4: bảng xếp hạng duy nhất 1, 3, 2, 4. Kết quả "1 2" là thừa nhưng không mâu thuẫn.

XHGIAIDAU.INP	XHGIAIDAU.OUT	Giải thích
3 1 1 2	0	Đội 3 có thể ở bất kì hạng nào: có tới 3 bảng xếp hạng.

XHGIAIDAU.INP	XHGIAIDAU.OUT	Giải thích
3 3 1 2 2 3 3 1	-1	1 trên 2, 2 trên 3, 3 trên 1: vô lí.



Hình 12.1. Ba ví dụ ứng với ba khả năng của đáp số.

CHÚ Ý

Khi vừa có mâu thuẫn vừa có chỗ "chưa rõ", đáp số là −1: hãy kiểm tra chu trình trước.

HƯỚNG SUY NGHĨ

- Chạy Kahn. Nếu số đỉnh lấy ra ít hơn n: in −1.
- Nếu có lúc hàng đợi chứa từ hai đỉnh trở lên thì hai đỉnh ấy có thể đổi chỗ cho nhau: in 0.

BÀI 12.2	DỰ ÁN XÂY NHÀ	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
XAYNHA.CPP	XAYNHA.INP	XAYNHA.OUT	1 giây · 256 MB

Kiến thức: *Đường đi dài nhất trên DAG, thời điểm sớm nhất và muộn nhất (Tập 1, Bài 12).*

Dự án xây một ngôi nhà gồm n công việc, đánh số từ 1 tới n. Công việc i cần đúng t_i ngày và chỉ được bắt đầu khi **mọi** công việc phải làm trước nó đã xong. Đội thợ rất đông, nên nhiều công việc có thể làm song song.

Dự án bắt đầu ở thời điểm 0. Hãy tính:

1) T: thời điểm sớm nhất toàn bộ dự án hoàn thành.

2) Các công việc **then chốt**: nếu riêng công việc đó kéo dài thêm 1 ngày (các công việc khác giữ nguyên) thì dự án sẽ hoàn thành muộn hơn T.

Dữ liệu vào

Dòng đầu chứa n. Dòng thứ i trong n dòng tiếp theo bắt đầu bằng hai số t_i, k_i, tiếp theo là k_i số hiệu các công việc phải làm trước công việc i. Dữ liệu đảm bảo không có vòng phụ thuộc.

Kết quả

Dòng 1: số T. Dòng 2: số công việc then chốt. Dòng 3: các công việc then chốt theo thứ tự tăng dần.

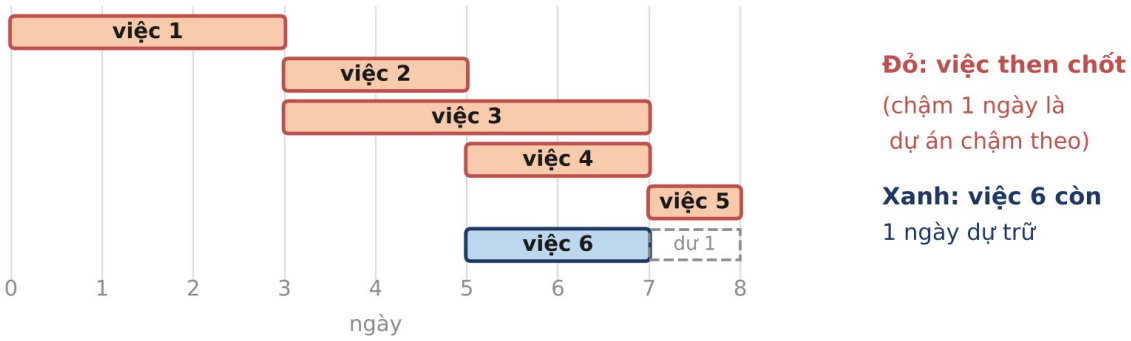
Ràng buộc

$1 \le n \le 10^5$; tổng các k_i không vượt quá $2 \cdot 10^5$; $1 \le t_i \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	40%	$n \le 1000$.
2	60%	Không có ràng buộc gì thêm.

Ví dụ

XAYNHA.INP	XAYNHA.OUT	Giải thích
6 3 0 2 1 1 4 1 1 2 1 2 1 2 3 4 2 1 2	8 5 1 2 3 4 5	Việc 1 xong ở ngày 3; việc 2 và 3 bắt đầu lúc 3; việc 4 bắt đầu lúc 5; việc 5 phải chờ cả 3 và 4, bắt đầu lúc 7; dự án xong lúc **8** (Hình 12.2). Việc 6 có thể chậm 1 ngày mà không ảnh hưởng.



Hình 12.2. *Biểu đồ thời gian khi mỗi việc bắt đầu sớm nhất có thể.*

CHÚ Ý

T có thể tới khoảng 10^{14}: dùng `long long`.

Một công việc **không có việc nào phụ thuộc vào nó** vẫn có thể là việc then chốt (ví dụ việc 5).

HƯỚNG SUY NGHĨ

- Đi xuôi thứ tự tô pô: es[v] = max(es[u] + t_u) với u phải làm trước v. T = max(es[v] + t_v).
- Đi ngược thứ tự tô pô: lf[u] là thời điểm muộn nhất u được phép xong mà dự án vẫn xong lúc T; lf[u] = min(lf[v] − t_v) với v phụ thuộc vào u, hoặc T nếu không có v nào.
- Công việc u then chốt khi và chỉ khi lf[u] − t_u = es[u] (không có ngày dự trữ nào).

BÀI 12.3	ĐẾM LỘ TRÌNH	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
DEMLOTRINH.CPP	DEMLOTRINH.INP	DEMLOTRINH.OUT	1 giây · 256 MB

Kiến thức: *Đếm đường đi trên DAG, phát hiện số lộ trình vô hạn (Tập 1, Bài 12).*

Thị trấn có n nút giao và m con đường **một chiều**; con đường thứ i đi từ u_i tới v_i. Có thể có nhiều con đường cùng nối một cặp nút, hoặc con đường đi từ một nút quay về chính nó.

Một **lộ trình** từ nút 1 tới nút n là một dãy con đường liên tiếp nhau, bắt đầu ở nút 1 và kết thúc ở nút n; lộ trình có thể đi qua một nút hoặc một con đường nhiều lần. Hai lộ trình khác nhau nếu dãy con đường của chúng khác nhau.

Hãy đếm số lộ trình từ 1 tới n. Nếu có vô số lộ trình, in −1; ngược lại in số lộ trình theo mô-đun $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v.

Kết quả

Số lộ trình theo mô-đun $10^9 + 7$, hoặc −1 nếu có vô số lộ trình.

Ràng buộc

$2 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$.

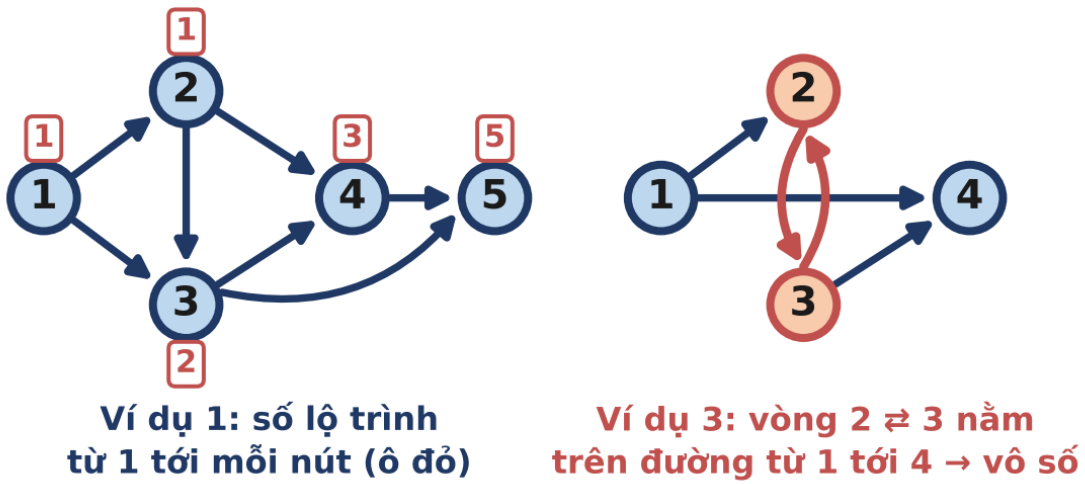
Subtask	Điểm	Ràng buộc thêm
1	30%	Mọi con đường đều có u < v.
2	30%	n, m ≤ 1000.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

DEMLOTRINH.INP	DEMLOTRINH.OUT	Giải thích
5 7 1 2 1 3 2 3 2 4 3 4 4 5 3 5	5	Năm lộ trình: 1-2-3-4-5, 1-2-3-5, 1-2-4-5, 1-3-4-5, 1-3-5. Số ở ô đỏ trong Hình 12.3 là số lộ trình từ 1 tới từng nút.

DEMLOTRINH.INP	DEMLOTRINH.OUT	Giải thích
5 5 1 2 2 5 1 3 3 4 4 3	1	Vòng 3 ⇄ 4 không dẫn tới nút 5, nên không ảnh hưởng: chỉ có lộ trình 1-2-5.

DEMLOTRINH.INP	DEMLOTRINH.OUT	Giải thích
4 5 1 2 2 3 3 2 1 4 3 4	-1	Từ 1 có thể vào vòng 2 ⇄ 3 bao nhiêu lần tùy thích rồi mới sang 4: vô số lộ trình.



Hình 12.3. *Một chu trình chỉ gây ra vô số lộ trình khi nó vừa tới được từ 1, vừa đi tới được n.*

CHÚ Ý

Đừng in −1 chỉ vì đồ thị có chu trình (ví dụ 2).

Nếu không có lộ trình nào thì in 0.

Đáp số sau khi lấy mô-đun có thể bằng 0 dù số lộ trình thật là hữu hạn và dương; đó không phải là lỗi.

HƯỚNG SUY NGHĨ

- Đánh dấu các nút tới được từ 1 (DFS trên đồ thị xuôi) và các nút tới được n (DFS trên đồ thị ngược). Chỉ giữ lại các nút có cả hai dấu.
- Có vô số lộ trình khi và chỉ khi đồ thị con còn lại có chu trình (kể cả khuyên). Kiểm tra bằng Kahn; nếu không có chu trình thì đếm f[v] = Σ f[u] theo thứ tự tô pô.

BÀI 13 Cầu và khớp

ÔN NHANH KIẾN THỨC · Tập 1, Bài 13

- DFS đánh số num[u] theo thứ tự thăm; low[u] là num nhỏ nhất tới được từ cây con của u bằng **một** cạnh ngược.
- Cạnh cây (u, v) là **cầu** khi low[v] > num[u]. Đỉnh u (không phải gốc) là **khớp** khi có con v với low[v] ≥ num[u]; gốc là khớp khi có từ 2 con trở lên.
- Đồ thị có cạnh lặp: khi bỏ qua "cạnh vừa đi tới" phải bỏ theo **chỉ số cạnh**, không theo đỉnh cha; nếu không, cạnh lặp sẽ bị coi nhầm là cầu.
- Nén mỗi thành phần không có cầu thành một đỉnh, ta được **cây cầu** (bridge tree).

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
13.1	**ĐƯỜNG HUYẾT MẠCH**	★☆☆ CƠ BẢN	Cài đặt tìm cầu không đệ quy, đúng với cạnh lặp.
13.2	**TRẠM BIẾN ÁP**	★★☆ TRUNG BÌNH	Không chỉ tìm khớp mà còn đo mức độ "quan trọng" của khớp.
13.3	**CÁP DỰ PHÒNG**	★★★ NÂNG CAO	Nén đồ thị thành cây rồi lập luận trên cây.

BÀI 13.1 ĐƯỜNG HUYẾT MẠCH ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`HUYETMACH.CPP`	`HUYETMACH.INP`	`HUYETMACH.OUT`	1 giây · 256 MB

Kiến thức: *Tìm cầu bằng num/low, xử lí cạnh lặp (Tập 1, Bài 13).*

Mạng giao thông của thành phố có n nút và m con đường hai chiều, đánh số từ 1 tới m; có thể có nhiều con đường nối cùng một cặp nút. Một con đường là **huyết mạch** nếu khi nó bị chặn, có hai nút trước đó đi được tới nhau nay không còn đi được nữa.

Hãy liệt kê mọi con đường huyết mạch để thành phố ưu tiên bảo trì.

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ i trong m dòng tiếp theo chứa hai số u, v (u ≠ v) mô tả con đường thứ i.

Kết quả

Dòng đầu là số con đường huyết mạch. Dòng thứ hai liệt kê số hiệu các con đường đó theo thứ tự tăng dần.

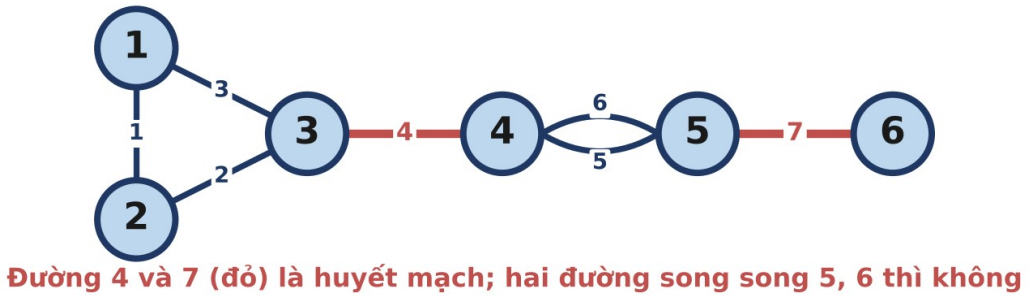
Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$. Mạng giao thông có thể không liên thông.

Subtask	Điểm	Ràng buộc thêm
1	30%	n, m ≤ 1000.
2	30%	Không có hai con đường nào nối cùng một cặp nút.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

HUYETMACH.INP	HUYETMACH.OUT	Giải thích
6 7 1 2 2 3 3 1 3 4 4 5 4 5 5 6	2 4 7	Chặn đường 4 thì {1, 2, 3} tách khỏi {4, 5, 6}; chặn đường 7 thì nút 6 bị cô lập. Đường 5 và 6 cùng nối 4 với 5: chặn một đường vẫn còn đường kia (Hình 13.1).



Hình 13.1. *Số trên cạnh là số hiệu con đường.*

CHÚ Ý

Với cách cài đặt "không đi ngược về đỉnh cha", hai con đường 5 và 6 trong ví dụ sẽ bị nhận nhầm là huyết mạch.

Mạng hình "đường thẳng" 10^5 nút làm DFS đệ quy rất sâu.

HƯỚNG SUY NGHĨ

- Lưu danh sách kề dạng (đỉnh kề, chỉ số cạnh). Khi duyệt từ u, chỉ bỏ qua **đúng** cạnh có chỉ số bằng cạnh đã dùng để đi xuống u.
- Gọi DFS từ mọi đỉnh chưa thăm vì đồ thị có thể không liên thông.

BÀI 13.2	TRẠM BIẾN ÁP	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`BIENAP.CPP`	`BIENAP.INP`	`BIENAP.OUT`	1 giây · 256 MB

Kiến thức: *Khớp, đếm số con có low ≥ num (Tập 1, Bài 13).*

Lưới điện của một huyện gồm n trạm biến áp và m đường dây hai chiều; từ một trạm bất kì đều truyền điện được tới mọi trạm khác. Khi một trạm gặp sự cố, trạm đó và mọi đường dây nối vào nó ngừng hoạt động; các trạm còn lại có thể bị chia thành nhiều **phần** không truyền điện được cho nhau.

Với mỗi trạm mà sự cố của nó làm lưới điện bị chia thành **từ hai phần trở lên**, hãy cho biết lưới điện bị chia thành bao nhiêu phần.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v (u ≠ v).

Kết quả

Dòng đầu là số trạm k thỏa mãn. Tiếp theo là k dòng, mỗi dòng hai số: số hiệu trạm và số phần, theo thứ tự tăng dần của số hiệu trạm.

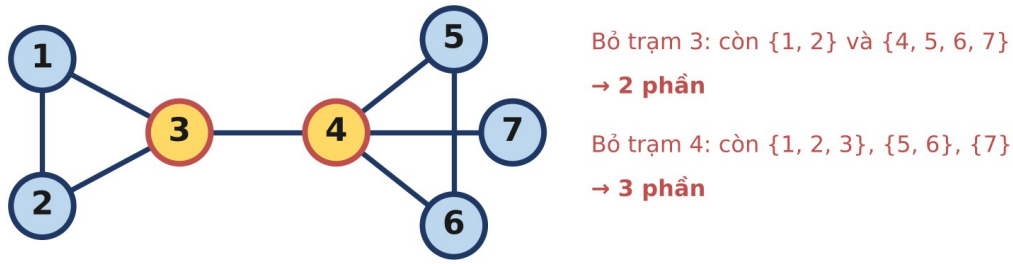
Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; lưới điện liên thông.

Subtask	Điểm	Ràng buộc thêm
1	30%	n, m ≤ 1000.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

BIENAP.INP	BIENAP.OUT	Giải thích
7 8 1 2 2 3 3 1 3 4 4 5 5 6 6 4 4 7	2 3 2 4 3	Trạm 3 hỏng: còn {1, 2} và {4, 5, 6, 7}, tức 2 phần. Trạm 4 hỏng: còn {1, 2, 3}, {5, 6} và {7}, tức 3 phần (Hình 13.2).



Hình 13.2. *Hai trạm tô vàng là khớp của lưới điện.*

CHÚ Ý

Gốc của cây DFS là trường hợp đặc biệt: số phần đúng bằng số con của nó trong cây DFS.

HƯỚNG SUY NGHĨ

- Với đỉnh u không phải gốc: mỗi con v có low[v] ≥ num[u] tạo ra một phần riêng; phần còn lại (chứa cha của u) là thêm 1 phần nữa.
- Vậy số phần = (số con v có low[v] ≥ num[u]) + 1 với u không phải gốc, và = số con với u là gốc. Đỉnh là khớp khi số phần ≥ 2.

BÀI 13.3	CÁP DỰ PHÒNG	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
CAPDUPHONG.CPP	**CAPDUPHONG.INP**	**CAPDUPHONG.OUT**	**1 giây · 256 MB**

Kiến thức: *Cây cầu (bridge tree), đếm lá (Tập 1, Bài 13).*

Mạng máy tính của một công ti gồm n máy và m sợi cáp hai chiều, liên thông. Giám đốc muốn mạng **chịu được một sự cố**: khi một sợi cáp bất kì bị đứt, mọi máy vẫn liên lạc được với nhau.

Hãy tính số sợi cáp **ít nhất** cần lắp thêm (mỗi sợi nối hai máy tùy chọn) để đạt được yêu cầu đó.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$); có thể có nhiều sợi cáp nối cùng một cặp máy.

Kết quả

Số sợi cáp ít nhất cần lắp thêm.

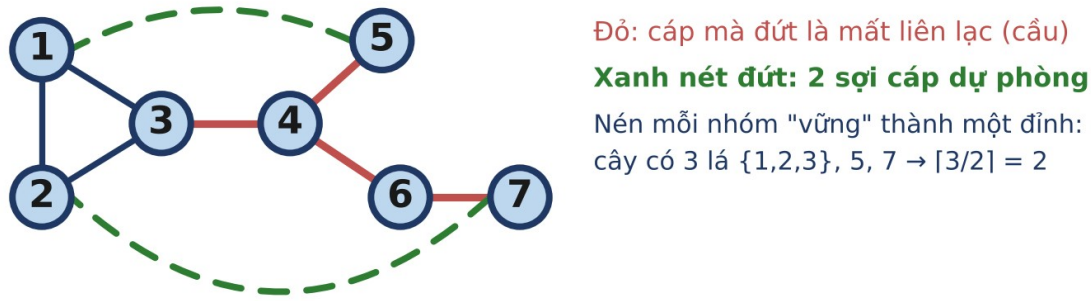
Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; mạng liên thông.

Subtask	Điểm	Ràng buộc thêm
1	20%	$m = n - 1$ (mạng là một cây).
2	30%	$n, m \le 1000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CAPDUPHONG.INP	CAPDUPHONG.OUT	Giải thích
7 7 1 2 2 3 3 1 3 4 4 5 4 6 6 7	2	Bốn sợi cáp đỏ là cầu. Nén vòng {1, 2, 3} thành một đỉnh, ta được cây có 3 lá: {1, 2, 3}, 5 và 7. Nối 5 với 1 và 7 với 2 là đủ: **2** sợi (Hình 13.3).



Hình 13.3. *Mỗi sợi cáp mới "khép vòng" được tối đa hai lá của cây cầu.*

CHÚ Ý

Nếu mạng ban đầu không có cầu thì đáp số là 0.

Hai sợi cáp nối cùng một cặp máy không phải là cầu.

HƯỚNG SUY NGHĨ

- Tìm mọi cầu; dùng DSU gộp hai đầu của các cạnh **không** phải cầu để được các thành phần "vững". Các cầu nối các thành phần này thành một cây.
- Một lá của cây cầu bắt buộc phải có cáp mới đi ra từ nó; mỗi cáp mới có hai đầu nên phục vụ được tối đa hai lá. Đáp số là $\lceil L / 2 \rceil$ với L là số lá (và 0 nếu cây chỉ có một đỉnh).
- Khó hơn (tự luyện): chứng minh $\lceil L / 2 \rceil$ là đủ, bằng cách sắp các lá theo thứ tự DFS và nối lá thứ i với lá thứ $i + \lfloor L / 2 \rfloor$.

BÀI 14 Thành phần liên thông mạnh

ÔN NHANH KIẾN THỨC · Tập 1, Bài 14

- Hai đỉnh cùng **thành phần liên thông mạnh** (TPLTM) khi đi được từ đỉnh này tới đỉnh kia và ngược lại. Tìm mọi TPLTM bằng Tarjan hoặc Kosaraju trong O(n + m).
- Nén mỗi TPLTM thành một đỉnh ta được **đồ thị nén**, luôn là một DAG. Tarjan đánh số các thành phần theo thứ tự tô pô **ngược**.
- Thành phần **nguồn**: không có cung đi vào từ thành phần khác; thành phần **hút**: không có cung đi ra.
- **2-SAT**: mỗi yêu cầu "x hoặc y" cho hai cung ¬x → y và ¬y → x. Vô nghiệm khi x và ¬x cùng một TPLTM; nếu có nghiệm, chọn x đúng khi thành phần của x đứng **sau** thành phần của ¬x theo thứ tự tô pô.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
14.1	**LAN TRUYỀN TIN ĐỒN**	★☆☆ CƠ BẢN	Đếm thành phần nguồn trên đồ thị nén.
14.2	**KHO BÁU THỊ TRẤN**	★★☆ TRUNG BÌNH	Kết hợp TPLTM với quy hoạch động.
14.3	**HỘI ĐỒNG TUYỂN CHỌN**	★★★ NÂNG CAO	Mô hình hóa ràng buộc logic thành đồ thị kéo theo.

BÀI 14.1 LAN TRUYỀN TIN ĐỒN ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TINDON.CPP`	`TINDON.INP`	`TINDON.OUT`	1 giây · 256 MB

Kiến thức: *Thành phần liên thông mạnh, thành phần nguồn (Tập 1, Bài 14).*

Trường có n học sinh. Có m quan hệ dạng "a b": nếu a biết một tin thì a chắc chắn sẽ kể cho b (nhưng b chưa chắc kể cho a). Tin cứ thế lan truyền theo các quan hệ.

Nhà trường muốn **mọi** học sinh đều biết một thông báo quan trọng. Hỏi cần trực tiếp báo tin cho ít nhất bao nhiêu học sinh?

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số a, b (a ≠ b).

Kết quả

Số học sinh ít nhất cần báo tin trực tiếp.

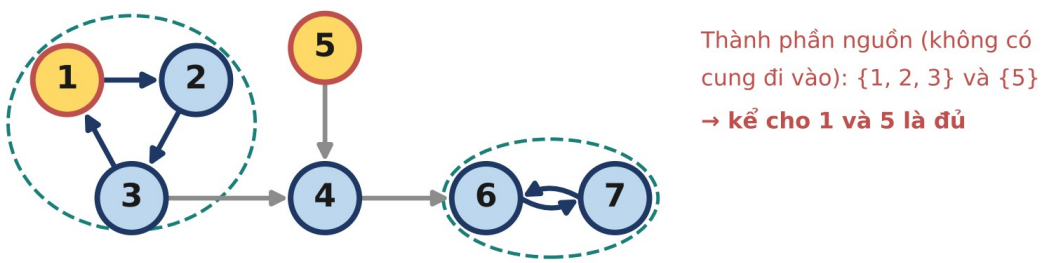
Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Không có vòng truyền tin (đồ thị không có chu trình).
2	30%	n, m ≤ 1000.
3	40%	Không có ràng buộc gì thêm.

Ví dụ

TINDON.INP	TINDON.OUT	Giải thích
7 8 1 2 2 3 3 1 3 4 5 4 6 7 7 6 4 6	2	Báo cho học sinh 1 thì cả nhóm {1, 2, 3} biết, rồi lan sang 4, 6, 7. Học sinh 5 không ai kể cho nên phải báo riêng: **2** người (Hình 14.1).



Hình 14.1. *Hai thành phần nguồn {1, 2, 3} và {5}.*

CHÚ Ý

Không chỉ đếm số học sinh có bán bậc vào bằng 0: trong ví dụ, không ai trong nhóm {1, 2, 3} có bán bậc vào bằng 0, nhưng vẫn phải báo cho một người trong nhóm.

HƯỚNG SUY NGHĨ

- Mỗi thành phần liên thông mạnh hoặc biết tin cả, hoặc không ai biết.
- Đáp số là số thành phần không có cung nào đi vào từ thành phần khác.

BÀI 14.2	KHO BÁU THỊ TRẤN	★★☆ TRUNG BÌNH	
Chương trình **KHOBAU.CPP**	Dữ liệu vào **KHOBAU.INP**	Kết quả **KHOBAU.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Đồ thị nén và quy hoạch động trên DAG (Tập 1, Bài 12, 14).*

Thị trấn có n địa điểm và m con đường **một chiều**. Ở địa điểm i có g_i đồng vàng. Mỗi khi tới một địa điểm, nhà thám hiểm nhặt hết vàng ở đó (tới lại lần sau thì không còn gì).

Nhà thám hiểm xuất phát từ địa điểm 1, đi theo các con đường bao lâu tùy thích và dừng ở bất kì đâu. Hỏi nhặt được nhiều nhất bao nhiêu đồng vàng?

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ hai chứa n số $g_1, ..., g_n$. Mỗi dòng trong m dòng cuối chứa hai số u, v ($u \neq v$): con đường một chiều từ u tới v.

Kết quả

Số vàng lớn nhất nhặt được.

Ràng buộc

$1 \le n \le 10^5$; $0 \le m \le 2 \cdot 10^5$; $0 \le g_i \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Đồ thị không có chu trình.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

KHOBAU.INP	KHOBAU.OUT	Giải thích
6 7 5 1 3 4 2 7 1 2 2 3 3 1 3 4 4 5 5 4 2 6	16	Đi vòng 1 → 2 → 3 → 1 nhặt 5 + 1 + 3 = 9, rồi từ 2 sang 6 nhặt thêm 7: tổng **16** (Hình 14.2). Nếu từ 3 sang 4 rồi qua lại 4, 5 thì chỉ được 9 + 4 + 2 = 15.



Hình 14.2. *Số trong ô vuông là số vàng ở mỗi địa điểm.*

CHÚ Ý

Đã vào một TPLTM thì nhặt được **toàn bộ** vàng trong đó rồi đi ra theo bất kì cung nào.

Tổng số vàng có thể tới 10^{14}: dùng `long long`.

HƯỚNG SUY NGHĨ

- Nén TPLTM, trọng số mỗi đỉnh nén là tổng vàng trong thành phần.
- Trên DAG nén, tìm đường đi có tổng trọng số đỉnh lớn nhất bắt đầu từ thành phần chứa địa điểm 1 (quy hoạch động theo thứ tự tô pô).

BÀI 14.3	HỘI ĐỒNG TUYỂN CHỌN	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
TUYENCHON.CPP	TUYENCHON.INP	TUYENCHON.OUT	1 giây · 256 MB

Kiến thức: *2-SAT bằng thành phần liên thông mạnh (Tập 1, Bài 14).*

Vòng chung kết có n thí sinh, mỗi thí sinh sẽ được **nhận** hoặc bị **loại**. Hội đồng có m giám khảo; mỗi giám khảo đưa ra đúng **hai** mong muốn, mỗi mong muốn có dạng "nhận thí sinh a" hoặc "loại thí sinh a". Một giám khảo **hài lòng** nếu ít nhất một trong hai mong muốn của mình được thực hiện.

Hãy tìm một phương án nhận/loại làm **mọi** giám khảo hài lòng, hoặc cho biết không có phương án nào.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên khác 0 là hai mong muốn của một giám khảo: số dương +a nghĩa là "nhận thí sinh a", số âm −a nghĩa là "loại thí sinh a" ($1 \le a \le n$).

Kết quả

Nếu không có phương án, in `NO`. Ngược lại dòng đầu in `YES`, dòng thứ hai in xâu độ dài n, kí tự thứ i là `1` nếu nhận thí sinh i, là `0` nếu loại. Nếu có nhiều phương án, in phương án bất kì.

Ràng buộc

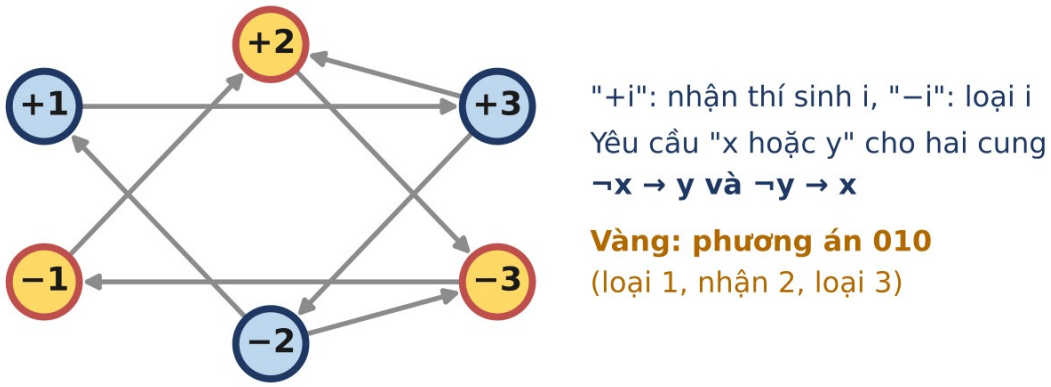
$1 \le n, m \le 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$n \le 20$.
2	80%	Không có ràng buộc gì thêm.

Ví dụ

TUYENCHON.INP	TUYENCHON.OUT	Giải thích
3 4 1 2 -1 3 -2 -3 2 -3	YES 010	Loại 1, nhận 2, loại 3: giám khảo 1 hài lòng vì 2 được nhận, giám khảo 2 vì 1 bị loại, giám khảo 3 và 4 vì 3 bị loại. Có thể kiểm tra đây là phương án **duy nhất**.

TUYENCHON.INP	TUYENCHON.OUT	Giải thích
1 2 1 1 -1 -1	NO	Giám khảo 1 muốn nhận thí sinh 1, giám khảo 2 muốn loại thí sinh 1: không thể chiều cả hai.



Hình 14.3. *Đồ thị kéo theo của ví dụ 1; mỗi giám khảo cho hai cung.*

CHÚ Ý

Một giám khảo có thể đưa ra hai mong muốn giống nhau (như ví dụ 2): khi đó mong muốn ấy **bắt buộc** phải được thực hiện.

Bài có nhiều đáp án: chương trình chấm sẽ kiểm tra phương án của em có làm mọi giám khảo hài lòng không.

HƯỚNG SUY NGHĨ

- Mỗi thí sinh i ứng với hai đỉnh: "nhận i" và "loại i". Mong muốn "x hoặc y" cho hai cung: (không x) → y và (không y) → x.
- Vô nghiệm khi và chỉ khi có i mà "nhận i" và "loại i" nằm trong cùng một TPLTM.
- Với cài đặt Tarjan (thành phần được đánh số theo thứ tự tô pô ngược), nhận i khi comp["nhận i"] < comp["loại i"].

BÀI 15 Chu trình và đường đi Euler

ÔN NHANH KIẾN THỨC · Tập 1, Bài 15

- Đồ thị vô hướng có **chu trình Euler** khi mọi cạnh nằm trong một thành phần liên thông và mọi đỉnh có bậc chẵn; có **đường đi Euler** khi số đỉnh bậc lẻ là 0 hoặc 2.
- Đồ thị có hướng: chu trình Euler khi mọi đỉnh có bán bậc vào bằng bán bậc ra (và các cung liên thông).
- **Hierholzer** tìm chu trình/đường đi Euler trong O(m): đi tới khi "kẹt", rồi lùi lại và ghi đỉnh; kết quả đảo ngược là đáp số. Nên cài bằng ngăn xếp.
- Mỗi khuyên góp 2 vào bậc của đỉnh.

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
15.1	**NÉT VẼ**	★☆☆ CƠ BẢN	Kiểm tra cả hai điều kiện: bậc và tính liên thông.
15.2	**GHÉP DOMINO**	★★☆ TRUNG BÌNH	Mô hình hóa: mỗi đối tượng là một cạnh, không phải một đỉnh.
15.3	**MẬT KHẨU KÉT SẮT**	★★★ NÂNG CAO	Xây đồ thị trừu tượng trên các xâu.

BÀI 15.1 NÉT VẼ ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`NETVE.CPP`	`NETVE.INP`	`NETVE.OUT`	1 giây · 256 MB

Kiến thức: *Điều kiện tồn tại đường đi Euler (Tập 1, Bài 15).*

Trò chơi "vẽ một nét": cho một hình gồm n điểm và m đoạn thẳng (có thể có hai đoạn cùng nối một cặp điểm, hoặc một vòng nhỏ đi từ một điểm quay về chính nó). Người chơi phải vẽ lại **mọi** đoạn, mỗi đoạn **đúng một lần**, mà **không nhấc bút**; điểm bắt đầu và điểm kết thúc có thể khác nhau. Các điểm không có đoạn nào thì không cần vẽ.

Với mỗi hình trong T hình, hãy cho biết có vẽ được bằng một nét hay không.

Dữ liệu vào

Dòng đầu chứa T. Mỗi hình gồm: một dòng chứa n, m; tiếp theo là m dòng, mỗi dòng hai số u, v mô tả một đoạn (u = v là một vòng nhỏ tại u).

Kết quả

Gồm T dòng, mỗi dòng `YES` hoặc `NO`.

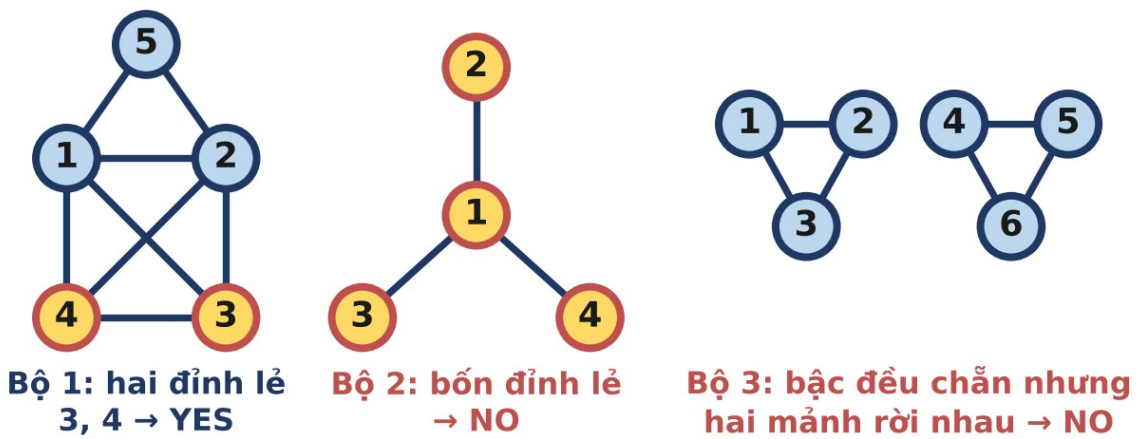
Ràng buộc

$1 \le T \le 10$; $1 \le n \le 10^5$; $1 \le m \le 2 \cdot 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	Mọi điểm đều có đoạn nối vào, hình liên thông, không có vòng nhỏ.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

NETVE.INP	NETVE.OUT	Giải thích
3 5 8 1 2 2 3 3 4 4 1 1 5 2 5 1 3 2 4 4 3 1 2 1 3 1 4 6 6 1 2 2 3 3 1 4 5 5 6 6 4	YES NO NO	Hình 1 là "phong bì" quen thuộc: chỉ có hai điểm bậc lẻ là 3 và 4, vẽ được bằng cách bắt đầu từ 3 và kết thúc ở 4. Hình 2: bốn điểm bậc lẻ. Hình 3: mọi điểm bậc chẵn nhưng hình gồm hai mảnh rời nhau (Hình 15.1).



Hình 15.1. *Ba hình của ví dụ; điểm vàng là điểm có bậc lẻ.*

CHÚ Ý

Đếm số điểm bậc lẻ là **chưa đủ**: còn phải kiểm tra mọi đoạn thuộc cùng một mảnh liên thông (bỏ qua các điểm cô lập).

Vòng nhỏ tại u làm bậc của u tăng 2.

HƯỚNG SUY NGHĨ

- Dùng DSU để kiểm tra mọi điểm có bậc dương nằm cùng một nhóm, và đếm số điểm bậc lẻ: vẽ được khi số đó bằng 0 hoặc 2.

BÀI 15.2	GHÉP DOMINO	★★☆ TRUNG BÌNH

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DOMINO.CPP`	`DOMINO.INP`	`DOMINO.OUT`	1 giây · 256 MB

Kiến thức: *Tìm đường đi Euler bằng Hierholzer trên đa đồ thị có khuyên (Tập 1, Bài 15).*

Có n quân domino, đánh số từ 1 tới n; quân thứ i có hai nửa với a_i và b_i chấm ($0 \le a_i, b_i \le 6$; có thể có các quân giống hệt nhau). Hãy xếp **tất cả** các quân thành một hàng ngang sao cho hai nửa kề nhau của hai quân đứng cạnh nhau có cùng số chấm. Mỗi quân có thể được xoay ngược.

Dữ liệu vào

Dòng đầu chứa n. Dòng thứ i trong n dòng tiếp theo chứa a_i, b_i.

Kết quả

Nếu không xếp được, in `NO`. Ngược lại in n dòng mô tả các quân từ trái sang phải; mỗi dòng gồm số hiệu quân và một kí tự: `+` nếu đặt quân theo đúng chiều $a_i \mid b_i$, `-` nếu xoay thành $b_i \mid a_i$. Nếu có nhiều cách xếp, in cách bất kì.

Ràng buộc

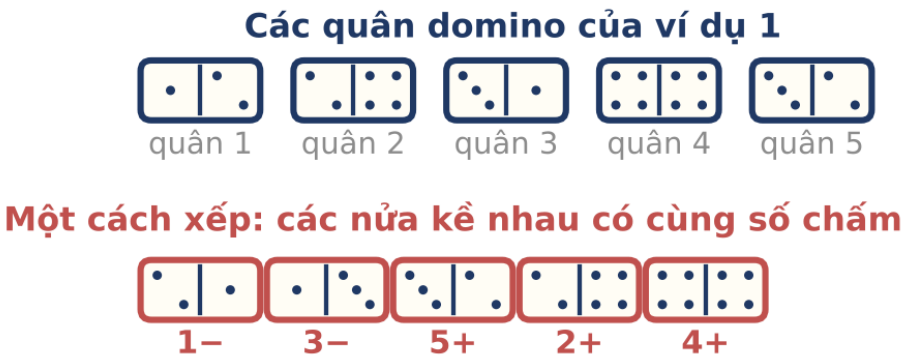
$1 \le n \le 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n \le 8$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

DOMINO.INP	DOMINO.OUT	Giải thích
5 1 2 2 4 3 1 4 4 3 2	1 - 3 - 5 + 2 + 4 +	Hàng domino 2|1, 1|3, 3|2, 2|4, 4|4 (Hình 15.2). Đây chỉ là một trong nhiều cách xếp.

DOMINO.INP	DOMINO.OUT	Giải thích
2 0 0 1 1	NO	Quân 0|0 và quân 1|1 không thể đặt cạnh nhau.



Hình 15.2. *Mỗi số chấm là một đỉnh, mỗi quân domino là một cạnh; hàng domino chính là một đường đi Euler.*

CHÚ Ý

Quân a|a là một **khuyên**. Ví dụ 2 cho thấy chỉ xét bậc là không đủ (mọi đỉnh đều có bậc chẵn).

Bài có nhiều đáp án; chương trình chấm sẽ kiểm tra cách xếp của em.

HƯỚNG SUY NGHĨ

- Đồ thị có 7 đỉnh (0..6) và n cạnh. Cần đường đi Euler: số đỉnh bậc lẻ là 0 hoặc 2; nếu là 2 thì bắt đầu từ một đỉnh lẻ.
- Hierholzer không đệ quy, ghi lại cả **chỉ số cạnh** dùng để đi tới mỗi đỉnh; nếu kết quả không dùng đủ n cạnh thì đồ thị không liên thông: in NO.
- Chiều đặt quân: nếu đi cạnh i từ đỉnh a_i sang b_i thì in +, ngược lại in −.

BÀI 15.3	MẬT KHẨU KÉT SẮT	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
KETSAT.CPP	KETSAT.INP	KETSAT.OUT	1 giây · 256 MB

Kiến thức: *Dãy de Bruijn qua chu trình Euler có hướng (Tập 1, Bài 15).*

Một két sắt có bàn phím gồm b phím, ghi các chữ số 0, 1, ..., b − 1. Mật khẩu là một dãy **k chữ số**. Két không có phím "Enter": két mở ngay khi **k chữ số bấm gần nhất** trùng với mật khẩu.

Một tên trộm không biết mật khẩu nhưng muốn bấm một dãy phím **ngắn nhất** đảm bảo chắc chắn mở được két, dù mật khẩu là gì. Hãy tìm một dãy như vậy, tức là một xâu ngắn nhất chứa mọi dãy k chữ số (trên b chữ số) như một xâu con liên tiếp.

Dữ liệu vào

Một dòng gồm hai số b, k.

Kết quả

Dòng đầu là độ dài L của xâu. Dòng thứ hai là xâu tìm được. Nếu có nhiều xâu ngắn nhất, in xâu bất kì.

Ràng buộc

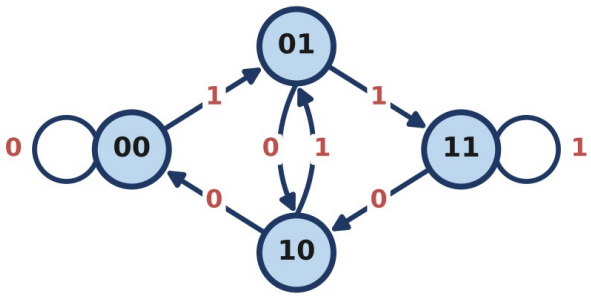
$1 \le b \le 10$; $k \ge 1$; $b^k \le 10^6$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$k \le 2$.
2	30%	$b^k \le 2000$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

KETSAT.INP	KETSAT.OUT	Giải thích
2 3	10 0001011100	Xâu chứa đủ 8 mã 3 chữ số: 000, 001, 010, 101, 011, 111, 110, 100 (Hình 15.3).

KETSAT.INP	KETSAT.OUT	Giải thích
3 2	10 0010211220	Chứa đủ 9 mã: 00, 01, 10, 02, 21, 11, 12, 22, 20.



Đỉnh: 2 chữ số cuối đã bấm ($k - 1 = 2$)

Cung: bấm thêm 1 chữ số → một mã mới

Mỗi cung là đúng một mã 3 chữ số; đi qua mọi cung đúng một lần = **chu trình Euler → xâu 0001011100**

Hình 15.3. *Đồ thị de Bruijn với $b = 2$, $k = 3$: đỉnh là xâu $k - 1$ kí tự, cung là xâu k kí tự.*

CHÚ Ý

Mỗi lần bấm thêm một phím tạo ra **nhiều nhất một** mã mới, nên mọi xâu đúng đều dài ít nhất $b^k + k - 1$. Hãy chắc chắn chương trình của em in đúng độ dài này.

Đường đi trong đồ thị có tới 10^6 cung: Hierholzer đệ quy sẽ tràn ngăn xếp.

HƯỚNG SUY NGHĨ

- Đỉnh là các xâu $k - 1$ chữ số (b^{k-1} đỉnh). Từ đỉnh x có b cung: bấm thêm chữ số c thì sang đỉnh gồm $k - 2$ chữ số cuối của x nối thêm c. Mỗi cung ứng với đúng một mã k chữ số.
- Mọi đỉnh có bán bậc vào = bán bậc ra = b và đồ thị liên thông mạnh, nên có chu trình Euler. Bắt đầu từ đỉnh 00...0, xâu kết quả là $k - 1$ số 0 nối với các chữ số trên các cung của chu trình.
- Mã hóa mỗi đỉnh bằng một số nguyên trong $[0, b^{k-1})$ để không phải dùng xâu.

CHƯƠNG 5. GHÉP CẶP VÀ LUỒNG

*Ghép cặp và luồng là "vũ khí hạng nặng" của lí thuyết đồ thị: rất nhiều bài toán phân công, xếp lịch, phủ, chọn tập... sau khi mô hình hóa đúng cách đều trở thành tìm bộ ghép cực đại hoặc luồng cực đại. Các bài tập của chương tập trung vào **mô hình hóa** và vào việc **dựng lại đáp án** (không chỉ in một con số).*

TRONG CHƯƠNG NÀY

Bài 16. Đồ thị hai phía và bộ ghép cực đại

- **16.1 XẾP PHÒNG THI** — ★☆☆ CƠ BẢN
- **16.2 ĐẶT DOMINO** — ★★☆ TRUNG BÌNH
- **16.3 CAMERA GIÁM SÁT** — ★★★ NÂNG CAO

Bài 17. Luồng cực đại và lát cắt nhỏ nhất

- **17.1 ĐƯỜNG ỐNG** — ★☆☆ CƠ BẢN
- **17.2 LỘ TRÌNH RỜI NHAU** — ★★☆ TRUNG BÌNH
- **17.3 PHÂN CÔNG TRỰC NHẬT** — ★★★ NÂNG CAO

LỜI KHUYÊN CHO CHƯƠNG NÀY

- Trước khi code, vẽ mạng luồng ra giấy: các đỉnh là gì, cung nào sức chứa bao nhiêu, và vì sao một luồng hợp lệ ứng với một phương án.
- Một giới hạn "mỗi X nhiều nhất k lần" thường là sức chứa của cung đi vào hoặc đi ra khỏi đỉnh X.
- Lát cắt nhỏ nhất luôn được đọc ra từ lần BFS cuối cùng trên đồ thị thặng dư.

BÀI 16 Đồ thị hai phía và bộ ghép cực đại

ÔN NHANH KIẾN THỨC · Tập 1, Bài 16

- **Bộ ghép**: tập cạnh không có hai cạnh chung đỉnh. Thuật toán **Kuhn** lần lượt thử ghép từng đỉnh bên trái bằng một **đường tăng** (DFS, có thể "dời" người đã ghép): $O(V \cdot E)$.
- Mô hình hóa quen thuộc: **lưới ô** tô màu bàn cờ (ô trắng, ô đen là hai phía); **hàng – cột** (mỗi ô (r, c) là một cạnh nối hàng r với cột c).
- **Định lí König**: trong đồ thị hai phía, kích thước bộ ghép cực đại = kích thước **phủ đỉnh** nhỏ nhất (tập đỉnh chạm mọi cạnh).
- Dựng phủ đỉnh nhỏ nhất: Z = các đỉnh tới được từ đỉnh trái **chưa ghép** theo đường đan xen; phủ đỉnh = (trái \ Z) ∪ (phải ∩ Z).

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
16.1	**XẾP PHÒNG THI**	★☆☆ CƠ BẢN	Cài đặt Kuhn chuẩn.
16.2	**ĐẶT DOMINO**	★★☆ TRUNG BÌNH	Nhận ra cấu trúc hai phía ẩn trong lưới ô.
16.3	**CAMERA GIÁM SÁT**	★★★ NÂNG CAO	Không chỉ tính kích thước mà còn dựng được đáp án.

BÀI 16.1 XẾP PHÒNG THI

★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`XEPPHONG.CPP`	`XEPPHONG.INP`	`XEPPHONG.OUT`	1 giây · 256 MB

Kiến thức: *Bộ ghép cực đại, thuật toán Kuhn (Tập 1, Bài 16).*

Kì thi bổ sung có n thí sinh và m phòng thi; mỗi phòng chỉ còn **một** chỗ trống. Vì lí do sức khỏe và đi lại, thí sinh thứ i chỉ có thể thi ở một số phòng nhất định.

Hãy xếp được nhiều thí sinh nhất vào phòng thi, mỗi thí sinh một phòng trong danh sách của mình, mỗi phòng nhiều nhất một thí sinh. Cho biết số thí sinh xếp được nhiều nhất.

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ i trong n dòng tiếp theo bắt đầu bằng k_i, sau đó là k_i số hiệu phòng (đôi một khác nhau) mà thí sinh i có thể thi.

Kết quả

Số thí sinh xếp được nhiều nhất.

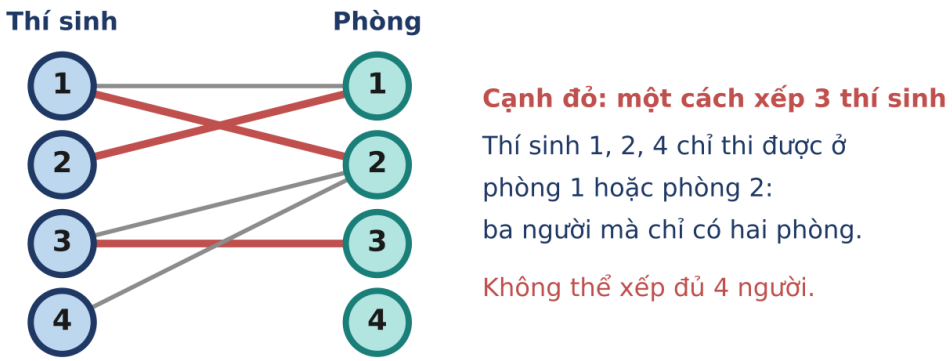
Ràng buộc

$1 \le n, m \le 1000$; tổng các k_i không vượt quá 10^5.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n, m \le 10$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

XEPPHONG.INP	XEPPHONG.OUT	Giải thích
4 4 2 1 2 1 1 2 2 3 1 2	3	Thí sinh 1, 2, 4 đều chỉ thi được ở phòng 1 hoặc 2, nên nhiều nhất hai người trong số họ được xếp. Cách xếp 3 người: 2 → phòng 1, 1 → phòng 2, 3 → phòng 3 (Hình 16.1).



Hình 16.1. *Mỗi thí sinh, mỗi phòng là một đỉnh; cạnh nối thí sinh với phòng có thể thi.*

CHÚ Ý

Xếp tham lam (mỗi thí sinh nhận phòng trống đầu tiên trong danh sách) có thể sai: ở ví dụ, nếu thí sinh 1 nhận phòng 1 thì thí sinh 2 không còn phòng nào. Kuhn sửa lại bằng cách **dời** thí sinh 1 sang phòng 2.

HƯỚNG SUY NGHĨ

- Kuhn: với mỗi thí sinh u, tìm đường tăng bằng DFS: thử từng phòng v; nếu v còn trống hoặc người đang ở v **dời được** sang phòng khác thì xếp u vào v.
- Nhớ làm mới mảng đánh dấu cho mỗi lần tìm đường tăng.

BÀI 16.2	ĐẶT DOMINO	★★☆ TRUNG BÌNH	
Chương trình **DATDOMINO.CPP**	Dữ liệu vào **DATDOMINO.INP**	Kết quả **DATDOMINO.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Bộ ghép cực đại trên lưới, tô màu bàn cờ (Tập 1, Bài 16).*

Cho bảng R × C, mỗi ô là ô trống (`.`) hoặc ô cấm (`#`). Mỗi quân domino phủ đúng **hai ô trống kề cạnh**. Các quân không được chồng lên nhau và không được đặt lên ô cấm.

Hỏi đặt được nhiều nhất bao nhiêu quân domino lên bảng?

Dữ liệu vào

Dòng đầu chứa R, C. R dòng tiếp theo mô tả bảng.

Kết quả

Số quân domino nhiều nhất.

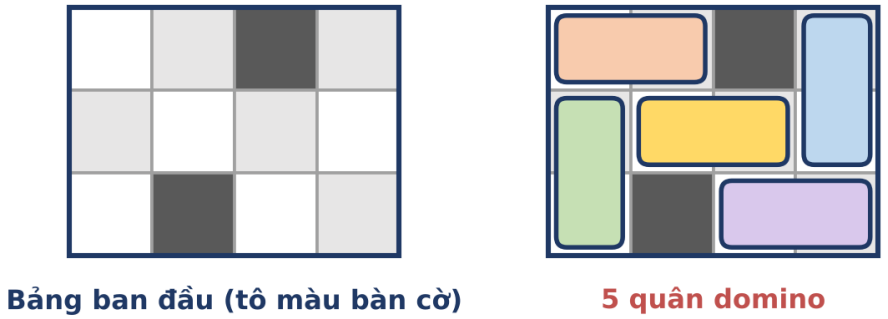
Ràng buộc

$1 \le R, C \le 100$.

Subtask	Điểm	Ràng buộc thêm
1	20%	Không có ô cấm.
2	30%	$R, C \le 10$.
3	50%	Không có ràng buộc gì thêm.

Ví dụ

DATDOMINO.INP	DATDOMINO.OUT	Giải thích
3 4 ..#. .... .#..	5	Có 10 ô trống, và đặt được **5** quân phủ kín cả 10 ô (Hình 16.2).



Hình 16.2. *Mỗi quân domino luôn phủ một ô trắng và một ô xám của bàn cờ.*

CHÚ Ý

Với bảng 100 × 100 có khoảng 5000 ô mỗi màu và $2 \cdot 10^4$ cạnh. Kuhn vẫn chạy kịp nếu cài cẩn thận (ví dụ ghép tham lam trước), nhưng Hopcroft-Karp an toàn hơn.

HƯỚNG SUY NGHĨ

- Tô màu bàn cờ: ô (i, j) trắng nếu i + j chẵn. Hai ô kề nhau luôn khác màu, nên đồ thị "ô trống – ô trống kề" là **hai phía**.
- Mỗi quân domino là một cạnh của bộ ghép. Đáp số là kích thước bộ ghép cực đại.

BÀI 16.3	CAMERA GIÁM SÁT	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`CAMERA.CPP`	`CAMERA.INP`	`CAMERA.OUT`	1 giây · 256 MB

Kiến thức: *Định lí König, dựng phủ đỉnh nhỏ nhất (Tập 1, Bài 16).*

Nhà kho là lưới R × C ô. Có K ô **quan trọng** cần được giám sát. Mỗi camera có thể lắp ở đầu một **hàng** (giám sát mọi ô của hàng đó) hoặc ở đầu một **cột** (giám sát mọi ô của cột đó).

Hãy lắp **ít camera nhất** sao cho mọi ô quan trọng đều được giám sát, và chỉ ra cách lắp.

Dữ liệu vào

Dòng đầu chứa R, C, K. Mỗi dòng trong K dòng tiếp theo chứa hai số r, c là một ô quan trọng (hàng r, cột c); các ô đôi một khác nhau.

Kết quả

Dòng đầu là số camera t. Mỗi dòng trong t dòng tiếp theo có dạng `H r` (camera giám sát hàng r) hoặc `C c` (camera giám sát cột c). Nếu có nhiều cách, in cách bất kì.

Ràng buộc

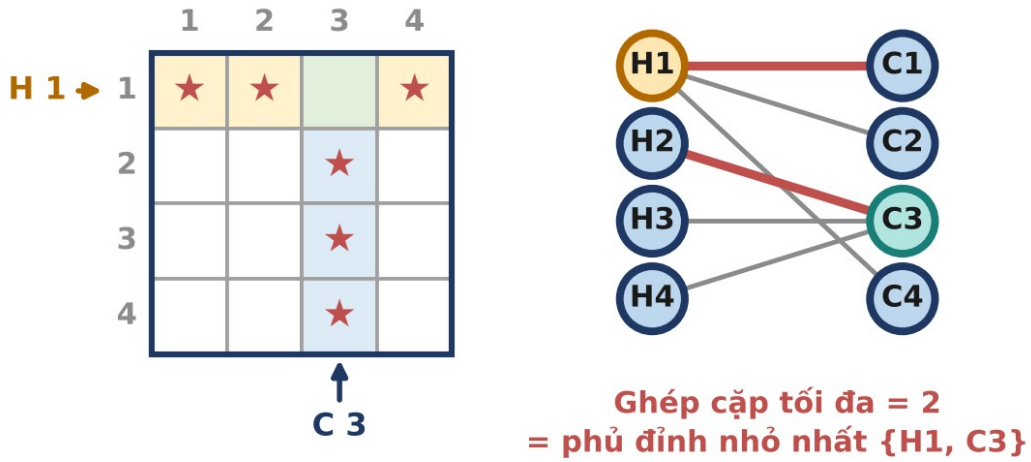
$1 \le R, C \le 1000$; $1 \le K \le 10^5$.

Subtask	Điểm	Ràng buộc thêm
1	20%	$R, C \le 8$.
2	30%	Chỉ cần in đúng dòng đầu tiên (số camera).
3	50%	Không có ràng buộc gì thêm.

Ví dụ

CAMERA.INP	CAMERA.OUT	Giải thích

`4 4 6`
`1 1`
`1 2`
`1 4`
`2 3`
`3 3`
`4 3`
	`2`
	`H 1`
	`C 3`
		Camera ở hàng 1 giám sát (1, 1), (1, 2), (1, 4); camera ở cột 3 giám sát (2, 3), (3, 3), (4, 3). Không thể chỉ dùng 1 camera (Hình 16.3).



Hình 16.3. *Mỗi ô quan trọng (r, c) là một cạnh nối hàng r với cột c; chọn camera là chọn đỉnh phủ mọi cạnh.*

CHÚ Ý

Bài có nhiều đáp án; chương trình chấm kiểm tra số camera là ít nhất và mọi ô quan trọng đều được giám sát.

HƯỚNG SUY NGHĨ

- Đồ thị hai phía: bên trái là các hàng, bên phải là các cột, mỗi ô quan trọng là một cạnh. Cần chọn ít đỉnh nhất phủ mọi cạnh: **phủ đỉnh nhỏ nhất**.
- Theo König, số camera ít nhất bằng kích thước bộ ghép cực đại.
- Dựng đáp án: từ mọi hàng chưa ghép, đi theo đường đan xen (hàng → cột bằng cạnh bất kì, cột → hàng bằng cạnh ghép), đánh dấu các đỉnh đi tới. Chọn các hàng **không** được đánh dấu và các cột **được** đánh dấu.

BÀI 17 Luồng cực đại và lát cắt nhỏ nhất

ÔN NHANH KIẾN THỨC · Tập 1, Bài 17

- **Luồng cực đại**: lặp lại việc tìm đường tăng từ s tới t trong **đồ thị thặng dư** (các cung còn sức chứa và các cung ngược), đẩy thêm luồng dọc đường đó. Edmonds-Karp dùng BFS: $O(V \cdot E^2)$; Dinic thường nhanh hơn nhiều.
- **Định lí luồng cực đại – lát cắt nhỏ nhất**: giá trị luồng cực đại bằng dung lượng lát cắt nhỏ nhất. Sau khi chạy xong, các đỉnh tới được từ s trong đồ thị thặng dư tạo thành phía S của một lát cắt nhỏ nhất.
- Với sức chứa 1, luồng cực đại là số đường đi **không chung cung** nhiều nhất.
- Mạng **hai phía có sức chứa**: s → trái (giới hạn của mỗi đỉnh trái), trái → phải (sức chứa 1), phải → t (yêu cầu của mỗi đỉnh phải).

Ba bài tập của bài học này

Bài	Tên bài	Mức độ	Kĩ năng rèn luyện
17.1	**ĐƯỜNG ỐNG**	★☆☆ CƠ BẢN	Cài đặt luồng cực đại và trích ra lát cắt nhỏ nhất.
17.2	**LỘ TRÌNH RỜI NHAU**	★★☆ TRUNG BÌNH	Không chỉ đếm mà còn truy vết các đường đi từ luồng.
17.3	**PHÂN CÔNG TRỰC NHẬT**	★★★ NÂNG CAO	Dựng mạng luồng từ các giới hạn "nhiều nhất" và "đúng bằng".

BÀI 17.1 ĐƯỜNG ỐNG ★☆☆ CƠ BẢN

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`DUONGONG.CPP`	`DUONGONG.INP`	`DUONGONG.OUT`	1 giây · 256 MB

Kiến thức: *Luồng cực đại, lát cắt nhỏ nhất từ BFS lần cuối (Tập 1, Bài 17).*

Hệ thống cấp nước có n trạm, trạm 1 là nhà máy nước và trạm n là khu dân cư. Có m đường ống **một chiều**, ống thứ i dẫn nước từ trạm u_i tới trạm v_i với lưu lượng tối đa c_i (m^3 mỗi giờ). Nước có thể chia nhánh và gộp lại tùy ý ở các trạm.

Hãy tính lưu lượng nước lớn nhất có thể dẫn từ nhà máy tới khu dân cư. Ngoài ra, công ti muốn biết những đường ống nào là "nút cổ chai": chia các trạm thành hai nhóm, nhóm S chứa trạm 1 và nhóm T chứa trạm n; **dung lượng** của cách chia là tổng lưu lượng tối đa của các ống đi từ S sang T. Trong các cách chia có dung lượng nhỏ nhất, chọn cách có **S ít trạm nhất** (cách này là duy nhất), và liệt kê các ống đi từ S sang T.

Dữ liệu vào

Dòng đầu chứa n, m. Dòng thứ i trong m dòng tiếp theo chứa ba số u, v, c (u ≠ v) mô tả ống thứ i.

Kết quả

Dòng 1: lưu lượng lớn nhất F. Dòng 2: số ống k trên lát cắt đã chọn. Dòng 3: số hiệu k ống đó theo thứ tự tăng dần.

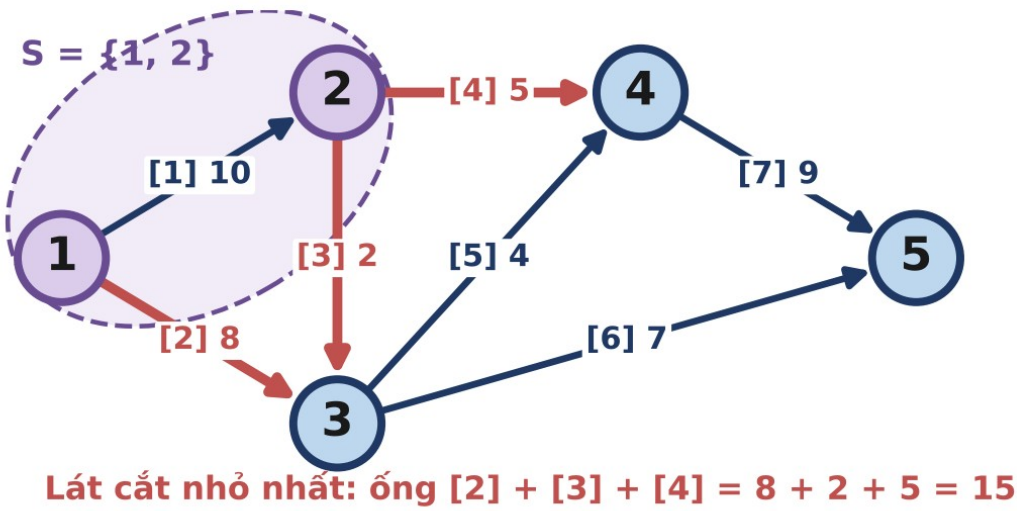
Ràng buộc

$2 \le n \le 500$; $1 \le m \le 5000$; $1 \le c \le 10^9$.

Subtask	Điểm	Ràng buộc thêm
1	40%	Chỉ cần in đúng dòng đầu tiên (giá trị F).
2	60%	Không có ràng buộc gì thêm.

Ví dụ

DUONGONG.INP	DUONGONG.OUT	Giải thích
5 7 1 2 10 1 3 8 2 3 2 2 4 5 3 4 4 3 5 7 4 5 9	15 3 2 3 4	Luồng cực đại bằng 15. Lát cắt S = {1, 2} gồm ba ống [2] 1 → 3, [3] 2 → 3, [4] 2 → 4, dung lượng 8 + 2 + 5 = 15 (Hình 17.1). Lát cắt S = {1} có dung lượng 10 + 8 = 18, lớn hơn.



Hình 17.1. *Nhãn [i] c: ống số i, lưu lượng tối đa c. Ống ngược chiều từ T về S không tính vào lát cắt.*

CHÚ Ý

Lưu lượng có thể tới $5 \cdot 10^{12}$: dùng `long long`.

Chỉ tính các ống đi **từ S sang T**; ống đi từ T về S không làm giảm dung lượng.

HƯỚNG SUY NGHĨ

- Chạy Edmonds-Karp hoặc Dinic. Sau khi xong, BFS một lần nữa từ trạm 1 trên đồ thị thặng dư (chỉ đi theo cung còn sức chứa dương). Tập đỉnh tới được chính là S có ít trạm nhất.
- Mọi ống từ S sang T khi đó đều đã được dùng hết công suất.

BÀI 17.2	LỘ TRÌNH RỜI NHAU	★★☆ TRUNG BÌNH	
Chương trình **ROINHAU.CPP**	Dữ liệu vào **ROINHAU.INP**	Kết quả **ROINHAU.OUT**	Giới hạn **1 giây · 256 MB**

Kiến thức: *Luồng với sức chứa 1, phân tích luồng thành đường đi (Tập 1, Bài 17).*

Thành phố có n nút giao và m con đường **một chiều**. Để tổ chức cuộc đua xe đạp, ban tổ chức cần chọn nhiều nhất các **lộ trình** từ nút 1 tới nút n sao cho **không có con đường nào thuộc hai lộ trình** (hai lộ trình được phép đi qua cùng một nút giao). Mỗi lộ trình không đi qua một nút hai lần.

Hãy tìm số lộ trình nhiều nhất và chỉ ra các lộ trình đó.

Dữ liệu vào

Dòng đầu chứa n, m. Mỗi dòng trong m dòng tiếp theo chứa hai số u, v ($u \neq v$) mô tả một con đường một chiều từ u tới v. Có thể có nhiều con đường cùng chiều nối một cặp nút.

Kết quả

Dòng đầu là số lộ trình k. Mỗi dòng trong k dòng tiếp theo mô tả một lộ trình: số nút L trên lộ trình, tiếp theo là L nút theo thứ tự đi qua (bắt đầu bằng 1, kết thúc bằng n). Nếu có nhiều cách, in cách bất kì.

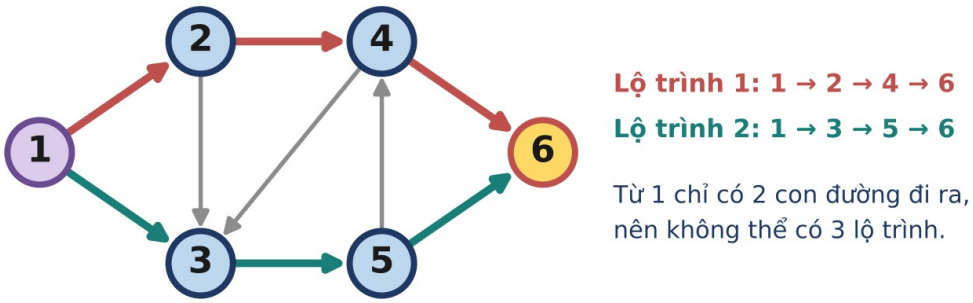
Ràng buộc

$2 \le n \le 1000$; $0 \le m \le 10^4$.

Subtask	Điểm	Ràng buộc thêm
1	40%	Chỉ cần in đúng dòng đầu tiên (số lộ trình k).
2	60%	Không có ràng buộc gì thêm.

Ví dụ

ROINHAU.INP	ROINHAU.OUT	Giải thích
6 9 1 2 1 3 2 3 2 4 3 5 4 3 4 6 5 6 5 4	2 4 1 2 4 6 4 1 3 5 6	Hai lộ trình 1 → 2 → 4 → 6 và 1 → 3 → 5 → 6 (Hình 17.2). Không thể có 3 lộ trình vì nút 1 chỉ có 2 con đường đi ra.



Hình 17.2. *Hai lộ trình không chung con đường nào.*

CHÚ Ý

Chọn tham lam "lộ trình ngắn nhất trước rồi xóa các con đường của nó" có thể không tối ưu; cần cung ngược của đồ thị thặng dư để "sửa" các lựa chọn trước.

Bài có nhiều đáp án; chương trình chấm kiểm tra các lộ trình của em.

HƯỚNG SUY NGHĨ

- Mỗi con đường là một cung sức chứa 1. Số lộ trình nhiều nhất là giá trị luồng cực đại từ 1 tới n.
- Truy vết: lặp k lần, xuất phát từ nút 1, đi theo một cung **có luồng** chưa dùng tới, đánh dấu đã dùng, cho tới khi tới n. Nếu lặp lại một nút thì cắt bỏ đoạn vòng vừa đi để lộ trình không qua một nút hai lần.

BÀI 17.3	PHÂN CÔNG TRỰC NHẬT	★★★ NÂNG CAO

Chương trình	Dữ liệu vào	Kết quả	Giới hạn
`TRUCNHAT.CPP`	`TRUCNHAT.INP`	`TRUCNHAT.OUT`	1 giây · 256 MB

Kiến thức: *Mạng luồng hai phía có sức chứa (Tập 1, Bài 17).*

Lớp có n học sinh và cần phân công trực nhật trong d ngày. Ngày thứ j cần **đúng** b_j học sinh trực. Học sinh thứ i trực **nhiều nhất** a_i ngày, và chỉ có thể trực vào một số ngày nhất định (do lịch học thêm). Mỗi học sinh trực một ngày nhiều nhất một lần.

Hãy lập một bảng phân công thỏa mãn mọi yêu cầu, hoặc cho biết không thể.

Dữ liệu vào

Dòng đầu chứa n, d. Dòng thứ hai chứa n số $a_1, ..., a_n$. Dòng thứ ba chứa d số $b_1, ..., b_d$. Dòng thứ i trong n dòng cuối bắt đầu bằng k_i, sau đó là k_i ngày (đôi một khác nhau) mà học sinh i có thể trực.

Kết quả

Nếu không thể, in `NO`. Ngược lại in `YES`, tiếp theo là d dòng: dòng thứ j liệt kê b_j học sinh trực ngày j (dòng trống nếu $b_j = 0$). Nếu có nhiều cách, in cách bất kì.

Ràng buộc

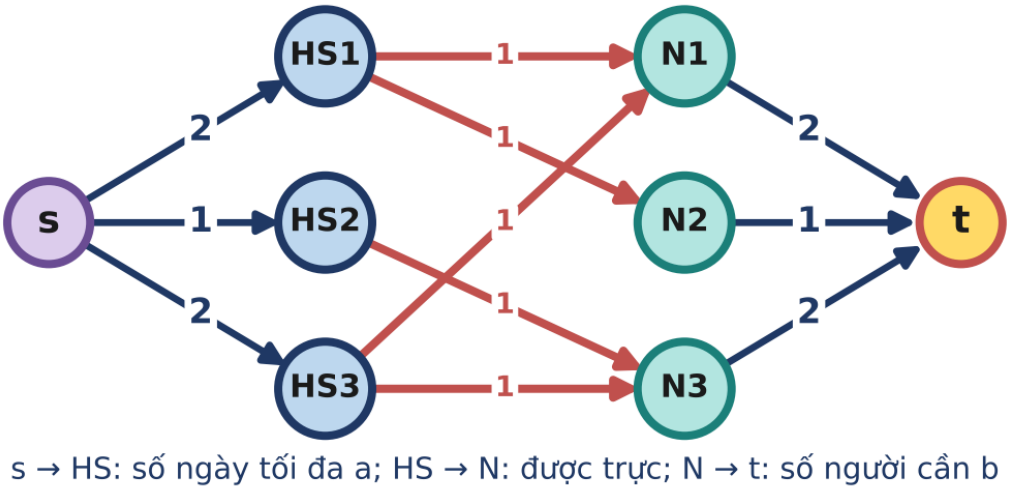
$1 \le n, d \le 200$; $0 \le a_i \le d$; $0 \le b_j \le n$.

Subtask	Điểm	Ràng buộc thêm
1	30%	$n, d \le 8$.
2	70%	Không có ràng buộc gì thêm.

Ví dụ

TRUCNHAT.INP	TRUCNHAT.OUT	Giải thích
3 3 2 1 2 2 1 2 2 1 2 1 3 2 1 3	YES 1 3 1 2 3	Ngày 1: học sinh 1 và 3; ngày 2: học sinh 1; ngày 3: học sinh 2 và 3. Học sinh 1 và 3 trực 2 ngày, học sinh 2 trực 1 ngày, đúng giới hạn (Hình 17.3).

TRUCNHAT.INP	TRUCNHAT.OUT	Giải thích
2 2 1 2 2 1 1 1 1 1	NO	Không học sinh nào trực được ngày 2.



Hình 17.3. *Mạng luồng của ví dụ 1; cung đỏ là các cặp (học sinh, ngày) được chọn.*

CHÚ Ý

Tổng các b_j phải bằng đúng giá trị luồng cực đại; nếu luồng nhỏ hơn thì in NO.

Bài có nhiều đáp án; chương trình chấm kiểm tra bảng phân công của em.

HƯỚNG SUY NGHĨ

- Mạng gồm nguồn s, n đỉnh học sinh, d đỉnh ngày và đích t. Cung s → học sinh i sức chứa a_i; học sinh i → ngày j sức chứa 1 nếu i trực được ngày j; ngày j → t sức chứa b_j.
- Có phân công khi và chỉ khi luồng cực đại bằng $b_1 + ... + b_d$. Các cung học sinh → ngày có luồng 1 cho bảng phân công.

PHỤ LỤC A. KIỂM TRA TRƯỚC KHI NỘP BÀI

Rất nhiều điểm bị mất không phải vì thuật toán sai mà vì những lỗi nhỏ. Trước khi nộp, hãy rà lại danh sách dưới đây; các lỗi này đã xuất hiện trong khung CHÚ Ý của nhiều bài tập.

ĐỌC GHI DỮ LIỆU

- ☐ Tên tệp đúng chính tả, viết hoa đúng như đề (BATTAY.INP, BATTAY.OUT).
- ☐ Đọc nhanh (`scanf` hoặc `ios::sync_with_stdio(false)`) khi dữ liệu có từ 10^5 số trở lên.
- ☐ In đúng khuôn dạng: số dòng, thứ tự, dấu cách; in −1 / NO đúng như yêu cầu.

KIỂU DỮ LIỆU VÀ GIỚI HẠN

- ☐ Tổng, tích, khoảng cách có thể vượt $2 \cdot 10^9$? Dùng `long long`.
- ☐ Giá trị "vô cùng" đủ lớn nhưng cộng thêm không bị tràn số.
- ☐ Mảng khai báo đủ kích thước (đồ thị vô hướng: danh sách kề có 2m phần tử).
- ☐ Ước lượng bộ nhớ khi khai báo mảng hai chiều hoặc bảng nhảy nhị phân.

ĐỒ THỊ

- ☐ Đồ thị có thể **không liên thông**? Đã duyệt từ mọi đỉnh chưa thăm chưa?
- ☐ Có **cạnh lặp**, **khuyên** không? Chương trình xử lí đúng chưa?
- ☐ Đỉnh xuất phát trùng đỉnh đích, n = 1, m = 0 đã thử chưa?
- ☐ DFS đệ quy có thể sâu tới 10^5–10^6 tầng? Hãy chuyển sang ngăn xếp tự quản lí.
- ☐ Với nhiều bộ dữ liệu trong một tệp, đã **xóa sạch** mảng và danh sách kề giữa các bộ chưa?

KIỂM TRA CUỐI CÙNG

- ☐ Chạy lại toàn bộ ví dụ trong đề.
- ☐ Tự tạo một test lớn nhất để đo thời gian.
- ☐ Đối chiếu với lời giải ngây thơ trên test ngẫu nhiên nhỏ (Phụ lục B).

PHỤ LỤC B. TỰ KIỂM TRA BẰNG CÁCH ĐỐI CHIẾU

Mọi lời giải mẫu của bộ sách đều được kiểm tra bằng kĩ thuật **đối chiếu** (còn gọi là "chấm trâu"). Ý tưởng rất đơn giản: viết một chương trình **ngây thơ** (thường là vét cạn, chậm nhưng chắc chắn đúng), sinh thật nhiều bộ dữ liệu **nhỏ** ngẫu nhiên, chạy cả hai chương trình và so sánh kết quả. Chỉ cần một bộ cho kết quả khác nhau là em đã có trong tay một test nhỏ làm lộ lỗi sai.

Ba chương trình cần có:

- `GEN.CPP`: sinh ngẫu nhiên một bộ dữ liệu nhỏ và in ra màn hình.
- `TRAU.CPP`: lời giải ngây thơ.
- `SOL.CPP`: lời giải cần kiểm tra.

Lưu ý: khi đối chiếu, hãy đặt các chương trình ở thư mục **không có** tệp `.INP` của bài, để kiểu đọc "có tệp thì đọc tệp" chuyển sang đọc từ bàn phím (luồng chuẩn).

GEN.CPP: sinh một đồ thị vô hướng ngẫu nhiên nhỏ (n ≤ 6), không có cạnh lặp

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));            // hạt giống lấy từ tham số dòng lệnh
    int n = rng() % 6 + 1;
    vector<pair<int,int>> e;
    for (int u = 1; u <= n; u++)
        for (int v = u + 1; v <= n; v++)
            if (rng() % 2) e.push_back({u, v}); // mỗi cặp có cạnh với xác suất 1/2
    shuffle(e.begin(), e.end(), rng);
    printf("%d %d\n", n, (int)e.size());
    for (auto [u, v] : e) printf("%d %d\n", u, v);
    return 0;
}
```

DOICHIEU.CPP: chạy 1000 test, dừng ở test đầu tiên có kết quả khác nhau

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    for (int t = 1; t <= 1000; t++) {
        string seed = to_string(t);
        system(("gen " + seed + " > test.inp").c_str());
        system("sol < test.inp > sol.out");
        system("trau < test.inp > trau.out");
        // Windows dùng "fc", Linux dùng "cmp -s"
        if (system("fc sol.out trau.out > nul") != 0) {
            printf("Sai o test %d, xem tep test.inp\n", t);
            return 0;
        }
    }
    printf("Ca 1000 test deu khop!\n");
    return 0;
}
```

CHÚ Ý

Với bài có nhiều đáp án đúng (14.3, 15.2, 15.3, 16.3, 17.2, 17.3), không thể so sánh trực tiếp hai tệp kết quả. Hãy viết một chương trình **kiểm tra**: đọc dữ liệu vào và kết quả của em, rồi kiểm tra kết quả có hợp lệ không và có tối ưu bằng kết quả của chương trình trâu không.

Test ngẫu nhiên nên **nhỏ** (vài đỉnh, vài cạnh): khi phát hiện sai, em sẽ chạy tay được ngay trên test đó.

PHỤ LỤC C. BẢNG THEO DÕI TIẾN ĐỘ

Sau mỗi lần làm bài, hãy ghi lại ngày làm, số điểm (theo subtask) và một dòng ghi chú về điều học được hoặc lỗi đã mắc. Đọc lại cột ghi chú trước mỗi kì thi.

Chương 1. LÀM QUEN VỚI ĐỒ THỊ

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
1.1	BẮT TAY	★☆☆			
1.2	NGƯỜI NỔI TIẾNG	★★☆			
1.3	TAM GIÁC BẠN BÈ	★★★			
2.1	CHUYỂN ĐỔI	★☆☆			
2.2	BẠN CHUNG	★★☆			
2.3	BẢN ĐỒ KHO BÁU	★★★			
3.1	ĐẾM ĐẢO	★☆☆			
3.2	CÂY HAY KHÔNG	★★☆			
3.3	MẠNG LƯỚI DỰ PHÒNG	★★★			
4.1	MÃ ĐI TUẦN	★☆☆			
4.2	CHẠY TRỐN ĐÁM CHÁY	★★☆			
4.3	CHIA ĐỘI	★★★			

Chương 2. ĐƯỜNG ĐI NGẮN NHẤT

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
5.1	ĐƯỜNG TỚI TRƯỜNG	★☆☆			
5.2	NÚT GIAO QUAN TRỌNG	★★☆			
5.3	XE BUÝT ĐÊM	★★★			
6.1	ĐỔI TIỀN	★☆☆			
6.2	CHUYẾN BAY GIÁ RẺ	★★☆			
6.3	LỊCH THI ĐẤU	★★★			
7.1	THỦ ĐÔ MỚI	★☆☆			
7.2	XẾP HẠNG KÌ THỦ	★★☆			

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
7.3	XE TẢI QUÁ KHỔ	★★★			
8.1	ĐÈN GIAO THÔNG	★☆☆			
8.2	PHÁ TƯỜNG	★★☆			
8.3	ROBOT XOAY	★★★			

Chương 3. CÂY VÀ CÂY KHUNG

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
9.1	MẠNG BẠN BÈ	★☆☆			
9.2	PHÁ CẦU	★★☆			
9.3	HAI PHE	★★★			
10.1	NỐI MẠNG TRƯỜNG HỌC	★☆☆			
10.2	ĐƯỜNG ÍT DỐC	★★☆			
10.3	GIẾNG VÀ ỐNG	★★★			
11.1	HỌ HÀNG	★☆☆			
11.2	CÁP QUANG YẾU NHẤT	★★☆			
11.3	NHÂN VIÊN CẤP K	★★★			

Chương 4. CẤU TRÚC CỦA ĐỒ THỊ

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
12.1	XẾP HẠNG GIẢI ĐẤU	★☆☆			
12.2	DỰ ÁN XÂY NHÀ	★★☆			
12.3	ĐẾM LỘ TRÌNH	★★★			
13.1	ĐƯỜNG HUYẾT MẠCH	★☆☆			
13.2	TRẠM BIẾN ÁP	★★☆			
13.3	CÁP DỰ PHÒNG	★★★			
14.1	LAN TRUYỀN TIN ĐỒN	★☆☆			
14.2	KHO BÁU THỊ TRẤN	★★☆			
14.3	HỘI ĐỒNG TUYỂN CHỌN	★★★			

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
15.1	NÉT VẼ	★☆☆			
15.2	GHÉP DOMINO	★★☆			
15.3	MẬT KHẨU KÉT SẮT	★★★			

Chương 5. GHÉP CẶP VÀ LUỒNG

Bài	Tên bài	Mức độ	Ngày làm	Điểm	Ghi chú
16.1	XẾP PHÒNG THI	★☆☆			
16.2	ĐẶT DOMINO	★★☆			
16.3	CAMERA GIÁM SÁT	★★★			
17.1	ĐƯỜNG ỐNG	★☆☆			
17.2	LỘ TRÌNH RỜI NHAU	★★☆			
17.3	PHÂN CÔNG TRỰC NHẬT	★★★			